The Australian National University

School of Computing

JHQ-GPU

Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

Research Provenance and Tool Disclosure

This thesis presents the JHQ-GPU project undertaken by Rui Luo for COMP8800 — Advanced Computing Research Project under the supervision of Dr Mengxuan Zhang.

The underlying JHQ representation and primary/residual hierarchy are attributed to Jiabao Han, Mengxuan Zhang and Goce Trajcevski [15]. IVF, product quantisation, existing GPU selection/merge techniques and baseline software retain their cited authorship. Rui Luo and Mengxuan Zhang are coauthors of the associated project manuscript [21]; their author order does not establish individual task allocation. The thesis expands the correctness arguments, measurements, supplementary checks and discussion of unsuccessful designs. Section 1.5 and table 1.1 identify the project's relationship to that manuscript and maps its technical contributions.

Generative-AI assistance. Project records document the following uses, identified by product name without inferring model versions: **OpenAI Codex** assisted with English/Chinese writing and revision, LaTeX, diagrams, analysis scripts, source/log inspection, offline numerical checks and PDF compilation. **ChatGPT** supplied frameworks and chapter reviews concerning evidence, scope and experimental design. **Anthropic Claude** assisted with implementation, experiment automation and review; supplementary records cover construction, representation/precision checks, budget traces, strict selection and small-batch baselines. The joint manuscript separately discloses Claude/ChatGPT assistance with implementation, harnesses, figures and writing, retaining that manuscript's versioned scope.

Traceability and responsibility. Source histories, run records and analysis ledgers establish artifact provenance. `scripts/` retains analysis code; `assessment/` records accepted and corrected review recommendations; the code repository's `experiments/thesis_evidence/` retains supplementary experiment records. Appendices A and B identify the experimental contracts and supporting evidence. Bibliographic citations acknowledge inherited methods. These records describe the retained workflow, rather than a complete interaction export or a signed personal attestation. Responsibility for the submitted text, attribution and interpretation remains with the thesis author.

Abstract

Efficient GPU approximate nearest-neighbour search requires compact representations, efficient scoring and selection, and suitable refinement effort. This thesis presents JHQ-GPU, a GPU implementation and evaluation of JHQ's inherited Cartesian primary/residual hierarchy for high-dimensional squared-L2 search.

The system combines GPU training and batched index construction with representation-aware query execution. Cartesian structure decomposes each 256-entry primary distance table into two 16-entry tables without changing the real-arithmetic represented distance. Packed access, lane-cooperative refinement and workload-dependent grouped scanning complement this design. Selection and shared-bound arguments specify correctness conditions, backed by a strict computed-score reference. A workload-level policy selects a refinement multiplier through unlabelled output agreement with a larger reference budget; its analysis separates sampled agreement from GT recall.

Archived measurements on an RTX 5090 cover six embedding datasets. The largest tested index searches 17.78 million vectors at 20.44 GiB resident device memory. Two-group tables lead in 22/24 configurations. Integrated scheduling improves geometric-mean throughput by 2.727× at batch 8 against the lane-refinement reference, with much smaller full-batch gains. At recall 0.97 and batch 1,000, interpolated comparisons place JHQ first in three E1 datasets and near parity with IVF-RaBitQ on arXiv. OpenAI E1 uses TF32. FP32 interventions under a TF32-trained cache retain numerical advantages, with cross-session comparisons distinguished from contemporaneous anchors; neither experiment fully retrains FP32 models. At batch 8 and the same recall, CAGRA fp32 instead leads on all four datasets; batch-32 rankings also depend on the finite tuning grid.

Full-pool calibration meets a 10^{-3} held-out recall-loss target relative to the best tested budget in all twelve cells, with maximum observed loss 0.0006. Smaller $S = 128$ samples have up to 40/64 threshold crossings. These fixed-pool measurements provide no population recall guarantee. Direct reuse measurements support net savings on suitable workloads. Negative designs show that arithmetic reuse and lower reconstruction error need not improve the search frontier. The findings support joint evaluation of representation, execution and effort under explicit quality, timing and capacity conditions.

Acknowledgements

I thank Dr Mengxuan Zhang for supervising this research project. The work builds on the JHQ research of Jiabao Han, Mengxuan Zhang and Goce Trajcevski, and on the open-source ANN and GPU software cited throughout the thesis. Their publications and implementations provided the foundations and comparison systems for this study.

The use of generative-AI tools in the project and thesis preparation is documented in the Research Provenance and Tool Disclosure.

Contents

Research Provenance and Tool Disclosure **i**

Abstract **ii**

Acknowledgements **iii**

Notation and Abbreviations **viii**

1 Introduction **1**
1.1 Context and Practical Objectives . . . 1
1.2 Problem Definition and Scope . . . 1
1.3 Three Linked Technical Challenges . . . 2
1.4 Research Questions and Falsifiable Claims . . . 3
1.5 Contributions and Research Ownership . . . 4
1.6 Thesis Roadmap . . . 4

2 Background and Critical Related Work **6**
2.1 ANN Objective and Metrics . . . 6
2.2 IVF, PQ, ADC and Refinement . . . 6
2.3 JHQ Representation and Assumptions . . . 8
2.4 GPU Execution Concepts . . . 9
2.5 Graph Search on GPUs . . . 9
2.6 Quantised Search and Fast Tables . . . 10
2.7 Research Gap and Design Choices . . . 11

3 Research Design and Measurement Methodology **13**
3.1 Research Questions and Comparisons . . . 13
3.2 Datasets, Queries, Ground Truth and Splits . . . 14
3.3 Timing, Variance and Memory Accounting . . . 15
3.4 Baselines and Frontier Construction . . . 16
3.5 Versioned Claim-to-Evidence Ledger . . . 17
3.6 Evidence Strength and Threat Controls . . . 18

4 System Architecture and GPU Index Construction **19**
4.1 End-to-End Build and Search Boundaries . . . 19
4.2 Training, Rotation and Codebook Generation . . . 20
4.3 IVF Assignment, IDs and Physical Memory Layout . . . 22
4.4 Resident Storage, Workspace and Index Lifecycle . . . 24
4.5 Construction Cost and Tested Scale . . . 26
4.6 Index Representation and Safety Contracts . . . 27

5 Representation-Aware GPU Search Execution **30**
5.1 Query Path and Four Distinct Ranking Objects . . . 30
5.2 Exact Cartesian LUT Factorisation . . . 31
5.3 Packed Codes and Candidate-Parallel Memory Access . . . 33
5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . 35
5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . 38
5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . 41
5.7 IVF Probe Selection as an Independent Primitive . . . 43

6 Workload-Level Refinement-Budget Calibration **45**
6.1 Why Fixed Refinement Budgets Can Be Wasteful . . . 45

6.2 GT-Free Sampled Output Agreement . . . 46
6.3 Budget Search, Nestedness and Tie Safety . . . 48
6.4 Budget Baselines and Their Questions . . . 50
6.5 Hold-out Risk, Calibration Cost and Payback . . . 50
6.6 Policy Failure Modes and Applicability . . . 51

7 Experimental Evaluation and Research Findings 53
7.1 Final Experiment Manifest and Comparison Families . . . 53
7.2 Modern GPU ANN Recall–QPS Frontiers . . . 53
7.3 Cartesian LUT, Packed Loads and Lane Refinement . . . 58
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation . . . 60
7.5 Held-out Budget Quality, Throughput and Payback . . . 62
7.6 Batch Regimes, G Scheduling and Workload Skew . . . 68
7.7 Cold Construction, Resident Memory and Tested Scale . . . 70
7.8 Numerical Robustness, Anomalies and Recall Attribution . . . 74

8 Discussion, Limitations and Future Research 76
8.1 Three Cross-Layer Insights . . . 76
8.2 Two Deep Negative-Result Case Studies . . . 77
8.3 Threats to Internal and External Validity . . . 78
8.4 Testable Research Directions . . . 79
8.5 Conditional Deployment Recommendation . . . 80

9 Conclusion 81
9.1 Answers to the Four Research Questions . . . 81
9.2 Closing Principle and Outlook . . . 82

Bibliography 83

A Reproducibility and Experiment Manifest 86
A.1 Source, Environment and Trained States . . . 86
A.2 Measurement Contracts . . . 86
A.3 Budget Harness Contract . . . 87
A.4 Supplementary Evidence . . . 87
A.5 Strict Selection, Small Batches and Artifact Access . . . 88

B Supplementary Results and Correctness Checks 89
B.1 Complete Parameter Registry . . . 89
B.2 Correctness and Numerical Evidence . . . 90
B.3 Secondary Results and Evidence Status . . . 90
B.4 Construction Capacities and Precision Anchors . . . 91
B.5 Artifact Access . . . 92

List of Figures

1.1 Problem, mechanism and evidence . . . 3
2.1 Candidate and precision cascade . . . 7
3.1 Host and device timing boundaries . . . 15
4.1 Build and query architecture . . . 19
4.2 List-ordered physical index layout . . . 23
5.1 Query stages and selection boundaries . . . 30
5.2 Packed candidate-parallel access . . . 34
5.3 Local coverage and truncated pairwise merging . . . 37
5.4 Shared complete-key filtering . . . 39
5.5 Sixteen-lane residual refinement . . . 42
6.1 Work affected by the refinement budget . . . 45
6.2 Workload-level calibration and evaluation . . . 47
6.3 Nested and non-nested candidate pools . . . 49
7.1 E1 recall–throughput frontiers . . . 54
7.2 TF32 and FP32 precision frontiers . . . 56
7.3 Small-batch competitive regions . . . 57
7.4 Cartesian lookup-table grouping . . . 59
7.5 Packed access and refinement controls . . . 60
7.6 Held-out risk across sample sizes . . . 64
7.7 Finite-pool risk prediction and observation . . . 67
7.8 Direct cumulative calibrated-reuse time . . . 68
7.9 Batch-dependent execution regimes . . . 69
7.10 Recorded construction costs . . . 71
7.11 Resident-memory decomposition . . . 72
7.12 Large-dataset search frontiers . . . 73
8.1 Mailman and rank-oriented coding . . . 78

List of Tables

1.1 Technical contributions and evidence boundaries 4
2.1 Design alternatives and research gap 11
3.1 Research questions and experimental controls 13
3.2 Dataset and training manifest 14
3.3 Principal E1 tuning dimensions 16
4.1 Per-vector storage model 24
4.2 Construction and query object lifetimes 25
4.3 Analytical row-array occupancy by construction phase 26
4.4 Original-input FP64 recoding audit 28
4.5 Representation contracts and audit coverage 29
5.1 Query-stage selection contracts 31
5.2 Cartesian lookup-table trade-offs 33
6.1 Budget controls and their roles 50
7.1 Cross-system comparison at recall 0.97 55
7.2 FP32 sensitivity at matched configurations 56
7.3 Small-batch matched-recall ratios 58
7.4 Refinement gains at 128 probes 60
7.5 Strict computed-score selection checks 61
7.6 Probe-selection throughput ratios 62
7.7 Full-pool budget calibration 63
7.8 Sampled-budget held-out quality 64
7.9 Sampled-calibration economics 65
7.10 Direct costs for 500 reuse batches 67
7.11 Integrated execution and scheduling gains 69
7.12 Cold construction and resident memory 71
7.13 E1 allocation ranges 73
8.1 Threats to validity 79
A.1 Component and run provenance 86
B.1 Exact E1 configuration counts 89
B.2 Validation coverage 90
B.3 Independent construction-buffer capacities 91
B.4 Same-session precision anchors 91

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	Configured add, staged input, and independently allocated assignment rows
n_b, S_{stage}	Pipeline buffer count and byte budget per input/rotated array
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Calibration sample size and allowed disagreeing result slots
$C_\alpha, \mathcal{R}_\alpha$	Valid primary survivor set and returned ID set at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Scan buffer capacity and threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	Number of LUT entries per primary subspace
$\kappa(D, p)$	Packed complete-key encoding for bound publication
ℓ_{ref}	Number of cooperating lanes per refined candidate
D_{raw}	Squared distance between original vectors
D_{P}	Distance under the primary reconstruction
D_{H}	Refined ranking score: reconstructed squared distance plus a query-only constant

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation (distinct from the ADC conference)
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE single-precision storage and arithmetic, with multiplication mode stated separately
TF32	Tensor-core multiplication format with reduced input precision and FP32 accumulation
CV	Coefficient of variation of candidate counts

Chapter 1

Introduction

Searching a large embedding database requires decisions about both information and work: how compactly to represent each vector, which vectors to inspect, and how much precision to spend on them. A GPU makes many operations parallel, but it does not make these decisions free. This thesis studies their interaction in JHQ-GPU, a GPU implementation and experimental analysis of JHQ's primary/residual quantisation hierarchy.

1.1 Context and Practical Objectives

High-dimensional embeddings support similarity search over images, documents and other objects. In the tested workloads, a vector has 768–3,072 coordinates and the largest database contains 17.78 million vectors. Exhaustive distance computation exposes substantial arithmetic, while retaining full vectors consumes capacity before any index overhead. For example, the tested Stella FP32 array alone would occupy 67.81 GiB (section 7.7). Approximate nearest-neighbour search exchanges some exact neighbour recovery for reduced access and scoring work.

Modern GPU approaches offer several ways to make that exchange. Graph methods such as CAGRA avoid many vectors through navigation, while compressed IVF methods combine selective list access with compact distance computation [17, 28, 31]. They make a systems question unavoidable: where do representation, selection and memory movement place the cost at a required recall? A short code saves storage, but table construction, dependent lookups, candidate retention and repeated launch setup can determine whether it also saves query time.

JHQ provides a useful setting for studying this question [15]. Its compact primary representation screens candidates, and a residual representation refines a shortlist. This architecture exposes two related decisions: how to execute the broad screen efficiently, and how many of its survivors to refine. It also permits a useful experimental separation: execution can change while keeping the represented score fixed, and budget can change while reusing a trained index.

The practical objective is to improve the attainable combination of Recall@10, host-visible query throughput/latency, resident memory and construction cost. The evaluation includes cases where an alternative is faster or a proposed change fails. This makes the contribution an account of competitive operating conditions and their mechanisms, rather than a single speedup detached from quality and resource constraints.

1.2 Problem Definition and Scope

For a database $\mathcal{X} \subset \mathbb{R}^d$ and query q, the target is the top-k set under raw squared L2. The implemented system routes through IVF, ranks the visited vectors by a primary reconstructed score D_{P}, retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ candidates, and selects results using a residual-refined score D_{H}. Both scores differ from raw-vector distance. The refined value additionally contains a query-only constant, as defined in section 2.3. Exactness of a selection procedure therefore concerns its computed score and tie contract; final ANN quality remains an empirical question against GT.

The representation follows JHQ: a square orthogonal rotation, Cartesian primary codes and residuals relative to primary reconstruction. This thesis studies GPU training/encoding, physical placement, query execution and one workload-level refinement multiplier. The multiplier is selected from unlabelled query outputs against a larger-budget reference, then is intended to be reused under a stable workload. It is not a per-query learned controller or an absolute-recall certificate.

The main cross-system experiments use one RTX 5090, squared-L2 top-10 and batches of 1,000 queries on four datasets. Six datasets support component and budget studies, with separate large-index measurements. T5 adds cross-system batches 8/32 alongside internal scheduling ablations (section 7.2.5). The implemented compressed index resides on one GPU; CPU model preparation and host-query transfers remain part of its actual architecture. Multi-GPU distribution, online updates, host-backed residual paging and billion-scale deployment are outside the tested system.

1.3 Three Linked Technical Challenges

Figure 1.1 connects three challenges to the mechanisms and evidence used to address them.

First, the primary codebook has useful mathematical structure that a direct GPU port can overlook. Each eight-bit subspace code indexes 256 distances. Its Cartesian coordinates permit an exact 16+16 factorisation, but more aggressive splitting adds lookups and arithmetic. The challenge is to exploit separability without assuming that the smallest table gives the fastest scan. Candidate-parallel word access and residual lane cooperation also need different data mappings.

Second, a query's primary winners can be concentrated in any part of its routed pool. Parallel groups cannot safely retain only a fraction of the required global survivors. Exact local retention, a global merge and a shared complete-score bound provide a principled route, but add buffers, staging and synchronisation. A large batch can already supply enough blocks; an imbalanced workload can still benefit from more groups. The challenge couples correctness with a workload-dependent execution decision. Selecting IVF lists remains a separate primitive from retaining database candidates.

Third, a fixed survivor multiplier can repeatedly refine irrelevant vectors, while a small multiplier can remove a true neighbour too early. Agreement with a large-budget reference is available without GT, but agreement does not repair the reference's own errors or guarantee future-query recall. Choosing a sample and budget also consumes time. The challenge is to evaluate selection quality, calibration cost and reuse together. These are questions organised around an iterative project record; observed benefits are not presented as preregistered predictions.

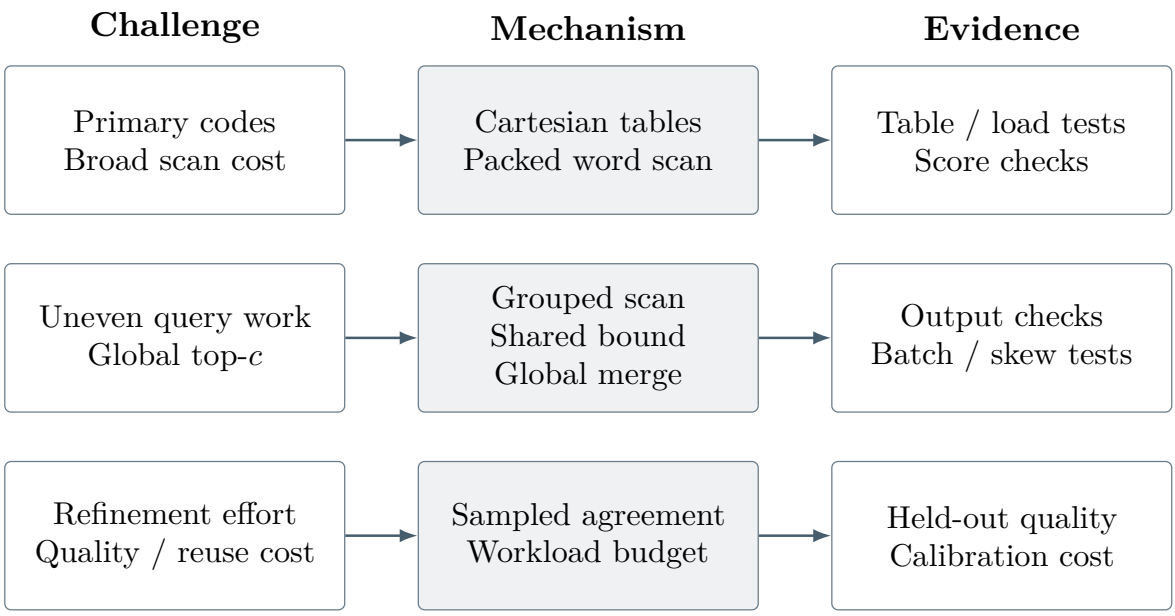


Figure 1.1. Problem, mechanism and evidence. The arrows organise research questions; they do not imply that independently measured speedups multiply or that an unlabelled agreement rule guarantees GT recall.

1.4 Research Questions and Falsifiable Claims

RQ1: Construction and capacity. How can JHQ's hierarchy be trained, encoded and arranged as a searchable GPU-resident index, and what construction cost and tested capacity result? The evidence is the physical layout, train/add records and resident-memory account (section 7.7). An unsuccessful build or an allocation beyond the device budget would limit the capacity claim; a resident sample alone does not establish construction peak.

RQ2: Correct and efficient execution. Which representation-aware scoring, retention and scheduling choices improve GPU search while respecting the specified score and tie semantics? Paired LUT, packed-load, refinement and scan experiments connect to reference checks and batch sensitivity (sections 7.3, 7.4 and 7.6). A non-tie selection error would challenge the correctness contract; consistent slowdowns would challenge the proposed execution choice.

RQ3: Budget quality and total cost. Can unlabelled workload agreement choose a useful survivor budget, and under what sampling and reuse conditions is calibration worthwhile? Disjoint query evaluation compares fixed budgets, sampled/FULL rules, enumeration and a GT-labelled oracle (section 7.5). Loss crossings challenge reliability; no positive saving after calibration challenges the net-cost benefit. The oracle is an evaluation control, not deployable label-free logic.

RQ4: Competitive operating regions. At which measured recall targets does the complete system compete with modern GPU ANN alternatives under a common host-to-host boundary? The answer uses independently tuned E1 frontiers and explicit memory definitions (section 7.2). Int8 graph competitors, high-recall reversals and absent coverage are part of the answer. The question does not seek a universal ranking across hardware, memory budgets or request sizes.

Chapter 3 turns these questions into controlled comparisons. Together they separate what was built, whether its execution is correct, what effort a workload requires, and whether the resulting system has useful operating regions.

1.5 Contributions and Research Ownership

Table 1.1 states three linked project contributions and their inherited foundations. The core contribution is the adaptation and evaluation of a particular representation–execution–budget combination. JHQ, Cartesian quantisation, IVF, bitonic/radix selection, merging and CUDA Graph execution retain their prior-art status.

Table 1.1. Technical contributions and their evidence boundaries.

	Inherited foundation	Project contribution	Evidence
C1	JHQ rotation, codebooks and hierarchy	GPU training/encoding, bounded batch construction and list-ordered packed representation	Search of 17.78M vectors at 20.44 GiB resident; repeated train+add 10.54–10.65 s; sampled construction peak 22.58 GiB
C2	Cartesian additivity and established GPU primitives	Integrated factorised scoring, complete-key grouped retention/bounds, lane refinement and execution scheduling	Two-group LUTs lead in 22/24 cells; scheduled execution 2.727× at batch 8; explicit correctness scope
C3	Candidate expansion and output-agreement ideas	Workload-level budget rule, conditional selection analysis, held-out controls and cost/risk evaluation	FULL meets the relative-loss target in 12/12 cells; smaller samples expose risk; four cells directly measure calibrated reuse

The results make the claims concrete without making them universal. On the full-batch E1 grid near recall 0.97, JHQ leads in three datasets and is near parity with RaBitQ on arXiv under the interpolated comparison. Other recall regions favour competitors. FULL's maximum observed held-out loss is 0.0006 relative to the best tested arm, while $S = 128$ reaches 40/64 loss crossings in one cell. These are empirical quality results; ten FULL cells retain conditional nestedness coverage, while all twelve FULL finite-grid choices now match enumeration (section 7.5.5). The OpenAI E1 results use TF32 GEMMs; a separate FP32 sensitivity retains smaller favourable margins (section 7.2.3).

This COMP8800 project is presented by Rui Luo under Dr Mengxuan Zhang's supervision. The coauthored ADC manuscript names Rui Luo and Mengxuan Zhang and shares the JHQ-GPU implementation and measurement artifacts [21]. The thesis expands the correctness contracts, newer experiment analysis and treatment of failed designs; historical manuscript numbers retain their own versions. Technical contribution labels identify the project scope rather than assigning sole ownership of collaborative work. Personal contribution allocation and AI assistance are documented separately in the declaration and contribution record. AI-assisted implementation, analysis and writing require explicit disclosure and author review; a source ledger does not substitute for that responsibility.

1.6 Thesis Roadmap

Chapters 2 and 3 establish the representation, alternatives and measurement design. Chapter 4 follows training and physical index construction; Chapter 5 develops GPU scoring and selection contracts. Chapter 6 analyses workload-level refinement calibration. Chapter 7 evaluates construction, execution, budget quality and competitive operating regions. Chapter

8 interprets those findings through negative results, validity limits and testable extensions. Chapter 9 answers the research questions and states the resulting systems principle. Appendix A maps claims to implementation and measurement artifacts.

Chapter 2

Background and Critical Related Work

An ANN system combines a candidate-access strategy, a representation and a selection procedure. Their costs interact: a compact code can require an expensive scorer, and an accurate scorer cannot recover a vector excluded earlier. This chapter establishes the technical vocabulary for those interactions, identifies what is inherited from JHQ, and compares the alternatives that motivate the GPU execution and budget questions.

2.1 ANN Objective and Metrics

Let $\mathcal{X} = \{x_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$ be a database with stable identifiers. For query q, the exact task selects the k smallest raw-vector distances

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

Ties require a specified ordering or a set of equally valid answers. This thesis evaluates the stored GT top-10 set, with its fixed tie choices. ANN reduces the cost of accessing and scoring vectors while permitting missed members of that set. The primary quality measure is Recall@10 (equation (3.1)); it is an empirical set-recovery measure. A high measured recall does not establish a worst-case multiplicative distance guarantee.

There are four relevant cost axes. Query throughput counts queries completed per second at a stated batch and timing boundary. Batch latency counts elapsed time for that call; throughput alone supplies no concurrent-service tail guarantee. Construction cost includes the specified training and encoding stages. Storage distinguishes persistent bytes per vector from model, workspace, context and temporary allocation. Chapter 3 defines the actual measurement boundaries. A method may improve one axis while worsening another; equal recall, equal memory and equal construction effort are different comparisons.

Tuning produces a finite set of operating points. Its Pareto frontier keeps points whose recall and throughput are not simultaneously improved by another measured configuration. At a target recall, the fastest qualifying point is an available configuration, while interpolation estimates between neighbouring points. An uncovered region supplies no evidence of either dominance or impossibility. These distinctions are especially relevant when the compared systems use different code precisions and graph/index sizes. The detailed frontier estimators and their endpoint tolerance are fixed in section 3.4.

2.2 IVF, PQ, ADC and Refinement

IVF partitions the database around coarse centroids and stores a list of vectors assigned to each centroid. A query chooses n_{probe} lists, producing a routed universe $U(q)$ of n_c candidates. Visiting fewer lists reduces access work but can exclude a true neighbour before any fine scoring. The centroids are a routing device; their distance ordering is a different selection problem from choosing database vectors.

Product quantisation divides a vector into M coordinate subspaces and represents each with a label into a small learned codebook [16]. If $\widehat{x}_m = \mathcal{C}_m[z_m]$, asymmetric distance computation

(ADC) keeps the query unquantised and scores

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

A query-specific table precomputes each subspace distance. Scanning then uses code loads, table lookups and accumulation. The product is between subspaces; the centroids *within* a learned subcodebook need not have a further coordinate product. This distinction determines whether Chapter 5's exact internal table factorisation is valid.

"Residual" has two meanings in this setting. IVF-PQ commonly encodes $x - \mu_{\ell(x)}$, making the coarse centroid part of reconstruction. JHQ instead routes vectors through IVF while defining its residual relative to the primary quantised reconstruction in rotated coordinates. These choices have different codebook dependencies; adding an IVF centroid to JHQ's residual reconstruction would count a term that its encoder did not subtract. The original PQ/ADC framework and GPU implementations also permit candidate expansion followed by a more accurate refinement stage [9, 17].

In the analysed regime with at least k valid routed candidates, JHQ's primary selection retains valid survivors $C_\alpha(q)$, capped by $c(\alpha) = \max\{k, \lceil \alpha k \rceil\}$, and refined selection returns $\mathcal{R}_\alpha(q)$. The set inclusion is

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

Intersecting with GT gives stage-wise coverage upper bounds on final recall. These inclusions do not quantify how much loss each stage causes; that requires fixed-universe counterfactual scoring. Refinement can improve ordering among retained vectors, but cannot restore excluded IDs.

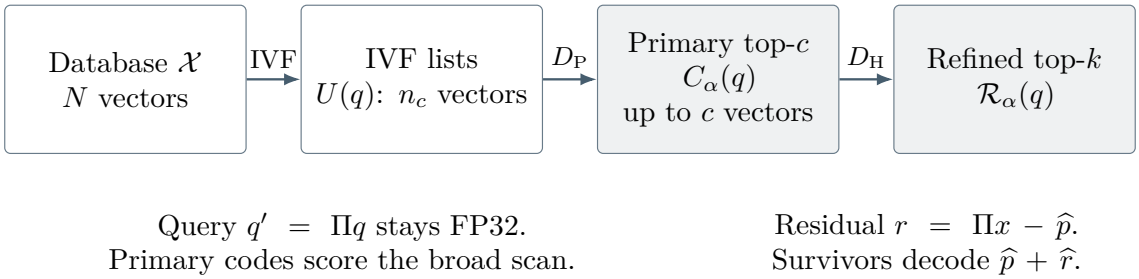


Figure 2.1. Candidate and precision cascade. IVF selects lists; primary scoring selects vectors; only survivors read residuals. The hierarchy is inherited. This diagram defines logical sets and score fidelity, not measured recall losses or GPU kernel boundaries.

Ignoring routing and selection, table-based broad scoring costs $O(n_c M)$ lookups and residual scoring costs $O(cd)$ coordinate work. This asymmetry motivates selective precision, but does not predict latency: memory placement, arithmetic, parallel work and candidate selection also matter. Increasing probes changes U; increasing α changes survivors within that universe. Treating these as one generic search-depth parameter hides their different quality and cost effects.

2.3 JHQ Representation and Assumptions

2.3.1 What is inherited

JHQ combines an orthogonal transformation, an analytical Cartesian primary code and a higher-precision residual layer [15]. Here the transform is square: $\Pi \in \mathbb{R}^{d\times d}$, $\Pi^\top\Pi = I$, $y = \Pi x$ and $q' = \Pi q$. Therefore

$$\|q - x\|^2 = \|\Pi(q - x)\|^2. \tag{2.4}$$

Although the prior work calls this an orthogonal JL transformation, this configuration does not reduce dimension. It preserves real-arithmetic distance exactly; quantisation subsequently introduces approximation. Actual FP32 orthogonality and GEMM error are separate implementation checks.

Partition y into M subspaces of dimension $D_s = d/M$. With B bits per primary subspace, $K = 2^B$ labels are formed from $L = 2^{B/D_s}$ scalar levels, requiring the implemented Cartesian parameters to make L an integer. Writing Φ for the standard normal CDF, the constructor uses

$$a_i = \sigma\Phi^{-1}\left(\frac{i - 1/2}{L}\right), \quad i = 1, \ldots, L, \qquad \mathcal{C}_m = \{a_1, \ldots, a_L\}^{D_s}. \tag{2.5}$$

At the evaluated $B = D_s = 8$, $L = 2$ and $\mathcal{C}_m = \{-a, +a\}^8$, with $a \simeq 0.67449\sigma$. Each byte records eight coordinate signs. The implementation estimates σ from vector energy; unit-normalised inputs motivate the scale $1/\sqrt{d}$. This representation is prior art, not an invention of the GPU execution layer.

If $\widehat{p}$ concatenates primary codewords, the residual is $r = y - \widehat{p}$. Quantising it gives $\widehat{r}$ and hierarchical reconstruction $\widehat{y} = \widehat{p} + \widehat{r}$. The tested residual layer uses eight bits per coordinate; primary and residual payloads consequently occupy $d/8$ and d bytes per vector, before metadata. Selective refinement changes how many residual rows are read, rather than how many are stored. The inherited coarse-to-fine search is implemented as a resident GPU pipeline in Chapters 4 and 5.

2.3.2 Model assumptions and score semantics

Random rotation can spread coordinate energy, motivating a shared Gaussian-inspired scalar model. It does not force an arbitrary realised database to have independent, zero-mean Gaussian coordinates. For a fixed rotation and empirical database mean μ, the transformed mean is $\Pi\mu$ and $\|\Pi\mu\| = \|\mu\|$. Averaging over random rotations is different from centring one fixed corpus. Even a uniformly random point on a sphere has dependent coordinates through its fixed norm. Consequently, isometry is an algebraic guarantee; the distributional approximation is a modelling assumption. Chapter 8's centred-coding pilot examines one practical consequence of that distinction.

Terminology also matters for scalar optimality. The implemented primary levels are Gaussian mid-quantiles. For a zero-threshold one-bit quantiser under an exactly Gaussian model, squared-error-optimal reconstruction levels are instead conditional means, $\pm\sigma\sqrt{2/\pi} \simeq \pm 0.79788\sigma$. Thus the inherited quantile construction is not, by itself, a proof of MSE-optimal levels. For fixed signs and a positive common scale, the choice of a cancels from primary ordering:

$$D_{\mathrm{P}} = \|q'\|^2 + da^2 - 2a\langle q', s\rangle, \qquad s \in \{-1, +1\}^d. \tag{2.6}$$

It still affects residual reconstruction. This thesis relies on the codebook's separability and measured quality, rather than a universal Gaussian or quantiser-optimality claim.

Finally, the three distance notions remain distinct. D_{raw} uses original vectors; $D_{\text{P}} = \|q' - \widehat{p}\|^2$ uses the primary reconstruction. The implemented refined score satisfies

$$D_{\text{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.7}$$

Its query-only constant preserves ranking but changes reported distance values. Exact selection under either computed score leaves representation and routing error intact. It is not raw-vector exact reranking.

2.4 GPU Execution Concepts

The implementation uses CUDA's SIMT execution: threads are organised into blocks and 32-thread warps, with registers, block-shared memory and device global memory serving different reuse scopes [25]. Coalescing groups neighbouring lane accesses into suitable memory transactions; it does not require fewer logical code bits. Candidate-parallel primary scanning assigns lanes to neighbouring vectors, while refinement assigns several lanes to one survivor's coordinates. These mappings exploit different available parallelism, so one mapping need not suit both stages.

Occupancy is constrained by registers, shared state and block geometry. More resident threads can hide latency, but more blocks per query can also duplicate tables and local candidate buffers. Shared memory is valuable when query data are reused enough to repay loading and synchronisation. These observations motivate Chapter 5's table grouping, packed access and batch-aware scheduling; the throughput ablations assess their combined effects. A claim of bandwidth saturation or a specific cache bottleneck would additionally require counters and a consistent physical-byte model.

Selection is itself substantial computation. Faiss's GPU work combines efficient distance processing with warp-oriented selection [17]. Bitonic networks, radix methods and merge-based approaches provide established building blocks with different k, input length and state costs. AIR top-k and GridSelect further illustrate why global selection design depends on workload and whether scores arrive on the fly [35]. This thesis adapts such ideas to JHQ's probe selection and survivor retention; it does not claim to invent these primitives.

CUDA Graph replay reduces repeated launch setup for a captured workload. Capture, workspace allocation and changes in graph shape belong to the index/search lifecycle, rather than the mathematical scoring rule. Likewise, publishing a shared threshold requires an explicit ordering and memory-access contract. Correct scalar scores do not alone prove concurrent admission safe. Chapter 5 separates these correctness conditions from performance measurements, and Chapter 3 keeps device profiling distinct from the host-visible search interval.

2.5 Graph Search on GPUs

Graph ANN replaces list scanning with navigation through promising neighbourhoods. Its benefit is to avoid scoring most database vectors; its GPU difficulty is the dependency between discovering a neighbour and deciding which neighbourhood to expand. SONG separates graph search into stages to expose distance parallelism and controls dynamic memory overhead [36]. This is an alternative access strategy, rather than a different implementation of JHQ's fixed routed candidate universe.

CAGRA designs graph construction and search for GPU parallelism, including small- and large-batch queries [28]. Its graph connectivity and search width expose a different tuning surface from IVF probes. Storing edges and vectors creates a capacity trade-off; quantised-vector CAGRA is also relevant, so fp32 CAGRA alone is an incomplete compressed-index comparison. The thesis measures both in E1. Their different operating regions are reported under the common boundary rather than inferred from cross-paper speedup numbers.

BANG targets a different capacity regime: it keeps compressed vectors on the GPU and accesses the host-resident graph through CPU/GPU cooperation [18]. Capacity is obtained by distributing the working structures, with transfer and coordination becoming part of the design. Its billion-scale results demonstrate an alternative to complete device residency, not capacity evidence for this thesis. The present JHQ implementation retains its compressed search index on one device and has no comparable host-graph execution path.

The comparison therefore turns on access regularity, representation and residency, not a general graph-versus-quantisation hierarchy. Graph navigation can be attractive for a small number of queries; list scans expose regular code access and abundant batch work. Either can be bottlenecked by selection or movement. Which wins at a target recall is an empirical question for a specified device, memory budget and workload. E1 tests that question at batch 1,000; internal small-batch JHQ gains do not answer it for graph competitors.

2.6 Quantised Search and Fast Tables

2.6.1 Learning a representation versus exploiting its structure

Optimised product quantisation learns the space decomposition and codebooks to reduce quantisation distortion [12]. Its rotation is optimised for the data, whereas JHQ's random square rotation supports a cheap analytical primary model. The trade-off involves fitting cost, reconstruction fidelity and scorer structure. Improving a general learned codebook does not automatically preserve the internal Cartesian product needed by an exact 16+16 decomposition.

ScaNN's anisotropic quantisation work instead weights error according to its relevance to maximum-inner-product retrieval, showing why reconstruction MSE is not the only useful objective [14]. Its objective and implementation setting differ from the squared-L2 hierarchical pipeline here. The useful comparison is the choice of loss: a rank-sensitive representation should be tested against actual retained-neighbour quality and execution cost. The stopped rank-oriented experiment in Chapter 8 provides an example of this distinction within JHQ.

2.6.2 Small lookup tables have substantial prior art

PQ Fast Scan replaces reliance on cache-resident tables with tables suited to CPU SIMD registers [1]. Quicker ADC generalises register lookup techniques and introduces irregular quantisers and split tables [2]. These works establish that table shape and the execution instruction set must be designed together. This thesis's claim is the exact decomposition available from its inherited Cartesian coordinate structure, and its measured implementation on a GPU. CPU SIMD results are conceptual precedents, not throughput measurements on the tested GPU. FP32 table association and cutoff ties still require separate numerical checking.

RaBitQ also uses randomised quantisation and compact sign information, with a distance estimator and theoretical error analysis [10]. Multi-bit extensions trade storage for estimator

accuracy [11]. Their guarantees attach to their specified constructions and estimator assumptions. They cannot be transferred to JHQ merely because both use rotations and binary components. In JHQ, primary signs select survivors and an explicitly stored residual supplies the next precision level; the resulting quality depends on both truncation and reconstruction.

GPU IVF-RaBitQ brings this family into a complete GPU build/search design, including GPU-oriented distance computation and fused search [31]. It is therefore a particularly relevant contemporary baseline, rather than background evidence of a missing GPU quantiser. Its fused execution makes clear that compact codes alone are insufficient: the access and selection pipeline must also be competitive. E1 includes multiple bit budgets, and its high-recall results limit any broad claim of JHQ dominance.

2.6.3 Refinement effort and adaptive stopping

Candidate expansion and reranking already appear in Faiss's `k_factor` and cuVS refinement controls [6, 9]. Faiss autotuning can use GT-labelled queries to search recall–time trade-offs [8]. Learned adaptive termination predicts sufficient search effort [19]; DARTH frames early termination around declarative recall targets [5]. These are relevant alternatives to spending a uniform effort everywhere. Chapter 6's particular policy selects one workload-level survivor multiplier using unlabelled agreement with a larger budget. It avoids a trained predictor, while inheriting the reference's quality limits. This distinction is the policy's scope, rather than a claim that adaptive search or refinement budgets are new ideas.

2.7 Research Gap and Design Choices

Table 2.1 compares the alternatives by the constraint they address. The literature already provides GPU graph search, compressed distance computation, register-oriented lookup, fast selection and adaptive effort. A useful contribution must therefore specify the representation contract, the integration decision and the evidence for it. "A GPU version" alone does not identify that contribution.

Table 2.1. Design alternatives and the thesis's narrower research gap. This is a mechanism comparison; measured rankings follow in Chapter 7.

Alternative	Useful structure	Principal trade-off	JHQ-GPU question
IVF-PQ / OPQ	Learned subspace codes; ADC	Distortion and training; arbitrary subcodebooks couple coordinates	What does an internally Cartesian primary code permit in GPU scoring?
Fast Scan / Quicker ADC	Small tables and SIMD lookup	Table precision, shape and ISA constraints	Does exact splitting repay extra GPU lookups?
SONG / CAGRA	Selective graph navigation	Dependent access, graph storage and construction	Where does a resident hierarchical scan compete?
BANG	Compressed GPU scoring with host graph	Capacity versus communication	What can complete compressed residency achieve?
IVF-RaBitQ	GPU code construction and fused search	Bit budget, estimator and probe effort	Which recall regions justify the residual hierarchy?
Adaptive effort	Refine ratios, autotuning, learned or declarative stopping	Labels, reference fidelity, prediction or calibration cost	Can unlabelled workload agreement select useful effort at an acceptable risk and total cost?

The first gap is an execution contract for this specific hierarchy: coordinate-separable tables and packed access must work with exact primary retention, concurrent threshold publication

and residual scoring. The second is a measured choice of execution regime across batch size and candidate skew, since splitting work can add more than it saves. The third is a quality-and-cost account of workload-level refinement selection, separating reference agreement from GT recall and selected-arm speed from net savings.

These gaps motivate preserving the inherited representation while changing execution and effort. More expressive learned codes and graph navigation remain meaningful alternatives, but changing them simultaneously would obscure the component questions. The thesis evaluates this constrained design through mathematical contracts, paired experiments and competitive frontiers. Its unresolved distribution, tie and calibration assumptions remain visible, making the resulting claims narrower and testable.

Chapter 3

Research Design and Measurement Methodology

The evaluation connects a systems intervention to an observable cost or quality change. It combines paired execution ablations, held-out budget selection and independently tuned system frontiers. These comparisons answer different questions: a faster kernel, a useful calibration policy and a competitive search system require different controls. This chapter defines those controls and the evidence needed to interpret them. Chapters 4–6 describe the mechanisms; Chapter 7 presents their measured outcomes.

3.1 Research Questions and Comparisons

Table 3.1 defines the comparison unit and what would challenge each research claim. A *cell* fixes a dataset, index configuration and workload setting; arms are alternative implementations or budgets within that cell. For paired execution measurements, the index state, queries, probe count, refinement budget and scoring representation are held fixed unless the named treatment changes them. The recorded paired ratio is $Q_{\text{treatment}}/Q_{\text{control}}$. Geometric means summarise multiplicative ratios over the stated cells, giving each cell equal weight.

Table 3.1. Research questions as testable comparisons. A capacity observation has a different control from a paired execution intervention.

Question	Intervention / comparison	Held fixed or accounted for	Measure and counter-evidence
RQ1: build and capacity	GPU construction; cached versus cold lifecycle; compressed versus raw storage	N, d, representation, routing and timed construction boundary	Train/add time and resident bytes; failed builds or model–allocation gaps delimit feasibility
RQ2: execution	LUT grouping, word access, refinement lanes, grouped scan, shared bound and probe selection	Paired state, workload and score; reference output semantics	Wall QPS, stage time and output checks; slowdowns or non-tie mismatches challenge the mechanism
RQ3: budget	Sampled / full-pool rule versus fixed 64, 100, 200, enumeration and GT oracle	One index and route; disjoint calibration and held-out query positions	Held-out recall loss, selected-arm QPS and calibration cost; threshold crossings challenge reliability
RQ4: operating region	Independently tuned JHQ, CAGRA, IVF-PQ and IVF-RaBitQ	Exported vectors, GT, k, device and host-to-host boundary; memory reported separately	Matched-recall QPS and coverage; stronger competitors at other recall targets challenge broad superiority

The design distinguishes a mechanism's implementation from its explanation. A paired word-access ablation can establish the throughput effect of that access path in the measured pipeline. Attributing the effect to cache misses or occupancy would additionally require hardware-counter evidence. Likewise, a cross-system frontier compares complete choices of representation, algorithm and implementation; it cannot isolate one kernel's contribution. Section 7.6 provides an integrated scheduling comparison so that component ratios need not be multiplied across incompatible baselines.

The research questions organise an iterative development record. They are not a preregistered test plan: the E1 grid was extended after observing incomplete recall coverage, and the

interpretation of workload-dependent refinement emerged from budget sweeps. Low-recall losses, high-recall reversals and unsuccessful alternatives remain part of the evaluation. Output agreement with a quantised reference and recall against raw-vector GT are separate quantities by definition. The budget rule uses the former; the held-out evaluation assesses the latter.

3.2 Datasets, Queries, Ground Truth and Splits

Table 3.2 records the six exported datasets used by the newer budget, LUT and scale campaigns. The poster's E1 comparison uses the first four. Vogue contains fashion-image embeddings; arXiv contains abstract embeddings; the two OpenAI datasets use Qdrant's DBpedia exports [4, 29, 30, 32]. BGE uses the Italian Wikipedia configuration, and Stella uses the TREC24 BioGen corpus export [33, 34]. Sizes describe the measured exports after preparation, rather than the current upstream cards.

Table 3.2. Data and JHQ training manifest. n_q is the actual query count; $n_t = 39n_{\text{list}}$ is the requested coarse-training sample size. E1 uses the first four rows. All rows use squared L2 and $k = 10$.

Dataset	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

The preparation sources specify L2 normalisation and seeded query removal from the database. The streamed OpenAI/BGE/Stella converter first removes non-finite rows, normalises vectors with norm above 10^{-12}, and uses seed 42 to choose query positions without replacement. Stella queries are sampled from its corpus, rather than its separate 65-query benchmark split. Its shortened export supplies 999 queries. These are vector-neighbour workloads; relevance-labelled retrieval is outside the measurement. The two OpenAI exports use the same positional sampling rule. Supplementary vector checks support row alignment for all queries and a 10,299-row base sample; original source-ID joins remain absent (sections 7.2 and 7.2.3).

All methods in a cell read the same query and GT files. Ground truth is computed by exhaustive squared-L2 search over exported FP32 vectors; the streamed converter maintains top-k across database tiles. Evaluation uses stored GT order, including its floating-point tie choices. With $\mathcal{T}_k(q)$ the first k GT IDs and $\mathcal{R}_k(q)$ the returned ID set,

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

GT rows are 100 wide for Vogue/arXiv and 20 wide for the other four; only ten entries are scored. Duplicate returned IDs do not earn repeated credit. Range, duplicate and repeat-call checks are recorded separately.

JHQ uses $M = d/8$, $B = B_r = 8$, eight coarse iterations and a requested 100,000-vector residual-training sample. Search sweeps reuse the `arms6_cache` trained states and re-encode the database; cold builds start without that state. The code requests training from database samples, separately from calibration queries. For budget evaluation, even query positions form a 500-query calibration pool and odd positions form a 500-query held-out pool, or 499 for

Stella. Seeded subsamples of sizes 32, 64 and 128 are drawn 64 times per cell; FULL uses all 500 calibration queries. GT labels enter the offline oracle and held-out assessment, while the deployable agreement rule uses search outputs (sections A.3 and 6.2).

Preparation code and shape logs establish this protocol; complete historical vector-file and trained-state hashes are unavailable. The ledger preserves that distinction between an inspected preparation procedure and an immutable identity of every run's data.

3.3 Timing, Variance and Memory Accounting

The principal comparison starts with FP32 queries in host memory and ends when top-10 IDs and scores are available in host memory after stream completion (figure 3.1). It includes pinned staging, H2D, query preprocessing, search and D2H. CAGRA int8 query quantisation is inside this boundary; quantiser training and index construction are outside it. Resource handles and output buffers are reused. JHQ returns to pageable host arrays and the baselines to pinned arrays; CAGRA/PQ use Python APIs, whereas JHQ/RaBitQ use C++. These interface differences remain visible when interpreting small margins.

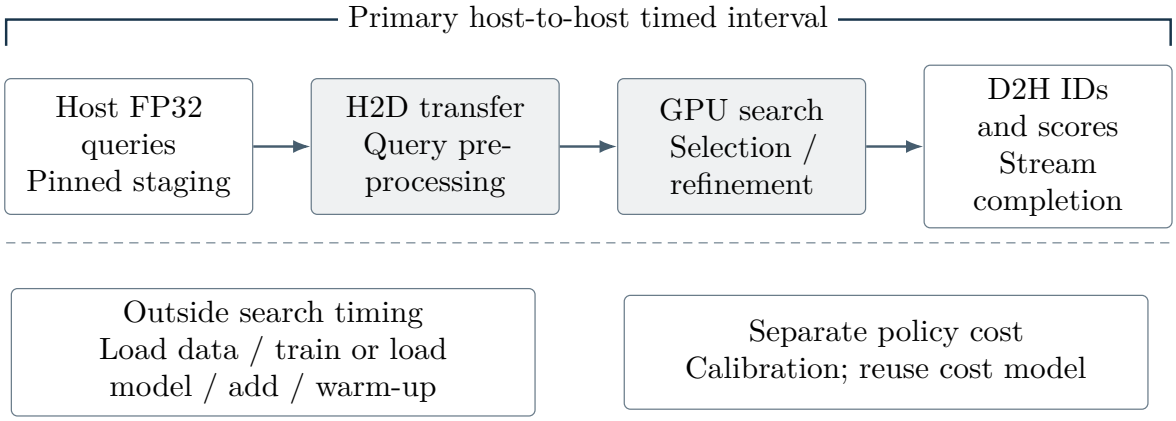


Figure 3.1. Timing boundaries. The host interval includes transfers and query-specific computation. Device-stage profiles explain parts of that interval; separate profiling and synchronisation can change execution, so their sum is not an interchangeable end-to-end latency.

For round r with m_r timed calls, each answering a batch of b queries,

$$Q_r = \frac{m_r b}{\sum_{i=1}^{m_r} t_{ri}}, \qquad Q = \operatorname{median}_r Q_r, \tag{3.2}$$

with t in seconds. E1 uses $b = 1{,}000$ and five rounds. JHQ performs one untimed call per variant per round, then normally 24 timed calls. Baselines perform three initial warm-up calls and normally 24 calls per round, reducing that count for points classified as slow. Actual counts are retained. Warm-up excludes JIT and graph capture from steady-state timing. A batch's median or p95 latency describes repeated batch calls, not the latency distribution of individual queries or concurrent requests.

The median of round throughputs is distinct from the reciprocal of median call latency. Both summaries are retained where available, together with round minima/maxima and baseline call traces. Slow valid calls remain in the primary estimate. Rounds share a trained index, and budget draws share a query pool: their counts measure timing variation or sample sensitivity, rather than independent training replications. Small paired differences are reported with their

spread and consistency; no independence-based significance claim is attached to the sweep cells.

The principal environment is an RTX 5090 with 32,607 MiB reported memory, driver 595.71.05, CUDA toolkit 13.0 and cuVS 26.08.01. Runs acquire a cooperative GPU lock and record environment snapshots; OpenMP uses 16 threads within a 25-CPU container quota. E1 records a 575 W power limit. These snapshots do not establish locked clocks or constant host load. JHQ's Release compute_80 PTX is JIT-compiled [27]; library baselines use their installed builds. Auxiliary device-only timing and stage profiles are labelled separately, since JHQ's main API accepts host queries.

Memory accounting separates persistent encoded arrays, model/IVF metadata, query workspace and runtime/context allocations. A process used-memory sample includes more than the index; a build-time allocation delta excludes memory pools created later. E1's context-subtracted JHQ totals and CAGRA's index deltas retain these different definitions. Component models explain resident totals and expose an unattributed remainder (sections 4.4 and 7.7); they do not supply an equal-memory performance comparison. Supplementary 20 ms device samples separately measure construction peaks. Cold train/add timers likewise exclude input loading and preceding object/host-model setup.

Supplementary evidence boundary. The supplementary campaigns add the checks catalogued in section A.4. Their hardware class and reported driver/toolkit match the archives, but timing is a separate session. Arithmetic modes and the cached-model precision intervention are specified in sections 4.2 and 7.2.3; historical baseline comparisons retain that session distinction.

3.4 Baselines and Frontier Construction

The competing families cover graph traversal (CAGRA fp32/int8), product-quantised IVF and GPU IVF-RaBitQ [16, 28, 31]. FP32 GPU brute force provides an exhaustive-search reference. Each family sweeps its own implementation parameters (table 3.3); the recorded index configurations can differ in storage and reconstruction fidelity. Consequently the main frontier measures achievable speed at a quality target within these grids. A separate memory account accompanies it.

Table 3.3. Principal E1 tuning dimensions. Exact per-point parameters and coverage extensions remain in the raw records. The grid is finite and differs between methods.

Family	Recorded search dimensions
JHQ	Probes and α; initial probes 64/128/256 and 0.5/1/2 times the workload budget; coverage extensions reach probe 1,024 and larger/smaller budgets
CAGRA fp32/int8	Graph degree 32/64; search width 1/2/4/8; intermediate top-k capacity 32–512
IVF-PQ	Eight-bit subcodes; 192/384/768-byte codes at $d = 768$, 384/768/1,536 at higher dimensions; fp16/fp32 lookup settings; probes 16–512
IVF-RaBitQ	4/8/9 bits per dimension, supported search modes, probes 16–1,024; additional 4,096-list OpenAI indexes
GPU brute force	Exhaustive FP32 squared-L2 search, common host boundary

The frontier retains points not dominated in both recall and QPS: point a dominates b when $R_a \geq R_b$ and $Q_a \geq Q_b$, with at least one strict inequality. This envelope can combine different

index configurations. For target recall r, the measured estimator is

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

It is undefined when no sampled point qualifies. The interpolated estimator uses adjacent frontier points (R_a, Q_a) and (R_b, Q_b):

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

The archived parser first accepts a frontier point within 5×10^{-4} of the target as a direct match, including near endpoints; otherwise it interpolates only inside the measured range. It extrapolates no segment. Both estimators are reported where comparisons are close: one selects an observed qualifying configuration, while the other reduces sensitivity to unequal grid spacing. Neither supplies an equal-storage comparison. T5 also retains the fastest qualifying measured endpoint when a target lies below a method's minimum frontier recall; interpolation being undefined does not make a more accurate configuration ineligible.

The parser retains failures and invalid-output diagnostics outside the valid-point frontier. Repeat-call ID changes alone do not discard a point; set changes and ordering changes are distinguished in the source records. Corrected isolated large-baseline reruns supersede contaminated builds, and Stella runs skipped at batch 1,000 contribute no search points. Section A.2 records those decisions. The thesis reconstructs its frontier from all declared baseline families, including int8 CAGRA and adverse operating regions (figure 7.1). Historical ADC and poster plots retain their own version and measurement boundaries.

3.5 Versioned Claim-to-Evidence Ledger

Every main empirical claim is associated with an experiment family, a metric and consumed records. The machine-readable registry is `data/ch3/audit.json`; detailed numeric reconstruction is in the Chapter 4–7 audit files. Records identify the intervention, execution, index/query configuration, measurement boundary and analysis inputs. A file hash verifies the input consumed by analysis; it cannot recover an executable or trained state that was never archived. Appendix A gives the reproduction entry points; the repository's supplementary evidence retains the complete field definitions.[1]

The E1 system frontier, newer table sweep, budget campaign and large-data extensions remain separate from the ADC paper's older measurements. Historical packing and refinement records answer their original paired questions. The poster links these mechanisms visually, but does not replace run metadata or extend a comparison's coverage. Version names therefore describe provenance; mechanism names describe the scientific intervention.

`scripts/build_ch7.py` reads archived measurements and regenerates the empirical plots and principal numeric tables, including figures 7.1, 7.4 and 7.11. The source ledger and family audits retain original paths and hashes; `scripts/audit_ch3.py` checks the common protocol against them. Source inspection and offline reconstruction establish correspondence between archived records and reported quantities. Repeating GPU measurements additionally requires the relevant binaries, datasets and trained states; missing diagnostic coverage is identified with each result. Appendix A supplies implementation and artifact details and specifies which reproduction inputs are available.

[1] Repository supplement: `docs/supplementary-evidence.md`.

3.6 Evidence Strength and Threat Controls

Proofs establish properties under their stated assumptions; independent references test implementation agreement on finite cases. Paired benchmarks measure an intervention within a fixed workload, and system measurements locate operating regions on the tested device and grids. Byte and payback models retain their accounting assumptions; proposed explanations require separate tests. Exact primary selection concerns the computed primary score, whereas raw-vector recall also depends on routing and reconstruction.

Shared-index dependence, finite pools, grid selection, interface asymmetry and one-device coverage motivate the controls above. The population and ordering assumptions are analysed in sections 6.2 and 6.3; section 8.3 consolidates the resulting limits to interpretation and deployment.

Chapter 4

System Architecture and GPU Index Construction

This chapter addresses RQ1: how to construct a searchable, GPU-resident JHQ index from high-dimensional vectors. JHQ supplies the transform and primary/residual representation [15]. The systems problem is to train that representation, encode a database larger than its available full-precision device storage, and expose consistent list-ordered arrays to the query path. The discussion follows training, placement, allocation and validation. Detailed query kernels follow in Chapter 5; measured comparisons are in Section 7.7.

4.1 End-to-End Build and Search Boundaries

The index has three lifecycle stages: training produces reusable model state, adding encodes and physically arranges database rows, and searching uses the completed arrays. This separation matters experimentally. A trained-state cache avoids learning the model again, but still requires database encoding and list construction. A cold train-plus-add run and a search over a reused state measure different operations.

The host owns the input vectors and supplies training samples. It constructs the square transform and analytical primary codebook. GPU matrix products apply the transform; the

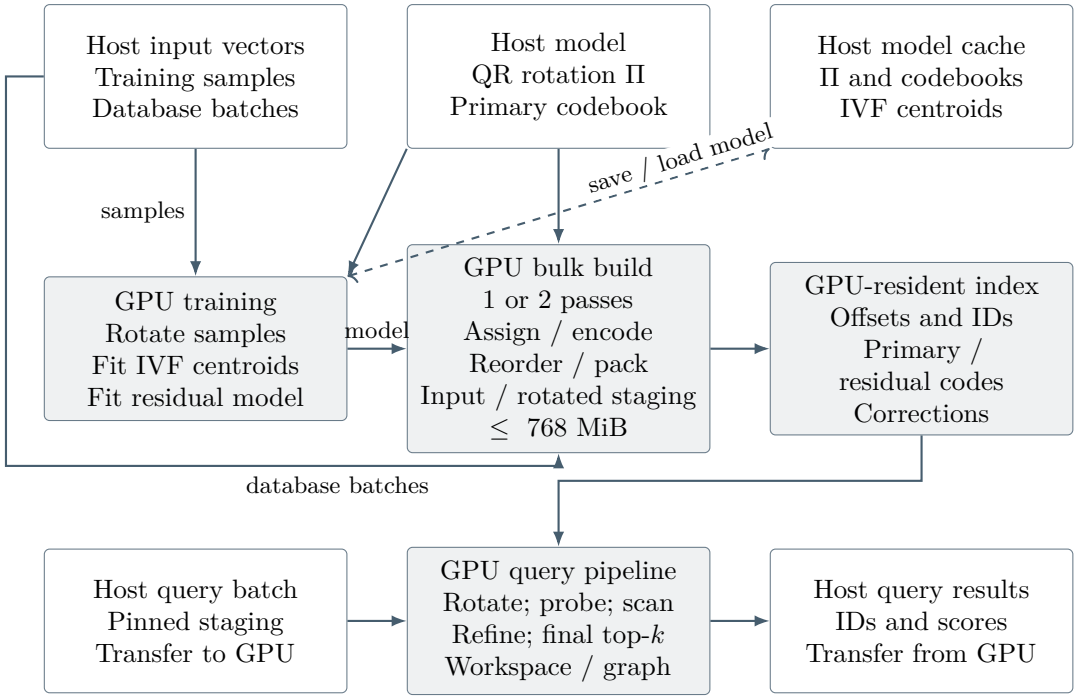


Figure 4.1. Implemented build and query boundaries. Host work includes sampling, orthogonal-matrix construction and analytical primary-codebook generation; the evaluated configuration fits IVF and residual models on the GPU. The cache stores trained state, whereas the completed database representation resides on the device. Dashed arrows denote trained-state reuse. The staging label bounds only raw/rotated float buffers at source defaults; other pipeline scratch is additional.

device training path fits coarse IVF centroids and scalar residual codebooks. Adding then transfers bounded batches, encodes both levels, determines IVF membership, and writes a common list-ordered representation. Host orchestration and device computation therefore coexist within a GPU-resident search system.

The model state consists of the transform, primary codebook, residual codebooks and IVF centroids. The database state adds list offsets, original row identifiers, primary codes, residual codes and correction factors. Search workspace has a shorter logical lifetime: rotated queries, centroid scores, probe offsets, LUTs and survivor buffers are overwritten on the next batch, although their allocations are retained. A captured CUDA graph belongs to this execution state and references the existing arrays.

The audited API builds a fixed snapshot with one successful `add` call; it rejects a subsequent add. Physical positions remain fixed while searching that snapshot. Changes are handled by building a replacement snapshot. This gives a simple bulk-rebuild lifecycle; mutable insert/delete operations and transactional publication would require additional index management.

Search accepts host queries and returns host IDs and computed scores. The query-only constant in those scores and its implications for range thresholds are derived in equation (5.20). The measured host-to-host call boundary is defined in section 3.3.

This design streams construction inputs from host memory while retaining the compressed database on the GPU. It supplies neither a disk-backed search path nor a claim that an arbitrarily large compressed index fits one device.

4.2 Training, Rotation and Codebook Generation

4.2.1 Transform and sampling

The implementation draws a seeded Gaussian matrix on the host and uses LAPACK QR routines to construct a square orthogonal matrix Π. For row-major training data X_t, device matrix multiplication computes

$$Y_t = X_t \Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

The output retains dimension d. This is an orthogonal change of coordinates; QR construction and device products have finite-precision error, whose measurement is part of C0 in section 4.6.

The harness selects database rows at a fixed stride. The primary/IVF sample has 159,744 rows for Vogue, 319,488 for arXiv and both OpenAI datasets, and 1,277,952 for BGE and Stella: exactly $39n_{\text{list}}$ in each case. This matches FAISS's default warning threshold of 39 training points per centroid [7]; it is a sampling rule, not a statistical guarantee for the eight-iteration IVF fit. Residual training uses a separate 100,000-row strided sample, keeping its cost independent of the larger coarse-training sample.

A historical controlled sweep held the primary model and IVF centroids fixed and varied only the residual sample size on Vogue and OpenAI-3072, with $B_r \in \{4, 8\}$ and 32/128/256 probes. Across its 12 configurations, 100K versus all rows changed recall by at most 0.0018: 11 values were lower and one was equal. Larger intermediate samples did not improve recall monotonically, and the residual codes themselves changed substantially. These results motivate the bounded training cost while preserving a measurable quality tradeoff. They used 25 residual Lloyd iterations and do not replace the later six-dataset evaluation. Historical diagnostics and their input hashes are recorded in `data/ch4/review-audit.json`.

4.2.2 Why retain the analytical primary model?

The primary constructor estimates a common scale from the sample:

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

At $B = D_s = 8$, the Cartesian codebook uses levels $\{-a, +a\}$, $a = 0.674\sigma$. Each byte represents eight signs. This preserves the analytical construction in JHQ [15], enabling coordinatewise $O(d)$ encoding and exact factorisation into the small LUTs of section 5.2. Freely refining 256 codewords per subspace with Lloyd updates generally removes that separability.

For a fixed sign reconstruction $\widehat{p} = as$, $s_j \in \{-1, +1\}$,

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

For $a > 0$, real-arithmetic primary ranking of these fixed sign codes is independent of the scale. This algebraic property does not require Gaussian coordinates. Scale still changes the residuals $r = \Pi x - \widehat{p}$, their fitted codebooks and the refined ranking; $a = 0$ is degenerate, and floating-point ties require the usual care.

The earlier frontier diagnostics illustrate the tradeoff. On Vogue with $M = 96$, $B_r = 4$ and 128 probes, the earlier and analytical-primary builds achieved primary-only recall 0.6618 and 0.6342, but full-system recall 0.9452 and 0.9409. Residual refinement reduced the observed gap. These were historical cross-version comparisons, including residual-training changes, rather than an isolated causal estimate for replacing the primary codebook. The design choice here is a structured representation whose GPU execution is inexpensive; Chapter 7 evaluates its complete quality–cost frontier.

4.2.3 Coarse assignment as filter and refine

IVF centroids are seeded from spaced transformed sample rows and updated for eight configured iterations. Assignment orders centroids by $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$. The source offers FP32 and accelerated assignment paths; under its usual dimensional constraints, the default uses int8 products to filter choices and FP32 recomputation to resolve ambiguity.

Lemma 4.1 (Assignment with certified score intervals)**.** *Let $\widetilde{y}, \widetilde{\mu}_\ell$ be dequantised approximations, $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$. Keeping centroid norms exact, the approximate score $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ has real-arithmetic error bounded by*

$$E_\ell = 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|). \tag{4.4}$$

Given valid intervals $[L_\ell, U_\ell]$ containing every target score, discarding only $L_\ell > \min_j U_j$ and exactly resolving the remaining scores preserves the minimum and its specified tie rule.

Proof. Expand the dot-product error as $\langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle$ and apply Cauchy–Schwarz. A discarded score exceeds an upper bound on some retained score, so it cannot attain the minimum. Keeping interval equality preserves possible ties. □

The implementation uses conservative norm bounds and FP32 headroom constants in this filter-and-refine pattern. The lemma explains why a valid bound permits exact rejection; a bitwise guarantee against every FP32 accumulation order additionally requires bounds for

those arithmetic errors. The archived assignment diagnostic compares against an FP16 path and reports 0/14,838, 6/31,748 and 3/16,359 differing assignments on Vogue, BGE and Stella tail batches. It is finite implementation evidence, not a full FP32 reference certificate. The cold-build logs do not trace the executed assignment precision.

4.2.4 Residual fitting and the iteration budget

IVF fitting precedes residual fitting. The latter encodes its sample with the fixed primary model and forms $r = \Pi x - \widehat{p}(x)$, independently of the assigned IVF centroid. Each subspace fits a sorted scalar codebook of $K_r = 2^{B_r}$ entries shared by its D_s coordinates. The GPU estimator sorts residual values and applies one-dimensional Lloyd updates with a 2,000-iteration cap in the evaluated configuration.

A historical 100K-sample diagnostic motivates a fixed budget with diminishing quality returns: 2,000 versus 25 updates gains 0.0063–0.0076 recall. The later changes can exceed the two-run variability reported for BGE and Stella, so the diagnostic does not establish a statistically exact plateau. It also reports persistent changes in sparse tail bins; this is an observation about the implemented floating-point updates, not a failure of the exact Lloyd descent theorem. Cold logs reach the cap, including Stella, so training is described as budgeted iterative fitting. At cap 20,000, the historical Stella all-row and 100K results were 0.9921 and 0.9920, respectively, under that earlier protocol.

4.2.5 Construction work and observed imbalance

Dense QR costs $O(d^3)$ once; dense rotation costs $O(n_t d^2)$. Direct coarse assignment costs $O(n_t n_{\text{list}} d)$ per iteration. Analytical primary construction writes $O(MKD_s)$ values; residual extraction touches $O(n_r d)$ coordinates, followed by sorting and scalar updates. These describe work, rather than predict latency. Synchronised phase diagnostics place about 83% of BGE/Stella training in IVF k-means (about 2.97 s). They use one add buffer and forced synchronisation, separately from the normal overlapped timings in section 7.7.

List-size imbalance enters the scan rule in section 5.5. The individual contributions of eight IVF iterations, spaced seeds and finite sampling to that imbalance were not isolated.

Arithmetic used in the measured implementation. The evaluated implementation (v69) stores FP32 matrices but enables cuBLAS TF32 tensor-core math for $d \geq 1{,}536$. In its compiled faithful branch, a null override falls through to the dimension-based default. This affects training/add/query rotation and centroid GEMMs on both OpenAI datasets; an environment override cannot disable that branch. Thus FP32 storage does not imply full FP32 multiplication. The supplementary FP32 variant (v72) corrects this switch. Its sensitivity experiment reuses the same TF32-trained cache and changes add/query arithmetic, rather than retraining the model in FP32 (section 7.2.3).

4.3 IVF Assignment, IDs and Physical Memory Layout

4.3.1 From database IDs to list-ordered positions

An input row has identifier $i \in \{0, \ldots, N-1\}$. Coarse assignment places its transformed vector in a list according to the computed centroid scores. The mathematical assignment target is

$$a(i) = \operatorname*{arg\,min}_{0 \leq \ell < n_{\text{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

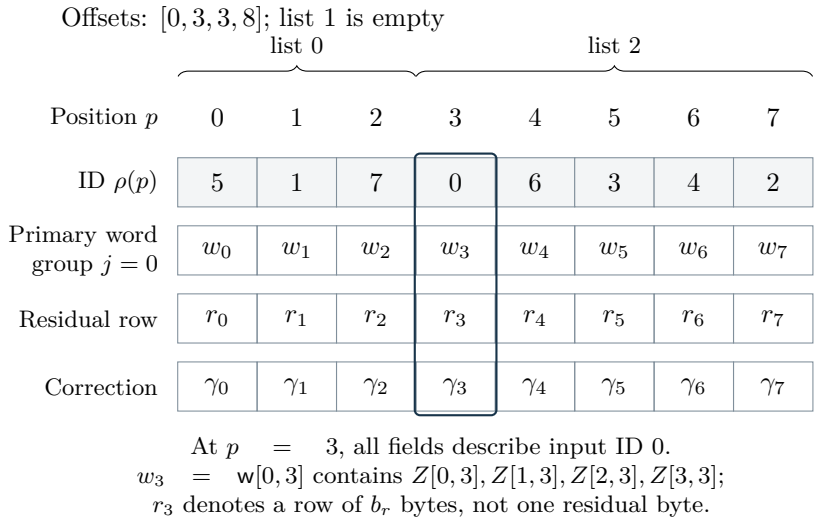


Figure 4.2. One common physical position joins IDs, packed primary words, residual rows and correction factors. This illustrative eight-row index has offsets $[0, 3, 3, 8]$, including an empty list. A primary word packs four subspaces of one vector. Values and list sizes are schematic.

Its implementation uses the selected numerical assignment path. Encoding the primary and residual representations does not subtract $\mu_{a(i)}$; list membership restricts the future search universe independently of those reconstructions.

The builder sorts assignments together with a sequence of input IDs. Let $\rho(p)$ be the input ID at sorted physical position p and let o_ℓ be the start of list ℓ. The completed arrays satisfy

$$0 = o_0 \le o_1 \le \cdots \le o_{n_{\text{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

Offsets are obtained by lower-bound searches over the sorted assignments. Equal consecutive offsets represent an empty list. Sorting by assignment does not promise stable ordering of equal-list rows across independent builds. A position is stable within the completed snapshot; its meaning is the row $\rho(p)$ shared by every associated array.

The resident ID array implements $p \mapsto \rho(p)$. Its inverse $\rho^{-1}(i)$ is a mathematical bijection, but a second permanent inverse array is unnecessary for search. The two-pass builder constructs that inverse temporarily so input batches can scatter into their final positions. Query results translate physical positions back to input IDs; Chapter 5 uses positions for primary ties and database IDs for final ties.

4.3.2 Exact addresses and representation widths

Let $Z[m, p]$ denote the primary byte for subspace m at physical position p. The intermediate subspace-major copy has byte offset $mN + p$. Final packing produces

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{addr}_{\text{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

Here $j = 0, \ldots, M/4 - 1$. Word layout requires $M \bmod 4 = 0$; the active word-layout build rejects other values. A separate byte-layout compilation retains the byte representation. Each packed array occupies NM bytes, but repacking temporarily holds both old and new copies, costing $2NM$. The one-pass builder temporarily gathers a vector-major $[N, M]$ primary array,

Table 4.1. Per-vector bytes. JHQ includes codes, ID and correction; SQ8 is vector codes alone; CAGRA int8 models vector codes plus 32 four-byte neighbours. Offsets, model parameters, workspace and other runtime allocations are excluded. These components are not complete process totals.

d	Primary	Residual	ID + corr.	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

transposes it, and then repacks; each obsolete copy is freed. The two-pass builder scatters directly into the subspace-major array before the same final repack. The search-time lane mapping is discussed in Section 5.3.

Residual codes remain row-major, with $b_r = \lceil dB_r/8 \rceil$ bytes per vector. In the eight-bit configuration, coordinate t of row p is at byte offset $pb_r + t$. In the four-bit path it is in byte $pb_r + \lfloor t/2 \rfloor$, with even coordinates in the low nibble and odd coordinates in the high nibble. A four-byte correction per physical row stores

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)}) \rangle. \tag{4.8}$$

Its value must move with that row's codes during gathering or scattering. Residual access then reconstructs the same object that primary screening identified, even though the two code arrays use different layouts.

This separation favours the workload's two access patterns: primary codes are scanned broadly in list order, while complete residual rows are read for selected positions. Retaining raw FP32 vectors on the device would provide exact reranking but consume $4Nd$ additional bytes and change the reported search design. This index instead refines compressed reconstructions, with the quality consequences evaluated in Chapter 7.

4.4 Resident Storage, Workspace and Index Lifecycle

4.4.1 Stored bytes and allocated bytes

The eight-bit-residual configuration has an int8-scale resident representation. For one-byte primary subspace codes, its row payload is

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

The extra eight bytes store a four-byte ID and FP32 correction. At $M = d/8$, $B_r = 8$, codes occupy $9d/8$ bytes: 12.5% more than an SQ8 vector's d bytes. A graph-free hierarchy is therefore a design choice, not evidence of a unique compression advantage over int8 alternatives.

The CAGRA comparison uses $d + 4d_g$ bytes at graph degree $d_g = 32$, consistent with the harness's 32-bit neighbour IDs. On Stella, JHQ's row payload is 19.20 GiB, while a successful degree-32 CAGRA int8 record reports 19,656 MiB (19.20 GiB) of index allocation. The near equality is useful context despite their different accounting boundaries. Chapter 7 reports both int8 and fp32 allocations alongside search quality and cost. JHQ's distinctions are graph-free construction and a query-adjustable residual-access budget; lowering α leaves resident residual codes intact.

Table 4.2. Object lifetimes. Sizes describe named arrays, rather than all allocations of a CUDA process. Construction phases are expanded in section 4.5.

Object	Producer	Location	Lifetime	Bytes / scale
Input / samples	Loader / harness	Host	Build input; external ownership	$4Nd$ raw; sampled subsets
Transform / models	QR / training	Host, GPU	Model and index lifetime	$4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1)$ basic GPU arrays
Trained cache	Training	Host file	Across processes; rebuild still needed	Serialised model state
Input staging	Add pipeline	Pinned host, GPU	During add	$4n_b a_{\text{in}} d$ host; $8n_b a_{\text{in}} d$ GPU floats, plus scratch
Assignment products	IVF assignment	GPU	During add; freed before gather	$4n_b n_{\text{list}} a_{\text{assign}}$; independent row capacity
IDs / offsets	Sort / lower bound	GPU	Completed snapshot	$4N + 4(n_{\text{list}} + 1)$
Codes / correction	Encode / reorder	GPU	Completed snapshot	$N(M + b_r + 4)$
Order / inverse	Sort / inversion	GPU	Construction only	$4N$ per array; phase dependent
Query workspace / graph	Search / capture	GPU, host	Allocations reused; batch contents replaced	Capacity dependent; graph references resident arrays

The basic FP32 model arrays require $4d^2$ bytes for the transform, $4MKD_s$ for the materialised primary codebook, $4MK_r$ for residual codebooks and $4n_{\text{list}}(d+1)$ for coarse centroids and norms. Derived centroid copies and encoder levels add allocations in the selected path. Query workspace includes two FP32 query matrices $[b_{\text{cap}}, d]$, centroid scores $[b_{\text{cap}}, n_{\text{list}}]$, LUTs, probes and survivor buffers. The optional residual LUT would add $4b_{\text{cap}}dK_r$ bytes; default refinement computes residual scores directly. Group scratch follows equation (5.13) and can retain a larger capacity than the current scan uses: recorded 128-probe samples reserve about 93.8 MiB even on the query-centric path.

4.4.2 Reuse, recovery and invalidation

The cache saves the rotation, codebooks and centroids. Its key mixes model parameters and a strided checksum of the training sample, but omits the code version and a checksum of every database byte. Reproducing a build therefore requires model provenance and explicit cache invalidation when implementation or training data change. The search experiments reuse trained states; cold-build records construct separate instances. A close recall result across those instances cannot establish byte-identical models.

A cache hit still executes constructor QR and library warm-up before the training call checks the cache, then loads/uploads the model, encodes rows and builds lists. Stella's 6.67 s add record is one component of that recovery path, not a measured process-restart time. The add call includes its initial workspace allocation; schedule calibration and search warm-up lie outside that isolated add number.

At fixed k, changing α changes the survivor budget without retraining or adding vectors. Search grows insufficient workspace before capture, and recaptures when the actual batch size, survivor count, probe count, refinement lanes, scan groups or shared-bound setting changes. Calibration can bypass capture while trying budgets. Releasing the index destroys its graph and device allocations; buffers are index-owned and reused across the sequential calls evaluated here.

Table 4.3. Analytical row-array occupancy (GiB), $d = 1{,}024$, $M = 128$, $B_r = 8$. BGE uses one pass; Stella uses two. The one-pass Stella column is counterfactual capacity analysis. Pipeline, model, offsets and runtime costs are excluded from these phase values.

Dataset	Final row payload	One-pass gather	Two-pass repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

4.5 Construction Cost and Tested Scale

4.5.1 Bounded input batches and two-pass construction

Rotating all N raw vectors at once requires two FP32 device arrays, together $8Nd$ bytes. The builder instead stages bounded batches through pinned host memory and device raw/rotated buffers. For n_b pipeline buffers, configured add rows a_{add} and per-array byte budget S_{stage}, the source stages

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

Three buffers and the default 128 MiB budget give at most 768 MiB of input/rotated device floats. Assignment products and encoder scratch are separate allocations.

The one-pass path encodes input-order primary/residual arrays, then gathers them into list order. Its temporary and final compressed copies coexist. When the capacity rule predicts insufficient memory, two-pass construction first assigns lists, sorts IDs and builds offsets, then revisits the input to rotate, encode and scatter directly into final positions. This trades another input pass and rotation for fewer simultaneous compressed copies. In the two-pass path, default assignment can compare raw inputs with back-transformed centroids; orthogonality establishes the corresponding real-arithmetic identity.

4.5.2 Phase occupancy and the process peak

The source's one-pass branch estimate is

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes}. \tag{4.11}$$

Its N-dependent term is the exact payload of ten arrays simultaneously live during gather/transpose: input-order primary, residual and correction; assignments and sort order; final IDs, residual and correction; and vector-major and subspace-major primary copies. These sum to $3NM + 2Nb_r + 20N$ bytes. The extra 1 GiB is an allowance used by a capacity heuristic, not an exact accounting of all other allocations.

In two-pass construction, the final repack simultaneously holds IDs, residuals, corrections and old/new primary arrays:

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

This is exact for those arrays at that phase. Earlier phases also hold pipeline scratch and, during scatter, a temporary inverse permutation. Consequently the repack expression is not by itself the total peak of the two-pass builder. Model arrays, sort/allocator scratch, runtime context and the later workspace allocation must be included when taking the maximum over the whole lifecycle.

Table 4.3 quantifies the compressed-copy saving. Stella's cold log selects two passes and prints 41.6 GiB for equation (4.11), against 30.6 GiB available at the tail. The separately sampled resident process totals, including context and query workspace, are 11.93 GiB for BGE and 20.44 GiB for Stella. Search working sets depend on batch/probe/candidate capacity as described in section 4.4. Thus static rows, construction phases and searchable resident memory have explicit, different accounting boundaries. The supplementary 20 ms device samples observe a 26.08 GiB BGE peak near the start of add, and a 22.58 GiB Stella peak during repack. These are sampled device totals including context; shorter-lived peaks can be missed.

Assignment-product capacity uses an independent configured row count a_{assign}, selected by `JHQ_ASSIGN_BATCH` or the source default: 32,768 for at least 8,192 lists, otherwise 8,192.

$$S_{\text{dots}} = 4 n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

Allocation capacity need not shrink with a_{in} or the rows in a particular call. Table B.3 checks four defaults: the 1,536/3,072-dimensional cases allocate 3 GiB, despite different staging rows. BGE/Stella still allocate three 4 GiB arrays, or 12 GiB, near add entry. The 23.05 GiB gather payload describes a later lifetime: adding 12 GiB to it would not give the actual peak. Peak modelling must take the maximum over stage-specific live arrays, including pipeline scratch. Stella's smaller two-pass repack payload, plus model/context, explains its lower observed peak despite its larger database.

4.5.3 Construction measurements

Section 7.7 evaluates these phase models with repeated empty-cache builds, broader startup intervals and sampled device peaks, alongside baseline construction and large-dataset search. Their timing boundaries follow section 3.3; the construction algorithm and allocation lifetimes above determine which costs can overlap.

4.6 Index Representation and Safety Contracts

Representation contracts protect both numerical geometry and the joins between encoded fields. Their practical importance is illustrated by a historical large-sample failure: a 32-bit $n_t d$ product overflowed at 2^{31}, produced zero primary scale, and collapsed the primary codebook. A later run initially loaded that model from an older cache because its key omitted the code version. Correct coarse routing alone could not detect the broken primary representation. The corrected implementation chunks the norm computation and accumulates in double. This motivates checking the model, encoded rows and placement separately.

4.6.1 Completed representation evidence

The 13 October residual-bound pilot exported trained state and encoded database arrays, including the actual Π, primary/residual codebooks, IDs, offsets and corrections. The exporter decodes the packed word layout into logical vector-major codes. The pilot reports reusing the arms6 trained cache rather than retraining. Its three complete databases are Vogue (932,328 rows), arXiv (2,253,000) and OpenAI-3072 (999,000).

The codebook audit found zero separability mismatches, scalar-level mismatches and unsorted residual-codebook pairs. Independently decoding the exported primary/residual codes, an FP64 loop checked $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ over every indexed row. Across the three databases, maximum

absolute correction errors range from 1.28×10^{-7} to 2.77×10^{-7}. Sampled GPU-returned top-10 scores have maximum relative errors from 2.38×10^{-7} to 4.85×10^{-5}, using $|s_{\mathrm{GPU}} - s_{64}|/s_{64}$ with the query-norm constant included. These are checks of the exported representation, not re-encoding from the original input vectors. The repository supplement retains the per-dataset diagnostics.

Across seven workloads and 896 query instances, the pilot additionally checked the proposed residual bounds against FP64 reconstructed distances for 72,800,590 routed candidate instances, with zero reported violations. That population also supports the FP64 composite-score identity check; it is different from comparing GPU-returned scores, which considers only top-10 slots. The historical harness skips returned IDs absent from the routed candidate map, so its maximum error alone is not an exhaustive output-membership certificate.

Offline comparison found identical encoder, primary-training, residual-training and index-construction translation units between the pilot and the evaluated implementation after namespace normalization and removal of the pilot-only export method. This supports applying the pilot's checks to the shared representation path. It does not establish that every later process used byte-identical trained state or verify the large-dataset two-pass path: the audited databases are smaller. The per-workload values, raw-summary checks and source correspondence are reproducible with `scripts/audit_ch4_review.py`.

4.6.2 Independent input and placement audit

The supplementary exporter audit uses the shared construction path on Vogue, arXiv, OpenAI-3072 and Stella. Offsets are monotone with endpoints 0 and N; IDs form a permutation of every database row in all four. FP64 products of the stored rotation give maximum entrywise $\|\Pi\Pi^{\mathsf{T}} - I\|$ errors of 3.96–4.34×10^{-7}. Sampled norm checks use FP64 rotation, rather than the GPU GEMM, and therefore measure the stored matrix's geometry. Table 4.4 summarises sampled recoding.

Table 4.4. Original-input FP64 recoding audit. Flips count coordinate decisions; the final column is the largest residual-boundary distance among mismatches. Structural checks cover all rows; recoding is sampled.

Dataset	Rows	Orth. error	Primary flips	Residual flips	Max margin
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

At 768/1,024 dimensions, mismatches lie within 1.12×10^{-7} of decision boundaries. OpenAI-3072 differs materially: residual mismatches affect 2.68% of checked coordinates, up to 2.67×10^{-5} from a boundary. This agrees with its TF32 rotation and prevents treating all datasets as FP32-equivalent. Residual comparisons condition on the stored primary digit, avoiding counting a primary flip twice.

For the three smaller databases, forced one-/two-pass builds of the same model have identical primary and residual bytes when joined by ID. Corrections differ by at most 8.94×10^{-8}; list assignment differs for 1 Vogue, 1 arXiv and 40 OpenAI rows. Stella validates the natural two-pass path only. IVF-centroid membership remains unverified because centroids are absent from the export; these structural checks do not prove the assignment decision itself.

Table 4.5. Representation contracts after the pilot and supplementary input audit. Coverage separates stored geometry, sampled arithmetic and row placement.

Boundary	Required property	Evidence / remaining check
C0: geometry	Common transform with bounded floating-point isometry error	Stored-matrix orthogonality checked on four datasets; TF32 GPU arithmetic distinguished
C1: codebooks / correction	Cartesian levels, sorted residual codebooks, correction matches decoded codes	Pilot checks completed on three databases; all-row correction errors in section 4.6
C1: input encoding	Stored codes encode the intended original input row	FP64 original-input recoding on four samples; boundary and TF32 discrepancies quantified
C1: placement	Offsets cover rows; IDs form a permutation; all fields follow it	All-row offsets/ID checks; three ID-joined path comparisons; centroid membership unverified
Execution	Valid indexing, synchronisation and score-order contracts	Archived query and sanitizer checks; finite coverage, detailed in section 7.4

4.6.3 Contract coverage and remaining checks

The constructor requires M to divide d and D_s to divide B; word packing requires $M \bmod 4 = 0$. Query-side LUT verification checks factorisation against actual codeword slices. Positions and IDs are signed 32-bit fields, while large counts and byte-address products require wider arithmetic. The input contract assumes finite vectors and representable counts, and reserves invalid sentinel positions.

The audit consumes the exported rotation and codebooks, rather than a matrix regenerated from its seed. Remaining work is narrower: independently verify membership against exported IVF centroids and test deliberately empty lists and adversarial tails. The strict computed-score CPU selection reference has now been completed on the T4 grid (section 7.4.1); it is separate from FP64 input recoding. Chapter 5 conditions selection on the resulting objects; Chapter 7 evaluates quality and cost.

Chapter 5

Representation-Aware GPU Search Execution

This chapter addresses RQ2: how JHQ's compressed representation can support correct and efficient GPU execution. The transform, Cartesian primary codebook and residual hierarchy are inherited from JHQ [15]; the focus here is their execution. The argument follows the query path from routing through primary selection to residual scoring. For each mechanism, it separates the mathematical target, implementation conditions and resource trade-offs, with the corresponding measurements in Chapter 7. Workload-level refinement-budget selection is developed separately in Chapter 6.

5.1 Query Path and Four Distinct Ranking Objects

For a fixed index, a query is transformed to $q' = \Pi q$, routed to selected inverted lists, screened using primary codes and refined only on the surviving candidates. The final selector returns k database IDs and their computed scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

Here μ_i is an IVF centroid in transformed coordinates, $\widehat{p}(x)$ is the primary reconstruction and $\widehat{r}(x)$ is the decoded residual. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

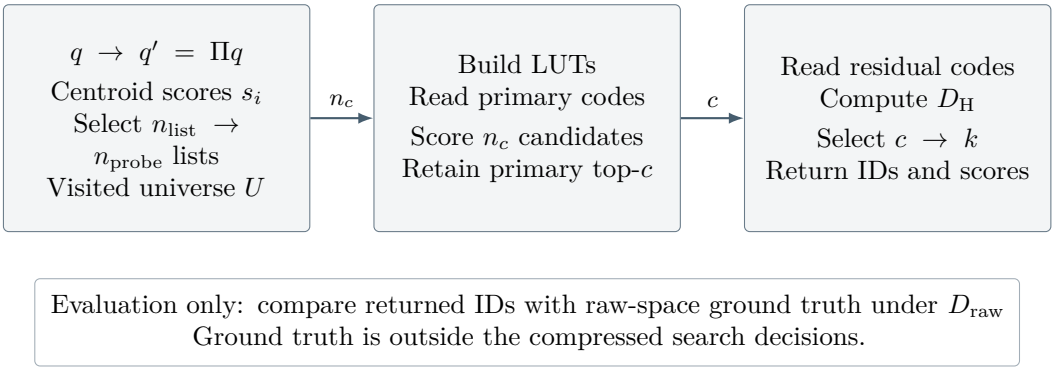


Figure 5.1. Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

Table 5.1. Query-stage contracts. $c_v = \min(c, n_c)$ counts valid primary survivors. Exactness refers to the stated computed scores; routing and quantised reconstruction determine raw-space approximation.

Stage	Input → output	Data read	Main work	Order / exactness
Transform	$q \to q'$	Query; transform	Apply Π	Transform (precision in section 7.2.3)
Probe	$n_{\text{list}} \to n_{\text{probe}}$ lists	FP32 centroids; list lengths	Score; radix; rank count	(s_i, i); exact probe selection
LUT	$q' \to$ tables	Primary codebook	$MD_s 2^{8/g_{\text{LUT}}}$ terms	Real-arithmetic factorisation; FP32 rounding
Primary, $g_{\text{scan}} = 0$	$n_c \to c_v$	Packed primary codes; LUT	$O(n_c M)$ scoring; compaction	Exact distance-order selection; arbitrary cutoff ties
Primary, $g_{\text{scan}} \geq 1$	$n_c \to c_v$	Same primary representation	Score; compact; merge	(D_{P}, p); exact under stated premises
Refine	$c_v \to c_v$ scores	Residual codes; correction	$O(c_v d)$	Computed D_{H}; lane-dependent FP32 association
Final	$c_v \to k$	Refined scores; IDs	$O(kc)$ slot scans	$(D_{\text{H}}, \text{ID})$; exact with k finite valid entries

Routing defines the visited universe U of n_c candidate occurrences; primary screening retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ for residual scoring. Table 5.1 records each stage's inputs, reads and costs; sections 5.2 to 5.7 develop their implementations.

The ordering and exactness columns of table 5.1 attach each claim to its score and tie rule. Chapter 7 tests survivor sets, ordered outputs and raw-space recall separately.

Table 5.1 gives the stage contracts. The E1/budget harness explicitly selects 16 refinement lanes and scan policy -1 (scheduling among 0/2/4/8), versus library defaults of scalar refinement and policy 0. Shared bounds are enabled but used only at $g_{\text{scan}} > 1$; the harness calibrates scheduling with 512 database pseudoqueries, a step the library does not perform automatically. Component ablations retain their stated overrides; table A.1 maps mechanisms to recorded implementations.

5.2 Exact Cartesian LUT Factorisation

The broad scan evaluates primary reconstructions through asymmetric distance computation: the query remains FP32 while a stored code selects reconstructed coordinates [15, 16]. For transformed query q', the ordinary table for primary subspace m is

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

With an eight-bit code, this requires 256 entries per subspace. The opportunity here comes from the codebook's internal Cartesian structure, not just the fact that the vector is partitioned into PQ subspaces.

The analytical builder uses two scalar levels per coordinate when $K = 256$ and $D_s = 8$:

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\mathrm{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a\sum_{j=0}^{d-1} s_j(x)q'_j. \end{aligned} \tag{5.3}$$

Here Φ is the standard normal CDF and $s_j(x) \in \{-1, +1\}$. Each byte encodes eight coordinate signs, with higher bits representing earlier coordinates. For $a > 0$, primary ranking is equivalent to an inner-product ranking with that sign vector. The two-level Cartesian product also explains why four- and eight-way coordinate splits remain algebraically exact. This describes the analytical construction; a learned, non-Cartesian replacement requires a new compatibility check.

5.2.1 Factorisation without changing the quantiser

For the tested $D_s = 8$ Cartesian codebook, the high nibble h selects the first four reconstructed coordinates and the low nibble l selects the remaining four. Thus $\mathcal{C}_m[16h + l] = (A_m[h], B_m[l])$, with no cross-dependence between the two coordinate groups. Define

$$T_m^{\mathrm{hi}}[h] = \|q'_{m,\mathrm{hi}} - A_m[h]\|_2^2, \qquad T_m^{\mathrm{lo}}[l] = \|q'_{m,\mathrm{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

Squared-L2 additivity over disjoint coordinates gives

$$T_m[z] = T_m^{\mathrm{hi}}[z \gg 4] + T_m^{\mathrm{lo}}[z \,\&\, 15]. \tag{5.5}$$

Only 32 entries are stored, rather than 256. This preserves the reconstruction and the real-arithmetic score; no code retraining, extra quantisation or change in the candidate universe is involved.

The builder reads representative codewords $16h$ and l from the existing full codebook and evaluates only their corresponding coordinate slices. The zero digits in the other slice are irrelevant. This is valid only when all codewords sharing a nibble share that slice. The `verify_split_lut` check compares the stored codeword slices against their representatives and rejects incompatible codebooks. An arbitrary learned 256-centroid PQ subcodebook need not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The same-candidate FP32 audit in section 7.3 measures numerical and ranking differences between table forms, including changes after identical primary survivor sets.

5.2.2 Grouping, construction work and resource cost

The table-grouping ablation generalises the split to $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has $2^{8/g_{\mathrm{LUT}}}$ entries; the total entries and FP32 storage per query are

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}}2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4Mn_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

Independent entry construction evaluates $D_s 2^{8/g_{\mathrm{LUT}}}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

Table 5.2. Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

The table is built in global workspace and staged cooperatively when its complete shared-memory footprint fits the queried per-block opt-in limit $S_{\max}$. For candidate-buffer capacity C_{buf}, the ordinary exact scan and the grouped scan respectively require

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

These formulas concern the evaluated scan without optional fused refinement or early-exit scratch. The code selects shared lookup exactly when the corresponding footprint is at most $S_{\max}$.

In the table sweep, $c = 1{,}000$, block sizes are 512 or 1,024, and $C_{\mathrm{buf}} = 2{,}048$. Full tables span 96–384 KiB, versus 12–48 KiB for two groups. NVIDIA specifies a 99 KiB per-block limit for compute capability 12.0 [24]. With this limit, the full-table footprint exceeds capacity even at $M = 96$ (114,696 bytes in the query-centric path), whereas the largest two-group footprint is 65,560 bytes in the grouped path. Thus the source-level placement rule sends full tables to global memory and two-group tables to shared memory in these configurations. This is a deduction from the launch rule and published limit; a device-attribute printout was not retained in the reviewed records.

Four and eight groups both store 16 entries per subspace but perform more lookups and combinations. Extraction, lookup dependencies, staging, synchronisation and residency jointly determine the measured trade-off. Multiple scan groups may stage the same table independently.

The $16 + 16$ representation is a nibble LUT for an eight-bit sign code. Related small-table execution appears in PQ Fast Scan, Quicker ADC and IVF-RaBitQ [1, 2, 31]. This algebraic connection does not identify JHQ's two-level score with RaBitQ's distance estimator. The contribution here is GPU execution of JHQ's primary/residual structure, including its scan, selection and refinement, with a measured table-grouping choice. Section 7.3 reports that two groups lead in 22/24 cells, with the remaining margins below 1%.

5.3 Packed Codes and Candidate-Parallel Memory Access

LUT factorisation reduces the query-specific table; code layout determines how lanes fetch each candidate's table indices. These are separate optimisations. The scan assigns neighbouring lanes to neighbouring candidate occurrences, so its broad phase benefits from storing codes by subspace across candidates rather than storing each candidate's full primary code contiguously.

5.3.1 Address mapping and four-byte packing

Let o_ℓ be the start of inverted list ℓ, r the row within that list and $p = o_\ell + r$ its physical position. If $Z[m, p]$ denotes the eight-bit code for primary subspace m, the byte-oriented scan copy stores it at byte offset $mN + p$. Candidate-major storage would instead use $pM + m$.

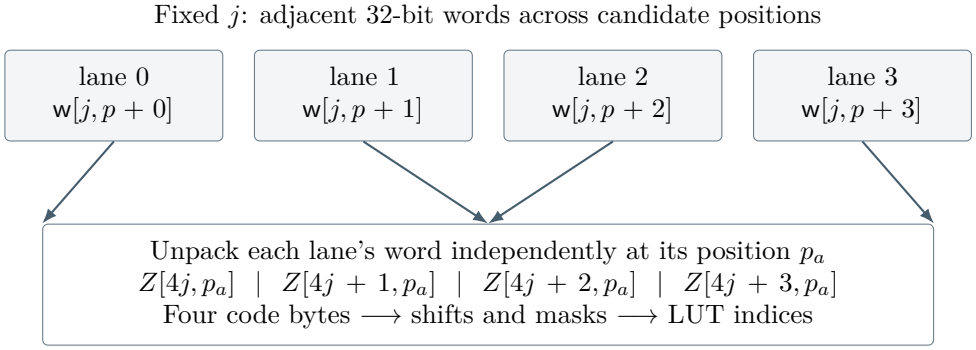


Figure 5.2. Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

When lanes visit consecutive positions within a list, the former makes their accesses at fixed m adjacent. At list boundaries, positions can jump, so the mapping does not guarantee one contiguous transaction for every warp.

The packed representation combines four subspace codes of the same candidate into one unsigned word:

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t, p], \qquad \mathrm{offset}_{\mathrm{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

The four codes are recovered exactly by $(\mathsf{w}[j,p] \gg (8t)) \,\&\, 255$. Each recovered code then supplies the high and low nibble indices in equation (5.5). Packing four subspaces is distinct from Cartesian splitting of one subspace's eight-bit label, and from packing codes for four different candidates.

The build path explicitly repacks the $[M, N]$ byte array into $[M/4, N]$ words. Merely reinterpreting the original byte array as `uint32_t` would combine consecutive candidate bytes rather than the required subspaces. The implementation requires M divisible by four and rejects a partial final word. The repack reads and writes NM code bytes once at construction, with old and new storage briefly coexisting; it is outside the search timing boundary.

5.3.2 What packed access saves, and what it preserves

At the source level, a candidate now obtains four codes with one word load instead of four byte loads, followed by shifts and masks in registers. Within a contiguous list segment, 32 lanes request a logical 128-byte word window. Actual transactions depend on alignment, active lanes and the memory hierarchy. A fourfold reduction in source-level load operations does not establish a fourfold reduction in transactions or elapsed time.

The logical payload remains M bytes per candidate: this repacking does not further compress the index. Both byte and word alternatives already organise the scan copy by subspace or subspace group. The historical on/off comparison therefore measures the net effect of the packed layout and its load/unpack path, not the isolated contribution of transposing candidate-major storage. It also cannot uniquely separate instruction count, addressing and cache effects.

Section 7.3 evaluates the packed path's throughput, recall and variability against the byte path.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Primary screening must expose enough GPU work without losing a candidate that should reach residual refinement. A query-centric scan launches one block per query; a small batch therefore offers few blocks even when each query visits many vectors. GPU selection is a substantial design problem in its own right [17]. The grouped-scan component partitions the visited candidates across blocks and combines their survivors. Its contract is exact selection under the computed primary scores. IVF routing and the quantised representation remain approximate.

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index stays fixed during search. For $g_{\text{scan}} \in \{1, 2, 4, 8\}$, block (q, j) owns the interval

$$U_j = \left[\left\lfloor \frac{j n_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1) n_c}{g_{\text{scan}}} \right\rfloor \right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = (D_{\text{P}}(q, x), p(x)), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database ID used to break final-result ties. With non-NaN scores, this defines a total order on distinct candidates. All local comparisons and merge decisions use that same order. Partition-independent selection additionally requires that each candidate receive the same computed score in every compared path; the ordering rule alone cannot remove floating-point scoring differences.

5.4.2 Exact local retention and global coverage

Write $\text{Top}_c(V)$ for the first $\min(c, |V|)$ candidates of V sorted by K_q, with $c = \max\{k, \lceil \alpha k \rceil\}$. Each independent group retains its full local top-c, not a share c/g_{scan}. This distinction is necessary: with $c = 4$ and two groups, all four global winners may belong to one group. Retaining only two from each group loses winners even though the combined capacity is four.

Lemma 5.1 (Local-buffer retention)**.** *Let V_t be the candidates scored by a block through chunk t, and B_t its valid buffer after any compaction or filtering at that point. Assume fixed complete*

keys, sufficient append capacity and exact top-c compaction. Let β_t be the minimum of the complete shared bounds observed so far, including the initial top sentinel. Then

$$\mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

With sharing disabled, the bound is the sentinel and the buffer contains the whole prefix top-c.

Proof. The empty prefix satisfies the invariant. A new candidate can fail the local admission threshold only if c previously retained keys precede it; those keys remain predecessors in every later prefix. Rejection by an observed shared bound excludes only keys above β_t. An old candidate outside a prefix top-c cannot improve its rank when the prefix grows. Thus every eligible new prefix winner is either retained already or admitted. Exact compaction preserves it: fewer than c members of the whole prefix, and hence of the buffer, precede it. Filtering removes only keys above an observed bound. Induction over appends, compactions and filtering proves the claim. The running minimum is essential if reads may return different stale published values. □

Proposition 5.1 (Coverage and truncated merging)**.** *For fixed keys under a common total order, let U be the visited universe and V any retained subset. If $\mathrm{Top}_c(U) \subseteq V \subseteq U$, then $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$. Pairwise sorted merging with truncation to c preserves this result. In particular, for disjoint groups,*

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

The statement includes universes smaller than c.

Proof. Every global winner is in V, and every other member of V follows the global winners. Selection from V therefore recovers them. For the independent-group special case, let x belong to the global top-c and to group U_j. Fewer than c candidates precede x globally, so fewer than c precede it within U_j. Consequently $x \in \mathrm{Top}_c(U_j)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

The CUDA implementation maintains a shared-memory buffer of C_{buf} key pairs, where C_{buf} is a power of two and $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$ for a scan block of T_{blk} threads. A compaction sorts this buffer with a bitonic network, retains at most c valid entries and records the last retained key as the local threshold when c entries exist. Before that point the threshold is a sentinel above every valid key. Between compactions, only candidates preceding the threshold are appended. Its c witness candidates make the threshold a safe upper bound for the best c of the scanned prefix, even as further candidates arrive.

Each chunk contributes at most T_{blk} appends. After a chunk, the block compacts whenever the next chunk would exceed capacity. Following compaction at most c entries remain, so $C_{\mathrm{buf}} \geq c + T_{\mathrm{blk}}$ provides space for the next chunk; otherwise the explicit capacity check provides it. This invariant prevents overflow without restricting how many competitive candidates one thread may encounter. A per-thread fixed-size retention buffer does not provide the same

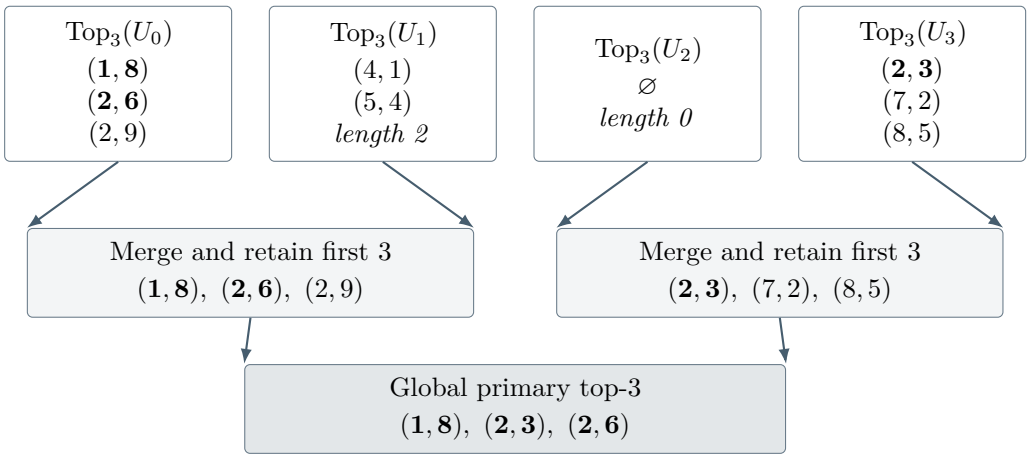


Figure 5.3. Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $g_{\text{scan}} = 4$. Entries are (D_{P}, p) keys, not measured data. Bold entries are the final global winners. A short group contributes its valid prefix; an empty group contributes no entries. Every merge uses the same key order and retains at most c entries.

coverage guarantee. Together with exact bitonic compaction, this capacity argument discharges the implementation premises of Lemma 5.1. With sharing, every observed bound is above the global cth key by Proposition 5.2; the lemma therefore retains every global winner belonging to this block, even when its full local top-c is filtered. The general coverage proposition then justifies the merge.

An alternative is to materialise all n_c score/position pairs and then invoke a separate selector. That preserves the target but adds an $O(n_c)$ intermediate array and its writes. Here packed-code scoring feeds a bounded local buffer directly, exchanging repeated local sorting for $O(g_{\text{scan}}c)$ intermediate output. Register-based selection is another established GPU design [17]; this campaign evaluates the chosen scan variants against the archived reference.

5.4.3 Merge implementation, storage and boundary cases

For $g_{\text{scan}} > 1$, one 256-thread block per query performs $\log_2 g_{\text{scan}}$ rounds of pairwise merging. A pair with valid lengths a, b emits $\min(c, a + b)$ keys. Merge Path partitions this output into contiguous thread slices: a binary search finds the input offsets for each slice, then the thread merges its slice sequentially [13]. All comparisons use equation (5.10); block barriers separate rounds. Figure 5.3 illustrates the truncation invariant. The tree emits at most $c(g_{\text{scan}} - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

The two global score/position workspaces have capacities $[b_{\text{cap}}, g_{\text{scan}}, c]$ and $[b_{\text{cap}}, g_{\text{scan}}/2, c]$ and alternate between rounds. With four-byte scores and positions, their combined payload is

$$S_{\text{pairs}} = 8b_{\text{cap}}c\left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2}\right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

Group counts require another $4b_{\text{cap}}g_{\text{scan}}$ bytes; shared bounds add $8b_{\text{cap}}$ when provisioned. These are additional to final survivor arrays and other search workspace. Allocation grows before graph capture and is retained for reuse, so choosing a smaller g_{scan} later does not necessarily release the reserved memory. The merge keeps only list lengths in shared memory rather than all $g_{\text{scan}}c$ keys.

Counts, rather than sentinel scores alone, delimit local valid prefixes. Compaction pads its sorting buffer with $(+\infty, \text{INT_MAX})$; valid positions must be below that reserved position.

Empty groups have count zero. The final survivor array pads unused slots with $(+\infty, -1)$, and refinement skips negative positions. These two sentinels serve different purposes: the negative padding position must not participate as a valid key in a merge. $g_{\text{scan}} = 1$ writes directly to the survivor arrays and omits the merge.

The older query-centric path is selected by the implementation label $g_{\text{scan}} = 0$, meaning one block per query, not zero blocks. Its default primary selection is exact by score, but its distance-only cutoff can retain a different member of a tie from the complete-key path. The lossy `K_LOCAL=4` alternative requires `JHQ_EXACT_TOPCK=0` in a build that permits it; paper-faithful builds force exact selection. It is not the measured default. For this distance-only path, the buffer argument uses a total preorder: after final compaction it retains all candidates strictly below the prefix's cth distance and enough candidates at that distance to fill $\min(c, |V_t|)$ slots. A rejected distance has c retained witnesses with no larger distance; exact distance sorting preserves this rank property at each compaction. Tied identities need not be unique. The strict CPU reference in section 7.4.1 now checks the same computed score/position target: all 55,296 grouped query instances match its complete-key order. All 13,824 one-block instances satisfy the distance preorder; 56 choose different cutoff-tie members. This complements the conditional buffer/merge argument with finite-case implementation evidence. The reference sorts GPU-produced scores, rather than recomputing original-space distances on the CPU.

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Full local top-c retention establishes correctness, but can produce $g_{\text{scan}}c$ intermediate survivors for only c global outputs. The shared-bound component reduces this excess selection work. The scheduler then chooses how much scan parallelism to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_j, satisfies $K_{(c)} \preceq \tau_j$. There are already c candidates no later than τ_j in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

Proposition 5.2 (Safe rejection by a witnessed bound)**.** *Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.*

Proof. Every published bound is at least $K_{(c)}$, and the minimum of such bounds is also at least $K_{(c)}$. A global winner satisfies $K_q(x) \preceq K_{(c)} \preceq \tau$, so it is never rejected. An older bound remains an upper bound and merely permits extra work. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

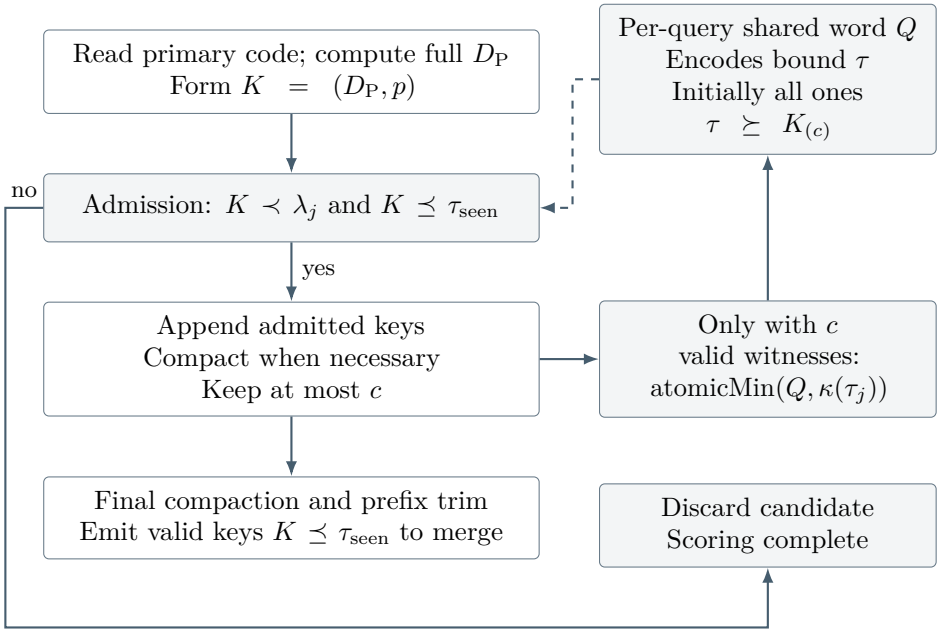


Figure 5.4. Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

This last distinction matters. With sharing enabled, a group need not return its own full local top-c: it can discard locally competitive candidates already certified as globally uncompetitive. Correctness requires the union still to contain every global winner. The truncated merge argument of Proposition 5.1 then applies to that union. Candidates equal to the shared bound remain eligible, including the witness that established it. The implemented admission condition is

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

where λ_j is the local compaction threshold and τ_{seen} is the group's last observed shared bound.

5.5.2 Publication, filtering and implementation limits

For nonnegative FP32 scores generated as sums of squares, the code packs the distance bits and nonnegative position into an unsigned 64-bit key:

$$\kappa(D, p) = 2^{32}\,\text{uintbits}_{32}(D) + p. \tag{5.16}$$

Unsigned integer comparison then agrees with the lexicographic order. This relies on positive-zero encoding and excludes NaNs and negative scores; it is not a general signed-float encoding. Positive infinity orders after finite scores, and the all-one initial word orders after valid encodings. Invalid positions are excluded from bound publication. These are score-domain and representation preconditions, not properties inferred from unchanged recall.

One thread per block reads the query's bound for each chunk. After compaction, it publishes the local cth key through 64-bit `atomicMin` when enough valid entries exist. At scan completion, another publication and a binary search retain only the sorted prefix no later than the observed bound. There is no cross-block barrier or wait for other groups. A late update can leave extra

entries in another group's output, which the exact merge safely removes. Bounds are reset before every search in the same stream, and the merge runs after the scan kernel finishes.

The archived binary's extracted PTX contains scalar `ld.volatile.global.u64` and `atom.global.min.u64`. PTX gives compatible, equally sized strong accesses single-copy atomicity [26]; these aligned 64-bit operations therefore supply the whole-key premise at PTX level. Reassembling that PTX with CUDA 13.0 `ptxas` for `sm_120` produces one 64-bit strong system load and one GPU-scope atomic-min reduction, recorded in section A.4. The toolkit disassembly corroborates the contract; the driver JIT's actual SASS was not captured. CUDA C++ `volatile` alone is not the basis of the argument [25].

Figure 5.4 also identifies what this optimisation saves. Every scanned candidate still loads its primary code and computes its complete primary score. The bound avoids some buffer appends, local sorts and intermediate writes, and shortens the lists consumed by the merge. It does not implement partial-distance early termination or reduce the number of primary scores evaluated. Its benefit competes with polling and atomic-update overhead. The observed paired speedup range includes a slight slowdown, as reported in Section 7.4.

5.5.3 Choosing parallelism without changing the budget

For batch size b, grouped scanning launches bg_{scan} blocks instead of b. This can expose otherwise unavailable parallelism, but duplicates table staging and local selection and introduces a merge. The fixed-group design therefore cannot be uniformly preferable. Smaller static quotas would reduce intermediate storage, but require a certificate or fallback to preserve coverage; the pilot's concentrated survivor distribution motivated using witnessed thresholds instead (Section 7.4).

The implemented scheduler uses a fill proxy and a workload-skew statistic. Let T_{blk} be scan threads per block, H the number of SMs times the maximum resident threads per SM, and $v = \operatorname{sd}(n_c)/\operatorname{mean}(n_c)$ the candidate-count coefficient of variation. Its rule is

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations. The one-time pseudoquery calibration is excluded from the reported steady-state search times. Supplementary timing separates host gathering (0.81–7.04 ms), the first calibration call (6.00–27.77 ms) and repeat medians (2.72–18.13 ms). This startup cost is separate from refinement-budget calibration (section A.4).

When $g_{\text{chosen}} = 1$, the implementation selects the faster query-centric path, labelled $g_{\text{scan}} = 0$. It leaves the index, probe count and refinement budget unchanged, but inherits that path's distance-only cutoff ties. The common-total-order premise must therefore be checked before claiming identical survivor sets across all scheduler choices. Section 7.6 reports the separate integrated-output checks and the scheduler's measured gains and regret.

5.5.4 Observed execution paths

For the 170-SM RTX 5090, the published 48-warps-per-SM limit gives $0.6H = 156{,}672$ threads [24]. At batch 1,000 with $T_{\text{blk}} = 512$, even two groups require a fill proxy of 1,024,000. The fill rule therefore selects one group; only sufficient skew forces four groups in this case.

The main throughput campaign exercises both implementations, rather than uniformly using grouped scanning. Across 38 E1 run records, nine have $v \geq 0.3$, all Vogue (approximately 0.338–0.622). Their 132 run–budget cells contain 27 Vogue cells scheduled to four groups and 105 arXiv/OpenAI cells scheduled to the query-centric path. Thus the non-Vogue large-batch QPS results do not exercise grouped retention, shared-bound filtering or merging. They still include the LUT, packed access, lane-refinement and probe components.

For Vogue, scheduled-versus-query-centric checks report 3–15 queries with tie-only primary differences per 1,000, zero non-tie differences, and bit-identical final outputs in every recorded cell. The narrower 7–11 range applies only to $n_{\text{probe}} = 128$.

The budget campaign makes the calibration/evaluation distinction concrete: 10 of 12 held-out configurations use the query-centric scan (batch 500, or Stella's 499); the two Vogue configurations use four groups. At calibration size $S = 128$, eight use two groups, two use four, and two remain query-centric. Execution paths therefore differ in 8/12 configurations. Grouped selection principally serves smaller batches and skewed workloads; there is no universal batch-256 boundary. The offline recount and input hashes are recorded in `data/ch5/review-audit.json`.

The rule is a heuristic rather than an occupancy calculation or a latency oracle: it omits register pressure, shared-memory residency, budget-dependent compaction cost and merge cost. A frozen pseudoquery statistic can also misrepresent future queries. In the recorded campaign the largest regret occurs at batch 256, where a fixed-group alternative is about 1.217× faster. The design combines parallel candidate scanning, witnessed filtering and workload-dependent execution, with its gains and regret measured under a fixed refinement budget. Refinement and final top-k consume the resulting valid survivors as described in Section 5.6.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Only the primary survivors access residual codes. This preserves a broad, cheap scan over n_c candidates followed by a narrower $O(cd)$ scoring stage. Reading residuals for every visited candidate would instead move that coordinate work towards $O(n_c d)$. Lane cooperation accelerates the chosen survivors; it does not decide the refinement budget.

5.6.1 Reconstructed scoring and the query-only constant

Let $\widehat{p}(x)$ be the primary reconstruction and $\widehat{r}(x)$ the decoded residual in transformed coordinates. The encoder stores $\gamma(x) = 2\langle \widehat{p}(x), \widehat{r}(x) \rangle$. The implemented refined ranking score is

$$D_{\text{H}}(q, x) = D_{\text{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.19}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.20}$$

The second equality follows by expanding the squared norms. Thus the score orders hierarchical reconstructions by squared distance, but the returned value includes a query-only constant. It is neither a direct raw-vector distance nor an unshifted reconstruction distance. The constant

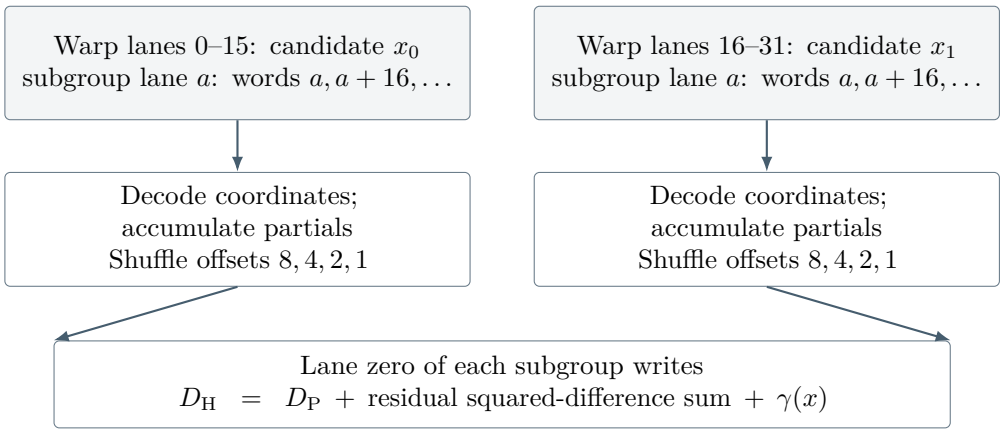


Figure 5.5. Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a+16, \ldots$. With eight-bit residual codes, each 32-bit word decodes four coordinates. Subgroup reductions combine partial scores before adding the retained primary score and correction. Candidate positions can be scattered even when each subgroup's word access is contiguous.

does not affect a fixed query's ordering. Reconstruction error remains even when selection under this score is exact [15].

Residual coordinates use scalar codebook entries shared within a subspace. The scalar path walks each candidate's dimensions and sums their squared differences from q'. Query coordinates and, when space permits, the residual codebook are staged in shared memory; the stored correction and primary score are added afterwards.

5.6.2 Mapping lanes to words and reducing partial sums

For $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$, a scan-independent refinement block assigns L adjacent lanes to a candidate. A 32-bit residual word holds $d_{\mathrm{word}} = 32/B_R$ coordinate codes, where B_R is the residual bit width. The tested eight-bit path has $d_{\mathrm{word}} = 4$; the four-bit path has $d_{\mathrm{word}} = 8$. Lane a reads word indices $a, a+L, a+2L, \ldots$, decodes their coordinates and accumulates a partial sum. A butterfly shuffle reduction combines the L partials, and lane zero writes the refined score.

The launch requires a word-aligned residual row and D_s divisible by d_{word}, so a word never crosses a subspace's codebook boundary. All lanes execute the shuffle sequence, including lanes assigned to invalid or out-of-range survivor slots; these contribute zero before the output is marked infinite. This avoids making warp participation depend on whether a particular candidate exists. At $L = 16$ a warp handles two candidates, rather than one candidate per lane.

The experiment separates two changes. L0 is the original scalar byte-load path; L1 uses word loads with one lane per candidate, preserving its accumulation order. L8/L16/L32 additionally distribute coordinate work across lanes. This shortens each lane's serial walk and provides useful work when c is small, but adds shuffles and reduces the number of independent candidates handled concurrently. Larger L is therefore not automatically better. Irregular survivor positions and shared/global codebook placement remain relevant costs.

5.6.3 Final selection and the numerical boundary

For at least k valid survivors with finite refined scores and unique database IDs, the final kernel performs exact top-k selection under $(D_{\mathrm{H}}, \text{database ID})$. One block per query loads the c scores and repeatedly reduces the minimum, emits its ID and score, and removes that score before the next round. All c slots are scanned at each round; there is no fixed per-thread candidate quota. This contract requires k finite valid entries. In a degenerate pool, selection sets the winning score to infinity but leaves its position valid: after finite entries are exhausted, the smallest valid ID can repeat with an infinite score. An entirely invalid pool returns invalid IDs and infinity. These cases lie outside the finite-pool contract used for the reported evaluation.

The final ID tie rule differs from the physical-position rule used for primary retention. An exact final selector cannot recover candidates discarded by routing or screening, nor remove quantisation error. Wider lane reductions also change floating-point association before selection, which can move near-tied candidates even when the real-arithmetic formula is unchanged.

Section 7.3 reports the load-only and cooperative controls, numerical differences, kernel times and end-to-end gains separately.

5.7 IVF Probe Selection as an Independent Primitive

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the FP32-valued computed routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i \rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The required output is the first $n_{\mathrm{sel}} = \min(n_{\mathrm{probe}}, n_{\mathrm{list}})$ centroid IDs in score order, using the smaller list ID at a tie, plus the prefix offsets of their list lengths.

The dot products inherit the cuBLAS math mode: in the measured v69, OpenAI rotation and centroid GEMMs use TF32 inputs with FP32 accumulation. The selector's exactness concerns these computed scores. Its subsequent FP32 `fma` and deterministic key ordering do not recover rounding already introduced by the GEMM (section 7.2.3).

5.7.1 Radix selection followed by deterministic ordering

The earlier bitonic selector repeatedly bitonic-sorted a growing candidate buffer. The radix replacement keeps one block per query but finds the cutoff by four most-significant-byte-first radix passes. Each pass counts only keys matching the prefix selected so far, then locates the bin containing the remaining order statistic. After four passes it has the cutoff key and the number of equal-cutoff entries still required.

The float mapping canonicalises both signed zeros to positive zero, then transforms their 32-bit representation u by

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

For non-NaN floats this preserves numerical order, including negative scores and infinities; canonicalisation makes equal zeros share a key. This differs from the nonnegative-score encoding in equation (5.16). NaNs are outside the selector's ordering contract. Every pass recomputes

scores with the same explicitly rounded `fma` expression, so changing the pass does not change its keys.

The histogram has 256 bins and 32 lane replicas, requiring 32 KiB for its counters alone. Replicas separate same-warp atomic updates to a popular bin; different warps can still contend. This exchanges shared storage and reduction work for less concentrated histogram updating, rather than eliminating all contention.

Collection first gathers every key below the cutoff. If too many keys equal the cutoff, an ordered rescan uses ballots and prefix counts to take exactly the required number in increasing list-ID order. Atomic arrival order therefore does not choose the final tied subset. For each of the n_{sel} selected entries, a rank-count step counts how many selected keys precede it under (s_i, i) and writes it to that rank. This restores deterministic sorted order without sorting all centroids.

Finally, a block prefix scan produces offsets of the selected list lengths and their total candidate count. When $n_{\text{probe}} > n_{\text{list}}$, the implementation pads the ID array by repeating the final selected ID but assigns those extra entries zero length. They do not scan the same list again. Nonempty centroid sets and a positive requested probe count are assumed.

5.7.2 Costs and separate correctness contracts

The histogram passes and collection revisit the centroid-score row; rank counting costs $O(n_{\text{sel}}^2)$ comparisons. The algorithm is consequently not a uniformly linear-cost substitute for every probe count. Large n_{sel} increases both ranking work and selected-entry shared storage, while the histogram reserves space even for small centroid sets. Radix selection is an established selection technique; the contribution here is its deterministic integration and measured execution trade-off for this query pipeline.

The primitive tests cover many ties, all-equal values, mixed signed zeros, mostly infinite values, extreme magnitudes and requested probe counts exceeding the list count. Their outcomes and real-query checks are reported in section 7.4.

The optional FP16 routing changes score precision and is evaluated as a separate approximation in section 7.4, alongside the cross-process pipeline-identity checks.

Chapter 6

Workload-Level Refinement-Budget Calibration

Chapters 4 and 5 establish a compressed index and its GPU execution. This chapter addresses RQ3: how much refinement work a workload needs, and whether selecting that budget repays its calibration cost. The policy compares unlabelled query outputs with a larger-budget reference and reuses one selected multiplier across batches. The method is analysed through its acceptance rule, conditional selection theorem, experimental controls and amortisation model. Chapter 7 reports the measured outcomes.

6.1 Why Fixed Refinement Budgets Can Be Wasteful

JHQ screens routed candidates with primary codes before reading residuals for a smaller survivor set [15]. For a query q, let $U(q)$ be the candidates in its selected IVF lists. The refinement multiplier sets

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha), |U(q)|\}} U(q). \tag{6.1}$$

Here C_α denotes valid survivors. The implementation reserves $c(\alpha)$ slots and can pad an underfull pool with sentinels. The analysis assumes at least k valid candidates. Raising α expands refinement capacity within the visited lists; raising n_{probe} changes the visited universe. Neither operation recovers information lost by the other automatically.

A large fixed α may repeatedly refine candidates that do not change the final top-k. A small value can exclude a useful neighbour before residual scoring. Which tradeoff is worthwhile depends on the query population, primary reconstruction, list coverage and refined-score ordering. One global default therefore need not suit every dataset–probe configuration. The evaluated grid spans $c = 20$ to 2,000 at $k = 10$. Changing it reuses the same trained index and stored residuals. Lowering the multiplier reduces residual rows accessed, rather than releasing the resident residual representation. Survivor buffers grow when capacity is insufficient and retain that capacity afterward, so a smaller budget is not a proportional reduction in allocated memory.

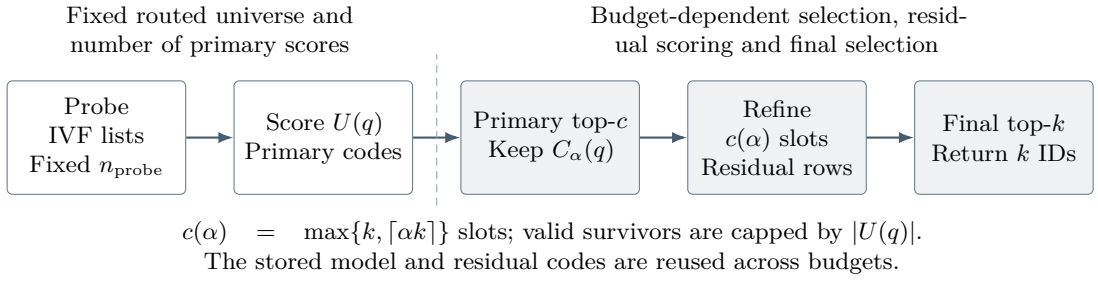


Figure 6.1. The refinement budget changes work after primary-score production. With a fixed query, index and probe count, it leaves the routed candidate universe unchanged. The highlighted stages depend on survivor capacity; this is a logical work decomposition, not a measured latency breakdown. The scan kernel also performs budget-dependent admission and compaction. Padding applies when fewer than c candidates exist.

An explanatory latency decomposition is

$$T(\alpha) = T_{\text{base}} + T_{\text{select}}(c(\alpha)) + T_{\text{refine}}(c(\alpha)) + T_{\text{final}}(c(\alpha)). \tag{6.2}$$

For a fixed batch and route, T_{base} includes transfer, rotation, routing and primary-score production; selection, residual scoring and final selection depend on survivor capacity (figure 6.1). Buffer capacities, occupancy and resource thresholds can change in steps, so this work model does not predict a monotone runtime. The measured small-budget plateau is evaluated in section 7.5.5; the policy minimises accepted budget rather than a noisy timing rank.

Each calibration trial still runs rotation, probing and primary scoring before refinement. Reusing the index avoids rebuilding or retraining; it does not cache every intermediate query result across budget trials. The cost of calibration must therefore count full searches, not merely the difference between two residual-scoring workloads.

The policy selects one budget for an index, routing configuration, k and query workload. It uses neither a trained per-query predictor nor an online choice for each arriving query. Scan-group scheduling remains the separate mechanism of section 5.5; its execution path must still be considered when interpreting budget calibration.

Position among adaptive ANN methods. The multiplier has the same candidate-pool role as FAISS `IndexRefine::k_factor` and cuVS's refinement ratio [6, 9]. FAISS auto-tuning explores speed–accuracy operating points using a ground-truth criterion [8]. Li et al. learn per-query termination conditions from search features [19]; DARTH predicts recall during execution to stop at a requested target [5]. Here the decision calibrates one unlabelled workload budget for reuse, using reference-output coverage rather than a trained recall predictor. The final scorer remains a compressed reconstruction, whereas the cuVS refinement example reranks source vectors. These distinctions separate the work parameter, its selection mechanism and the scored representation.

6.2 GT-Free Sampled Output Agreement

Let $\mathcal{R}_\alpha(q)$ be the set of k valid, unique database IDs returned at budget α on a fixed index. Define $\mathcal{R}_{\text{ref}}(q) = \mathcal{R}_{\alpha_{\text{ref}}}(q)$. The descending grid is

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\text{ref}} = A[0] = 200. \tag{6.3}$$

For S calibration queries $\mathcal{Q}_S$, define

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\text{ref}}(q)|\right), \qquad D_S(\alpha) \le \epsilon. \tag{6.4}$$

The evaluated $\epsilon = 1$ permits one missing reference ID across the whole sample. It ignores output order and distance differences; replacing one ID counts once, although the symmetric difference contains two IDs. The finite-grid objective is

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \le \epsilon\}. \tag{6.5}$$

The reference supplies a nonempty fallback. The objective minimises an accepted multiplier; section 6.3 states when bisection solves it exactly.

At $k = 10$, the normalised allowance $\epsilon/(Sk)$ is 0.003125, 0.0015625 and 0.00078125 for $S = 32, 64, 128$, and 0.0002 at $S = 500$. Increasing S therefore changes both coverage and

effective strictness. Calibration and held-out evaluation use disjoint queries; subsamples reuse a finite calibration pool (section 7.5).

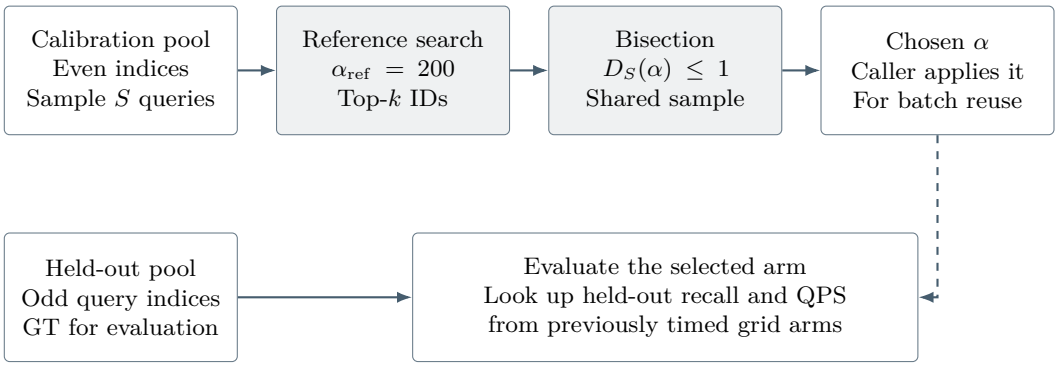


Figure 6.2. Workload-level calibration and its evaluation. Budget decisions use reference IDs; GT assesses held-out quality. Selected-arm results are looked up from timed fixed arms. Implementation boundaries are in section A.3.

6.2.1 What agreement controls

Under Proposition 6.1, disagreement counts reference answers excluded by primary screening:

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

For valid fixed-size output sets, let $r_{\mathcal{Q}_S}$ denote GT recall on these same queries. Each missing reference ID removes at most one GT hit, so

$$r_{\mathcal{Q}_S}(\alpha) \geq r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}. \tag{6.7}$$

This bound is sample-local and reference-relative. Agreement controls additional loss relative to the reference; the reference's own quality requires GT evaluation. Section 7.8.2 quantifies that distinction with the reconstruction pilot.

6.2.2 Sampling resolution and ordered testing

Let H_S count queries with any missing reference ID. Since $H_S \leq D_S$, acceptance implies $H_S \leq 1$. For a prespecified budget and i.i.d. queries, the one-sided 95% exact binomial upper limit after one event solves [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

The query-event limits are 0.03652 at $S = 128$ and 0.009452 at 500. Treating all Sk result indicators as homogeneous independent Bernoulli trials is stronger: the corresponding limits are 0.003701 and 0.0009484, approximately $4.74/(Sk)$. Query i.i.d. alone does not justify dividing the query-event limit by k; neither model certifies repeated subsamples of this fixed pool.

For monotone disagreement and a fixed grid order, bisection selects the accepted prefix. If its per-arm decisions are valid level tests, the first-unsafe-arm argument gives fixed-sequence error control without a Bonferroni factor [3]. The implemented one-slot rule is a stability threshold; a GT-risk guarantee would require a valid test for that target.

A finite-pool extension. Suppose every query has a fixed disagreement count $X_q(\alpha)$ under the intended sample-size execution regime, unchanged by subset membership or position. Let N_{cal} be the pool size and n_j count queries with $X_q = j$. Under uniform sampling without replacement, $D_S \leq 1$ requires either only zero-count queries or one single-count query [23]:

$$\Pr\{D_S(\alpha) \leq 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\text{cal}}}{S}}. \tag{6.9}$$

Out-of-range binomial coefficients are zero. This conditional acceptance model additionally needs the grid's acceptance structure and selected-arm quality map to predict selection risk. Acceptance uses no labels; GT-loss crossings use held-out fixed-arm quality. The subset-sum checks in section 7.5.5 test the count-invariance assumption on the recorded draws.

6.3 Budget Search, Nestedness and Tie Safety

6.3.1 Bisection and complete enumeration

Write $J = |A|$. The implementation computes the sample reference once, then searches for the largest acceptable index in the descending grid:

1. Set $l = 0$, $h = J - 1$. Index zero is the reference fallback.
2. While $l < h$, test $j = \lfloor (l + h + 1)/2 \rfloor$ by running $A[j]$ and computing $D_S(A[j])$.
3. If $D_S(A[j]) \leq \epsilon$, set $l = j$; otherwise set $h = j - 1$.
4. Return $A[l]$. A caller applies it before processing reuse batches.

The eight-point grid needs at most three complete candidate searches in addition to the reference. Invocation, host-comparison complexity and timing boundaries are specified in section A.3.

Complete enumeration repeats the reference and tests all seven other budgets on exactly the same sample, selecting the smallest accepted multiplier. Including a fresh reference in both timers aligns their cost boundaries. Enumeration evaluates the finite grid even if acceptance has holes; bisection requires an accepted prefix to guarantee the same minimum.

6.3.2 A sufficient condition for monotonicity

Proposition 6.1 (Nested survivors preserve reference membership)**.** *For one query, assume fixed routing and survivor sets $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\text{ref}}}$ whenever $\alpha \leq \beta \leq \alpha_{\text{ref}}$. Assume at least k valid survivors, fixed refined scores and one deterministic total order for final ties. Each result is the exact top-k in that order. Then*

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\text{ref}}| = |C_\alpha \cap \mathcal{R}_{\text{ref}}|, \qquad D_S(\alpha) \geq D_S(\beta). \tag{6.10}$$

The latter statement holds when these conditions hold for every sampled query.

Proof. A reference item has fewer than k predecessors in the largest survivor set. If it is present in a smaller set, restriction cannot introduce new predecessors, so it remains in that set's exact top-k. Thus the intersection with the reference equals the reference items present in the survivor set. Nestedness can only add such items. Summing missing reference counts over queries proves the inequality. □

Exact primary top-c under one fixed total key supplies the nesting premise as c grows. A fixed refined total key then makes acceptance monotone, so acceptable indices form a prefix

Case	C_1	C_2	C_3	Reference overlap
Nested	$\{b\}$	$\{b, c\}$	$\{a, b, c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b, c\}$	$\{a, b, c\}$	1, 0, 1

Figure 6.3. Constructed $k = 1$ example with refined order $a \prec b \prec c$ and reference $\{a\}$. The nested case preserves reference membership. With all primary scores tied, different tie choices can produce the non-nested middle pool and break monotonicity. These are explanatory sets, not measured GPU outputs.

and bisection finds the smallest accepted grid budget. Neither GT recall nor execution speed appears in this theorem. Approximate refined scores can promote a false neighbour as the pool grows, and runtime remains hardware dependent.

6.3.3 A corollary for the evaluated grouped path

The run-head source defaults to exact primary selection. Within one calibration draw, S, routing and scan threads are fixed; `scheduled_groups(S)` is independent of c. Changing the budget changes buffer capacity and admission work, while the grouped scorer uses the same FP32 LUT entries and accumulation order. Refinement fixes B_r and 16 lanes, preserving each candidate's residual reduction; final selection uses $(D_{\mathrm{H}}, \mathrm{ID})$.

Corollary 6.2 (Grouped calibration)**.** *For the evaluated grouped configuration, under the representation and shared-bound contracts of Chapters 4 and 5, primary survivors are nested across budgets. Consequently Proposition 6.1 applies and bisection returns the same minimum accepted grid budget as enumeration.*

Proof. Lemma 5.1 and propositions 5.1 and 5.2 give exact primary top-c under the common key $(D_{\mathrm{P}}, \mathrm{position})$. Its prefixes nest as c grows. The fixed per-candidate refinement and exact final total order supply the remaining premises of Proposition 6.1. □

The corollary covers 2,176 of the 2,304 subsampled draws at $S = 32, 64, 128$. The remaining 128 use OpenAI-3072's distance-only one-block selector at $S = 128$. The supplementary trace records every grid count: all 2,304 curves are monotone and all decisions match enumeration (section 7.5.5).

Full-pool coverage. The twelve $S = 500$ calls are separate: two Vogue cells use grouped selection and ten use $g_{\mathrm{scan}} = 0$. The supplementary harness adds full-pool enumeration: all twelve curves are monotone and all selected budgets match the minimum accepted enumerated budget. This closes the finite-grid empirical check. The ten distance-only cells still lie outside Corollary 6.2; a common cutoff order, or no tie straddling any grid cutoff including the reference cutoff, remains a sufficient condition for a general nestedness guarantee. Matching the held-out regime aids transfer without proving that broader condition.

A rank-based extension. Under the same nestedness and fixed-score conditions, let $\mathrm{rank}_{\mathrm{P}}(x)$ be a reference answer's one-based rank under the common primary key. Then

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\mathrm{ref}}(q)} \mathbf{1}\{\mathrm{rank}_{\mathrm{P}}(x) > c(\alpha)\}. \tag{6.11}$$

Table 6.1. Budget controls and the questions they answer. Held-out GT is used to assess every result, but only the diagnostic oracle uses it to choose a budget. The reference is fixed at 200 for agreement-based arms.

Control	Selection information	Question
Fixed 100	No calibration	Gain relative to the inherited default?
Fixed 64 and smaller grid arms	No calibration	Does a simple fixed choice remove the need for calibration?
Sampled bisection	S unlabelled queries; slot agreement	How well does a low-cost sample choose a reusable budget?
Same-sample enumeration	Same S, reference and threshold; all grid arms	Did bisection miss the smallest accepted grid budget?
Full-pool calibration	All 500 calibration queries; same bisection rule	What changes with broader coverage and a stricter normalised allowance?
Held-out oracle	GT recall and timed QPS for all arms	Which measured arm is fastest under the diagnostic quality constraint?

Exporting these ranks would recover the grid curve from one reference search. The current harness reruns each budget; rank extraction and transfer remain prospective work under the same ordering conditions.

6.4 Budget Baselines and Their Questions

Table 6.1 separates budget selection from its evaluation. Fixed arms share the index, route and held-out queries; invocation and timing details are in section A.3.

Let $\widehat{r}(\alpha)$ and $\widehat{Q}(\alpha)$ be measured held-out recall and QPS, and let $\widehat{r}^* = \max_{\alpha \in A} \widehat{r}(\alpha)$. The oracle is

$$\alpha_{\text{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^* - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

The recall slack δ differs from the slot allowance ϵ. The oracle consumes held-out labels and timings: it is a finite-grid diagnostic, not a deployable label-free decision rule.

Full-pool calibration remains label free and uses bisection, not a GT-constrained scan of all budgets. Its larger sample also tightens $\epsilon/(Sk)$. Same-sample enumeration instead holds sample coverage and threshold fixed, isolating the search procedure. The oracle solves a quality-constrained speed problem, whereas agreement minimises an accepted multiplier. Even exact bisection and perfect calibration-pool coverage need not select the oracle arm.

Fixed 64 tests whether a smaller universal default already captures the benefit; fixed 200 supplies a high-budget control. Their observed quality and selected-budget outcomes are in section 7.5 and table 7.8.

6.5 Hold-out Risk, Calibration Cost and Payback

6.5.1 Choosing a calibration scale

The full-pool working point is supported by the held-out results in table 7.7; sampled alternatives are evaluated in figure 7.6. The independent-slot model in section 6.2 provides a planning scale: at $k = 10$, $4.74/(Sk) \approx 10^{-3}$ suggests S of order 500, while 128 has coarser resolution. This strong model concerns reference disagreement, not GT loss, and does not make the empirical recommendation a prespecified quality guarantee.

The path and common-order conditions for FULL are addressed in section 6.3.

6.5.2 Risk and selected-arm performance

Over m calibration draws, let draw r select $\widehat{\alpha}_r$. Define held-out loss and threshold-crossing frequency by

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m}\sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

Mean loss, crossing frequency and tail size answer different questions. Table 7.8 and figure 7.6 report the archived summaries; the largest supplementary per-draw losses are reported in section 7.5.5.

Reported selected-arm rate is $\overline{Q} = m^{-1}\sum_r \widehat{Q}(\widehat{\alpha}_r)$, a lookup from timed fixed arms. It is distinct from the median budget's rate and from the throughput of a calibrated query stream. Quality and speed must be read together (table 7.9).

6.5.3 Cost of reuse

For B_{reuse} equal-sized subsequent batches, model

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}}T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}}T_f. \tag{6.14}$$

When $T_{\text{chosen}} < T_f$, the break-even count is

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

Strictly exceeding B^*_{reuse} gives positive saving. If the chosen batch is no faster, there is no throughput payback. Convert QPS to milliseconds as $T = 1000n/Q$ for an n-query batch. Using $1/\overline{Q}$ is optimistic when choices vary, because $1/\mathbb{E}[Q] \le \mathbb{E}[1/Q]$. Measured call boundaries are in section A.3; direct calibration-plus-stream totals validate this model in four measured cells (section 7.5.6), with quality reported alongside time. Table 7.9 gives the campaign's estimates.

A worked reuse calculation. On OpenAI-1536 at 128 probes, full-pool calibration selects $\alpha = 8$, requesting 80 survivors versus fixed 100's 1,000. Both arms have recall 0.9280; their rates are 267,690 and 167,451 QPS. For 500 queries this gives $T_{\text{chosen}} = 1.868\,\text{ms}$ and $T_f = 2.986\,\text{ms}$. With 9.5 ms calibration, $B^*_{\text{reuse}} = 8.50$: nine subsequent batches give positive modelled saving. These are calculations from archived arms, not directly timed stream costs.

Recalibration every H batches would add T_{cal}/H per batch; drift detection and automatic recalibration remain future work.

6.6 Policy Failure Modes and Applicability

Reuse is justified only for a sufficiently stable index, k, routing configuration and query population. Changing the index or routing changes the calibration target; a different batch regime may also change ties and arithmetic through scheduling. Interpreting agreement requires valid, unique output IDs, held-out GT quality and measured tail loss. Reuse additionally requires common ordering or matched enumeration, repeated timing and enough subsequent work to recover calibration cost (sections 6.3 and 6.5). These are applicability conditions; the harness does not automatically detect workload changes.

For the observed workload family, the resulting choice is full-pool calibration when quality takes priority, with $S = 128$ as the cheaper variant whose crossing risk is measured separately. A caller must install the selected α explicitly and reuse it with the same index, route and k. The contribution is a label-free finite-grid decision, a conditional exact search for that decision and a cost model for reuse. Chapter 7 tests its quality and speed against fixed budgets and the diagnostic oracle, and directly measures calibrated reuse on four cells.

The supplementary trace contents and completed strict CPU-reference checks are documented in sections A.4 and A.5. Remaining checks concern arbitrary-subset per-query invariance, rearchiving candidate dumps for independent re-sorting, the Vogue cross-process discrepancy and future query changes. Their status is maintained in `assessment/ch7-pending-experiments.txt`.

The reported budget campaign and its trace rerun use v69 arithmetic, including TF32 on the OpenAI datasets. The separate FP32 sensitivity measures a search frontier; it does not recalibrate or validate these budget choices under FP32.

Chapter 7

Experimental Evaluation and Research Findings

This chapter evaluates construction and capacity (RQ1), execution correctness and cost (RQ2), held-out budget quality and reuse cost (RQ3), and competitive search regions (RQ4). The experiment families below retain their own controls; section 9.1 synthesises the answers to the research questions.

7.1 Final Experiment Manifest and Comparison Families

The evidence comprises system frontiers, component ablations, selection checks, budget calibration and construction/scale measurements. Appendix A records implementation and run provenance; the repository supplement retains the full family manifest.

7.1.1 Evidence families and measurement boundaries

Historical packing/refinement ablations and newer v69/v71 measurements retain the family-specific boundaries defined in section 3.5. Their ratios are interpreted within each comparison, as required by section 3.1.

Sections 3.2 and 3.3 define the dataset manifest, environment and measurement protocol. The main system comparison uses host-to-host steady-state throughput on the RTX 5090. Construction and budget calibration are measured separately.

The authors' CPU JHQ artifact is represented in separate archived CPU experiments, rather than in the E1 frontier. The October locality campaign uses Vogue with 1,024 lists and level bits $[8, 4]$, whereas E1 uses 4,096 lists and a different residual budget. Older CPU CSVs come from a local reimplementation; separate CPU audit notes document legacy recall-protocol issues. These records support CPU-local questions, but supply no matched-protocol CPU/GPU acceleration factor here.

Adaptive call counts and auxiliary device-only timing are documented in section A.2.

7.1.2 Metrics and reproducibility

Metrics and frontier estimators follow sections 3.2 and 3.4; g_{LUT} and g_{scan} retain the distinct meanings in section 5.1.

Numeric tables and figures are regenerated from archived measurements using the ledger procedure in section 3.5.[1]

7.2 Modern GPU ANN Recall–QPS Frontiers

This section answers RQ4 by locating common-recall operating regions, including regions in which competing methods are faster.

[1] The report repository provides `scripts/build_ch7.py`, `data/ch7/audit.json`, `data/ch7/matched.csv` and the source ledger `data/ch7/sources.csv`. The script requires the sibling `JHQ_GPU` checkout containing the archived measurements.

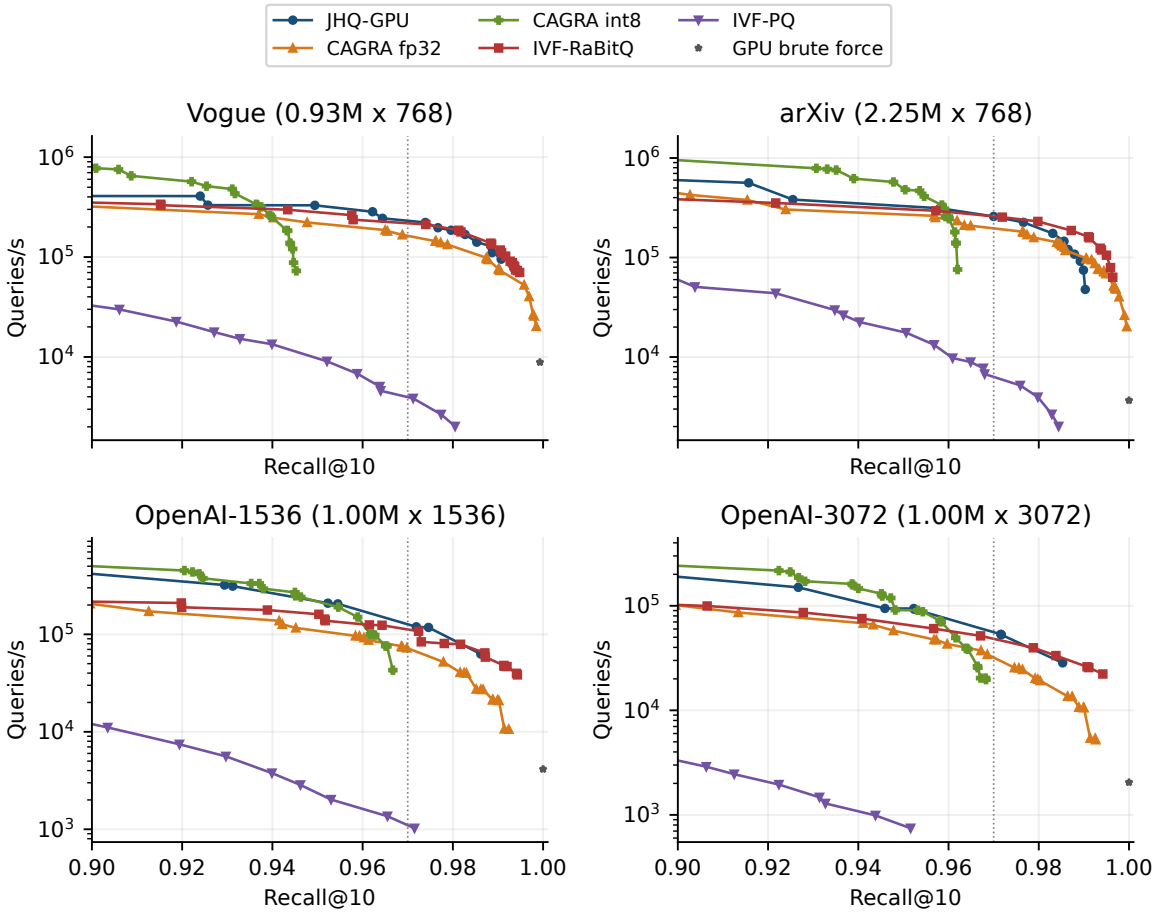


Figure 7.1. Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.90, with a vertical line at 0.97; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis.

7.2.1 Comparison scope and frontier construction

Figure 7.1 compares JHQ-GPU with the graph, compressed-IVF and brute-force baselines of section 3.4 [16, 28, 31], using the first four datasets in table 3.2. Each uses one batch of 1,000 queries with common GT. T5 separately evaluates small batches in section 7.2.5; section 7.6 reports internal ablations.

The recorded schedule uses $g_{\text{scan}} = 0$ for all 105 arXiv/OpenAI run–budget cells and $g_{\text{scan}} = 4$ for all 27 Vogue cells, triggered by Vogue's candidate-count CV of 0.338–0.622. Thus the non-Vogue frontiers exercise query-centric scanning, while Vogue also exercises grouped retention, shared bounds and merging (section 5.5).

Applying the frontier rule in section 3.4, JHQ contributes 27–39 points per dataset and each baseline family 54–264. E1 contains 1,590 valid points, no failure records and no invalid-ID or duplicate-row flags. These are parameter-sweep points, rather than independent index builds.

The tuning dimensions are listed in table 3.3, with complete coverage in section B.1. A curve ending before a target records missing coverage of that finite grid.

Table 7.1. Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator.

Dataset	JHQ kQPS	vs CAGRA fp32	vs IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

These curves are not memory-matched. Nine-bit RaBitQ has a nominal $9d/8$ code budget comparable to JHQ, while its 4/8-bit arms use less. Table 4.1 and section 7.7 give representation costs and measured allocations alongside this performance envelope.

7.2.2 Matched recall and opposing operating regions

Table 7.1 reports recall 0.97 using the interpolated and qualifying-measured estimators defined in section 3.4.

Against CAGRA fp32, interpolated ratios at this target range from 1.317× on arXiv to 1.747× on the two OpenAI datasets. Against IVF-RaBitQ, the corresponding values are 1.059×, 0.996×, 1.130× and 1.157×. The arXiv result is a near tie: its interpolated rate is slightly lower, while the measured threshold comparison is slightly higher. Reporting this as an unequivocal win would give the interpolation convention more evidential weight than the measurement supports.

The paired E1 one-block measurements connect RQ2 to this RQ4 result. Rebuilding Vogue's frontier with $g_{\text{scan}} = 0$ reduces its 0.97 ratio to RaBitQ from 1.059 to 0.933; at 0.96 the corresponding ratios are 1.250 and 1.072. Both use the same run–budget grid and log-QPS interpolation at batch 1,000. The favourable direction therefore depends on the scheduled grouped path in this grid. The comparison holds the index, probes, budget and final outputs fixed within each pair; it measures the integrated path, rather than isolating the shared bound's contribution alone.

Across targets 0.90–0.985, JHQ's interpolated advantage over CAGRA fp32 is approximately 1.10–2.44× in the covered comparisons. However, adding CAGRA int8 changes the low-recall interpretation: around 0.90–0.92, its lower-precision graph search can be faster than JHQ. At higher recall, IVF-RaBitQ often regains the lead, including several targets at 0.98 or above. On arXiv around 0.99, CAGRA fp32 is also faster than JHQ in the recorded grid. The defensible claim is therefore that JHQ has useful competitive regions, particularly around moderately high recall, rather than that it dominates modern GPU ANN methods.

CAGRA int8 reaches maximum measured recalls of 0.9453, 0.9620, 0.9667 and 0.9683 on Vogue, arXiv, OpenAI-1536 and OpenAI-3072, respectively. It therefore has no qualifying point at 0.97 in these grids. At targets 0.90–0.92 it is approximately 1.20–1.98× faster than JHQ by the same interpolation rule; it remains faster at 0.95 on arXiv and at 0.94 on both OpenAI datasets. The 0.97 result concerns methods with measured coverage and does not extend to every target below 0.97.

The two estimators can disagree substantially where the grid is sparse. For example, at recall 0.97 the OpenAI-1536 comparison with CAGRA fp32 is 1.747× by interpolation but 2.294× using qualifying measured points. The latter includes the cost of whichever baseline point next exceeds the threshold. It should not be presented as a directly measured same-recall

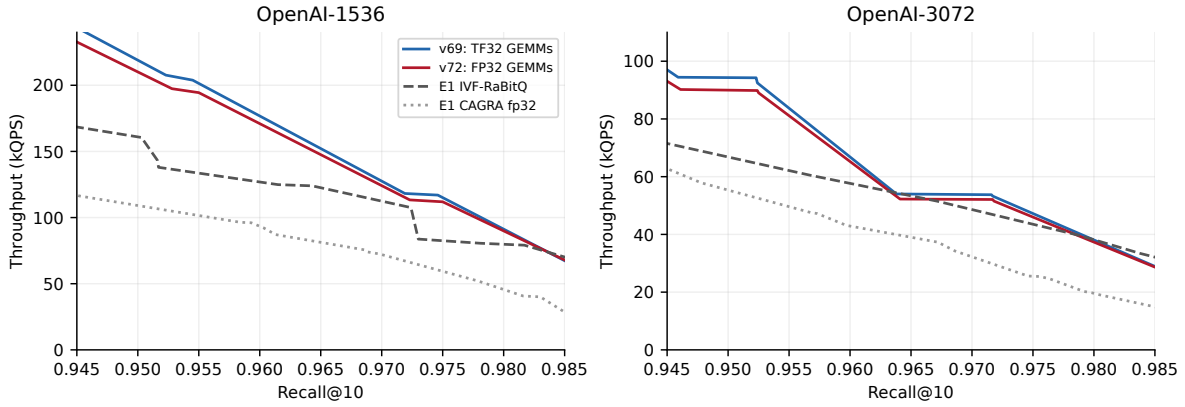


Figure 7.2. Campaign D precision frontiers. Solid curves are same-session JHQ variants; dashed/dotted baselines are historical E1 measurements. All use the chapter's frontier and interpolation rules.

advantage of that magnitude. The companion matched table preserves both estimators and marks targets without coverage, enabling readers to distinguish an operating-region result from a sampling artefact.

7.2.3 Precision sensitivity on the OpenAI datasets

Supplementary source inspection establishes that the E1 implementation uses TF32 multiplication with FP32 accumulation for OpenAI rotation and centroid GEMMs, despite FP32 input/storage and a misleading source comment (section 4.2). The main E1 frontier remains the historical measurement of that implementation. In campaign D, a separate FP32 variant changes the cuBLAS math-mode switch and replays 37/39 OpenAI configurations at batch 1,000, five rounds each. Execution order alternates between variants across probe settings. Both reuse the same TF32-trained model; the intervention changes add/query arithmetic, including database encoding and routing. It is not a comparison of independently FP32-trained models. Table 7.2 summarises the paired changes.

Table 7.2. FP32 sensitivity. QPS ratio is FP32/TF32 at equal configuration (geometric mean); recall differences have the same direction. The last two columns compare the FP32 frontier at recall 0.97 with archived E1 IVF-RaBitQ and CAGRA fp32 frontiers. Campaign D does not remeasure the baselines.

Dataset	Pairs	QPS ratio	ΔRecall	vs RaBitQ	vs CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

At equal configuration, FP32 reduces throughput by about 5% and increases recall by 0–0.0005. At matched recall 0.97, its throughput is about 3% below the same-session TF32 frontier. It remains 1.086× and 1.079× archived RaBitQ, and 1.679× and 1.629× archived CAGRA fp32. New-session v69 measurements give 1.118/1.114 against archived E1 RaBitQ, versus historical 1.130/1.157. Only the v69–v72 intervention is contemporaneous in D; these baseline ratios remain cross-session numerical comparisons. Figure 7.2 shows the two JHQ frontiers with their archived comparators. A later T1 campaign remeasures baseline anchors in the same session; section B.4.1 reports those separate, grid-sensitive results.

7.2.4 Interpretation and comparison limits

The OpenAI pair supplies a row-aligned embedding-width contrast from Qdrant's DBpedia `text-embedding-3-large` corpus family [29, 30]. The supplementary check compares all 1,000 query rows and every 97th base row (10,299 rows): truncating and renormalising the 3,072-dimensional vector gives cosine at least 0.9980 against its 1,536-dimensional counterpart, with median 1.000000. GT top-10 overlap averages 9.046/10. This strongly supports matching row order in the inspected exports; original entity-ID mappings were not exported. Doubling dimension leaves the historical 0.97 CAGRA fp32 ratio at 1.747 in both datasets and changes the RaBitQ ratio from 1.130 to 1.157. The FP32 sensitivity likewise retains comparable advantages. Dimension alone therefore does not explain the observed advantage within this paired corpus/embedding contrast.

The advantage instead co-varies with refinement demand: the E1 setup's 128-probe budget anchors are 4–8 for OpenAI, 64 for Vogue and 100 for arXiv (the later full-pool rule selects 200 there). Their 0.97 RaBitQ ratios are respectively 1.130–1.157, 1.059 and 0.996. This four-dataset association agrees with the design of selective residual access; the anchors are neither equal-recall frontier parameters nor a causal isolation of primary quality. The ablations below test execution components directly.

The interface, memory and compilation differences in section 3.3 remain relevant to small margins. Section 7.8 examines their empirical sensitivity.

7.2.5 Small batches change the competitive region

T5 compares scheduled JHQ, CAGRA fp32/int8 and IVF-RaBitQ at batches 8/32 on the four E1 datasets. Five synchronous host-to-host rounds use 1,000/992 queries, respectively. Its 1,808 points include JHQ's one-block control; IVF-PQ and brute force are outside this campaign. Figure 7.3 and table 7.3 compare the resulting regions.

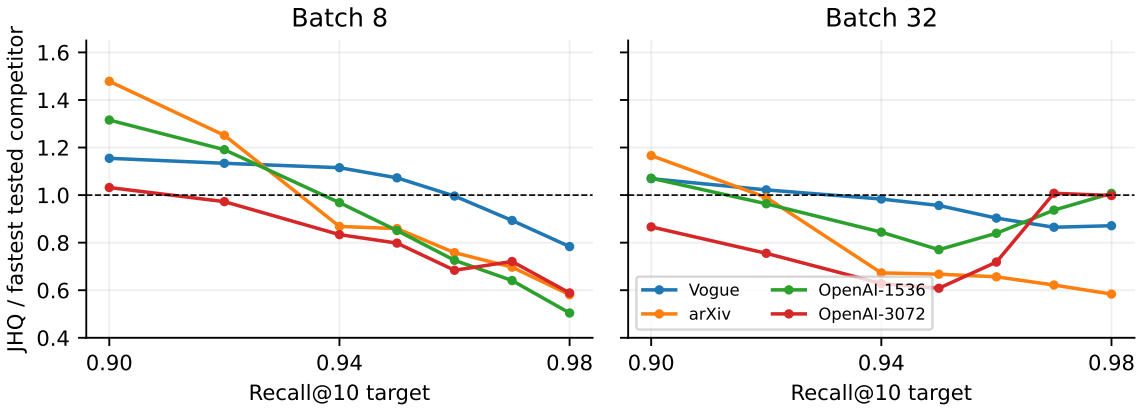


Figure 7.3. T5 JHQ / fastest-competitor QPS under interpolation with qualifying measured endpoints. Above one favours JHQ. Strict measured thresholds are in table 7.3.

At recall 0.97 and batch 8, CAGRA fp32 leads on all four datasets under both estimators: interpolated JHQ ratios are 0.641–0.893. At batch 32, these ratios are 0.622–1.008, with OpenAI-3072 close under interpolation but favourable to JHQ under the measured threshold. At recall 0.95, Vogue's batch-8 lead likewise appears only under interpolation. Grid resolution therefore matters to borderline rankings.

Table 7.3. T5 QPS ratios: interpolation with qualifying endpoints / strict measured threshold. The fastest competitor is selected independently for each estimator; higher-recall qualifying endpoints remain eligible without extrapolation.

Dataset	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

Lower-recall comparisons must retain competitors whose fastest frontier point already exceeds the target. At 0.90, corrected interpolated ratios are 1.032–1.479 at batch 8 and 0.866–1.166 at batch 32; the strict measured estimator gives three and one JHQ wins, respectively. Omitting qualifying endpoints would overstate these low-recall leads. Scheduling is still useful: at recall 0.95 its interpolated gain over one block is 1.7–3.3×. An internal execution gain therefore need not produce a cross-system lead.

Null-call medians by dataset, method and batch span 0.012–0.072 ms; this overhead remains in search timing. CAGRA uses Python bindings, while JHQ/RaBitQ use C++; warm-up policies also differ. Protocol and grid details are in section A.5. E1's full-batch high-recall leads do not generalise to these small batches.

7.3 Cartesian LUT, Packed Loads and Lane Refinement

For RQ2, the ablations separate representation-preserving table and load changes from cooperative residual scoring and measure their application effect as well as their kernel effect.

7.3.1 Exact table factorization and its throughput consequence

The factorisation and resource costs are derived in section 5.2. Here the ablation tests their throughput consequence and FP32 ranking effects.

Figure 7.4 compares Cartesian table grouping on six datasets, probe counts 8/32/128/512, fixed $\alpha = 100$, and $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$. Each arm has five timing rounds; batch size is 1,000 except Stella's 999 queries. Two groups have the highest median throughput in 22 of 24 cells. The exceptions are arXiv at 128 probes, where one group is about 0.23% faster, and OpenAI-3072 at eight probes, where four groups are about 0.84% faster. These narrow differences are best treated as near ties. The newer count is 22/24, rather than the historical ADC paper's 23/24. In the specific two-group versus one-group comparison, two groups win 23/24 cells; each dataset's four-probe geometric-mean ratio exceeds one (1.021–1.318). This supports the choice across the tested workloads without treating configurations sharing an index as independent trials.

The two-group/one-group ratio ranges from 0.998 to 1.914. Four and eight groups each require only 16 total entries, but differ in the number and granularity of lookups. Their performance provides a useful control: reducing table storage beyond the two-group design does not necessarily improve search. Additional indexing and accumulation work can outweigh storage savings. The measurements establish the net trade-off; without a separate occupancy or instruction-counter experiment, they do not identify one hardware effect as the sole cause.



Figure 7.4. Cartesian table grouping at fixed $\alpha = 100$ (v71). Group counts describe g_{LUT}, not parallel scan groups. Curves are medians of per-dataset QPS ratios to the one-group arm; shading shows their range.

The maximum reported recall spread within any four-arm cell is 10^{-4}. At 128 probes and $\alpha = 100$, T4 compares identical candidates on Vogue, arXiv and OpenAI-3072. Full versus $16+16$ tables differ in 22.6–26.1% of candidate-score bit patterns, with maximum relative error 4.53×10^{-7}. Across all forms, 20.5–29.4% differ, with maximum relative error 4.64×10^{-7}. Vogue has 2–4/256 final-ID differences, while arXiv/OpenAI have none. Unchanged primary survivor sets can still produce different final IDs because the rounded primary score enters refinement (section A.5).

7.3.2 Packed access and cooperative residual scoring

The historical packing experiment compares byte-oriented and word-oriented access while holding the dataset, probe count and table grouping fixed. Across six datasets, four probe counts and $g_{\mathrm{LUT}} \in \{1, 2\}$, the 48 pairs yield throughput ratios from 1.012 to 1.658, with maximum recall difference 10^{-4}. Gains are generally smaller at eight probes and larger at 512. Both alternatives already use a subspace-major layout; this experiment supports packed access, not a claim that a physical transpose alone produced the improvement. Each arm has one reported aggregate rate, so this older evidence has weaker uncertainty information than the five-round v71 sweep.

Figure 7.5 and table 7.4 then separate wider loads from lane cooperation in residual refinement. L0 is the original scalar path; L1 changes the load path without distributing a candidate across multiple lanes; L16 assigns 16 lanes to a candidate. The same process rotates arm order across five timing rounds, with three search calls per arm per round. For the twelve ordinary configurations, L16 improves end-to-end throughput by 1.015–1.789×. An additional arXiv configuration at $\alpha = 200$, explicitly a high-budget control, reaches 2.383×; it is not the gain range of the ordinary selected budgets.

The load-only control is bit-identical to L0 in the tested cases, yet already improves arXiv throughput by 1.368×. Lane cooperation adds a larger reduction in refinement time. On arXiv at $\alpha = 100$, the profiled refinement kernel falls from 4,277.5 to 716.8 μs, a 5.97× gain, while total throughput improves 1.789×. OpenAI-3072's refinement kernel improves 8.81×, but total throughput improves 1.171×. This is consistent with Amdahl's law: acceleration of a small stage cannot remove the cost of routing, screening, final selection or transfers. It also motivates the subsequent scan and probe experiments, which target the remaining work.

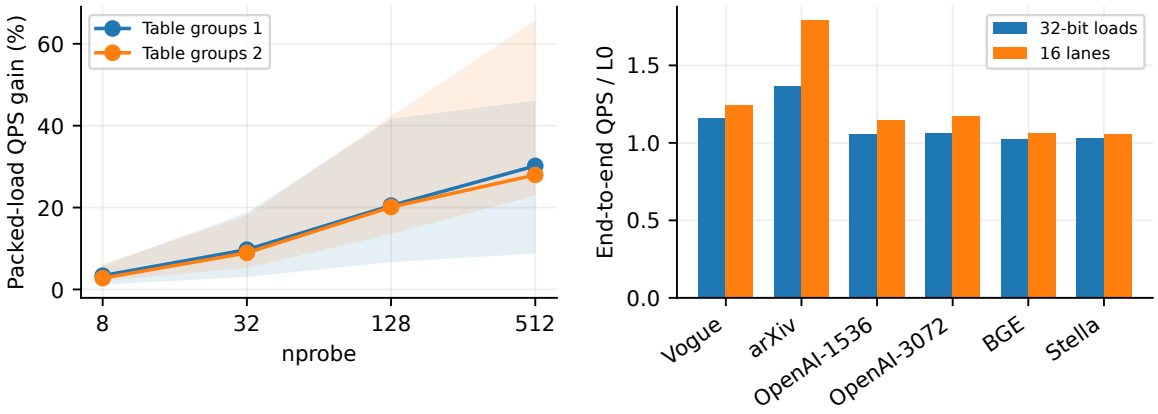


Figure 7.5. Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. These panels come from different experiment families.

Table 7.4. Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged.

Dataset	α	$L1/L0$ QPS	$L16/L0$ QPS	Kernel gain
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

A quantitative Amdahl check uses the refinement kernel's baseline time as a fraction f of host-to-host baseline latency and its kernel speedup s, giving $1/(1 - f + f/s)$ if the remaining time is unchanged. The fractions are approximately 0.529 on arXiv and 0.168 on OpenAI-3072, producing estimates of 1.787× and 1.175×, close to the measured 1.789× and 1.171×. These combine separate kernel-profile and wall-time measurements and are explanatory estimates.

Cooperative reductions alter floating-point addition order. The tested multi-lane variants have very small distance differences, with maximum reported relative error about 1.6×10^{-6}, and occasional changes near ranking boundaries. The evidence therefore supports numerically close scoring with measured end-to-end gains, rather than universal bitwise equivalence. Sanitizer checks cover the recorded refinement and top-k kernels on small query subsets; they are useful implementation checks, not a proof for arbitrary inputs.

7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation

The representation checks in section 4.6 concern the stored index; this section evaluates query selection and its cost.

7.4.1 Strict CPU reference for computed-score selection

The supplementary T4 campaign sorts every visited candidate on the CPU under $(D_{\mathrm{P}}, \text{position})$, using GPU-produced FP32 scores. Its diagnostic exporter follows the packed scan layout and accumulation order; exported scores match all returned valid primary scores bit for bit. Source comparison confirms that the diagnostic version changes only export, relative to the table-sweep implementation. Vogue, arXiv and OpenAI-3072 are tested at 1/8/128 probes, $\alpha \in \{4, 100, 200\}$, $g_{\mathrm{scan}} \in \{0, 1, 2, 4, 8\}$ and both shared-bound settings: 270 configurations of 256 queries, or 69,120 query instances (table 7.5).

Table 7.5. T4 CPU-reference checks. Set and order equality use the complete primary key. Valid count checks $\min(c, n_c)$ survivors; final equality sorts retained computed refined scores by $(D_{\mathrm{H}}, \mathrm{ID})$, including their FP32 bits. Counts are query instances, with queries reused across configurations.

Path	Queries	Set / order exact	Tie-only	Valid count	Final exact
$g_{\mathrm{scan}} \geq 1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}} = 0$	13,824	13,768 / 8,758	56	13,824	13,824

All 55,296 grouped instances match the complete-key reference in order. The one-block path satisfies the distance preorder in all 13,824 instances; its 56 set differences are exclusively cutoff ties. There are 19,120 underfull instances with the correct number of valid survivors, and all 69,120 final selections match the retained-score reference. This directly tests the stage contracts of lemma 5.1 and propositions 5.1 and 5.2. The earlier 64 scan comparisons and 48 shared-bound comparisons provide separate paired-implementation evidence; archived sanitizer runs cover smaller subsets. Export and retention details are in section A.5.

7.4.2 Shared bounds preserve the selection target

Proposition 5.2 establishes the selection guarantee. The paired experiment measures the work saved by sharing that bound.

Across four workloads and four batch sizes, all 48 shared-bound versus independent-bound comparisons agree on primary and final outputs. Five-round paired throughput ratios span 0.994–1.562×. On arXiv at $\alpha = 200$, batch 1,000 and eight groups, the paired profile reduces merge time from about 2,629 to 471 μs and scan from 7,968 to 5,513 μs. Sharing therefore reduces intermediate work in suitable workloads, with small regressions elsewhere.

The rejected static-quota design faces concentrated winners: the first candidate group holds 46–81% of global survivors in the pilot workloads. Equal quotas would require a correctness fallback; calibrated unequal quotas leave the dominant group on the critical path. These observations motivate shared bounds within that pilot's scope.

7.4.3 Exact probe selection and the limits of pipeline identity

Once screening becomes faster, IVF probe selection becomes a larger fraction of small-batch time. The radix probe primitive (v69) changes how the smallest computed centroid scores are selected while retaining the preceding score arithmetic. On OpenAI this includes TF32 GEMMs (section 7.2.3). Its synthetic checker covers 1,792 cases and reports zero failures. Checks of the real arXiv, Vogue and OpenAI-3072 workloads find identical probe identifiers and identical primary top-c positions and distances for all 1,000 queries. These are more specific

Table 7.6. v69/v65 end-to-end throughput ratios for the exact computed-score probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Values above one favour v69.

Dataset	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

observations than a single unchanged recall number: they test the routing and screening outputs directly.

Table 7.6 shows end-to-end improvements of 1.034–1.167×. At batch 8, the profiled probe stage falls from 56.5 to 22.6 μs on arXiv, 52.7 to 14.7 μs on Vogue, and 77.8 to 16.7 μs on OpenAI-3072. Smaller full-batch gains are expected because the unmodified stages account for more of total elapsed time. These are within-family ratios; they should not be multiplied by the preceding v62 or refinement gains as if all measurements shared a single baseline.

Vogue isolates an important distinction between the probe primitive and cross-process pipeline reproducibility. The probe/primary dump is byte-identical across the two binaries for all 1,000 queries in the one-group, batch-8 test, yet final recall is 0.9642 versus 0.9644. The same baseline binary also changes from 0.9642 in the identity run to 0.9644 in each of its two timing runs. Thus the observed variation does not require the probe change; in the dump-controlled case it arises after the recorded primary-selection boundary. Residual representations, inputs and score computation remain possible sources, whose root cause is unconfirmed. Within each checked process, scheduled and one-block outputs remain bit-identical. Appendix A records the dump coverage so that it is not mistaken for every timed path.

Finally, the optional v70 FP16 experiment is a separate approximation. Its probe sets match the v69 reference on 936/1,000 arXiv queries, 979/1,000 Vogue queries and 998/1,000 OpenAI-3072 queries, with very high mean set overlap. High overlap and small recall changes do not make the primitive exact. Those results are excluded from the main E1 frontier and from the exact-probe claim.

7.5 Held-out Budget Quality, Throughput and Payback

The policy and its sufficient monotonicity conditions are defined in Chapter 6 and proposition 6.1. Here RQ3 concerns quality and amortized cost against credible fixed budgets. Accordingly, the comparison includes both the inherited $\alpha = 100$ and the stronger universal fixed control $\alpha = 64$.

7.5.1 Unlabelled selection and a labelled diagnostic oracle

The six datasets are tested at 128 and 512 probes with the fixed, sampled, FULL, enumeration and oracle controls in section 6.4. The disjoint pools and 64 draws per sample size follow section 3.2.

Using the grid and reference defined in section 6.2, the experiment accepts a budget when

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(10 - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{200}(q)|\right), \qquad D_S(\alpha) \leq 1. \tag{7.1}$$

Reference agreement has the GT-recall interpretation established in section 6.2.

Table 7.7. Full-pool calibration ($S = 500$). Loss is relative to the best measured held-out recall; rates are kQPS from the selected fixed arm. "Cal." is one full-pool calibration call per cell, not a repeated stream timing. Held-out counts are 500, or 499 for Stella.

Dataset	Probe	α_{full}	Loss	Q_{full}	vs 100	vs oracle	Cal. ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

The oracle uses held-out labels, whereas FULL applies the unlabelled rule to all calibration queries (table 6.1).

7.5.2 Full-pool calibration as the quality-first working point

Full-pool calibration uses all 500 calibration queries once per cell. Its selected arms meet the held-out 10^{-3} loss target in 12/12 cells; the largest observed loss is 0.0006. Their rates are 0.908–1.000 of the GT oracle's, and calibration costs 9.5–60.3 ms. The geometric-mean rate ratio to fixed 100 is 1.111×, or 1.193× over the ten cells where fixed 100 also meets the target. Table 7.7 shows the complete comparison. Against fixed 64, full-pool selection is faster in six cells, equal in two and slower in four: its main advantage is the observed coverage of all twelve quality targets.

FULL is therefore the retrospective quality-first working point for these cells; $S = 128$ is the lower-cost alternative. The statistical scope remains that of section 6.2.

7.5.3 Lower-cost sampling and its quality risk

Table 7.8 reports the $S = 128$ result. The median selected budget matches the oracle budget in ten of twelve cells; it differs on arXiv at 128 probes and OpenAI-3072 at 512. The selected budgets span 4–200, indicating substantial workload dependence. However, a median budget and a mean recall describe different aspects of the draw population. They need not correspond to the performance of one arm. In addition, mean recall in the table is reconstructed from the printed best recall minus printed mean loss, so its four-decimal precision is approximate.

The important counterexample is arXiv. At 128 probes, 40/64 draws lose more than 10^{-3} recall even though mean loss is only 0.0009; at 512 probes, 32/64 draws cross the threshold. BGE at 512 probes has 24/64 threshold crossings and a 95th-percentile loss of 0.0014. Stella has fewer crossings but a 95th-percentile loss of 0.0030 at both probe depths. Thus mean quality, violation frequency and tail magnitude must be read together. OpenAI-3072 has no observed threshold crossings in these draws, which supports a favourable result on this pool, not a zero-risk statement about unseen queries.

Under the lower-order percentile convention in section A.3 (zero-based index 59), four nonzero tail losses can coexist with $L_{95} = 0$, as on OpenAI-1536. The crossing count in the table preserves that distinction.

Table 7.8. Held-out quality at $S = 128$. Rule α is the median across 64 draws; oracle α uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; L_{95} is the logged lower-order 95th-percentile loss estimate. R_{100} is fixed-budget recall.

Dataset	Probe	α: rule/oracle	R_{100}	$\overline{R}_{\text{rule}}$	Losses	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

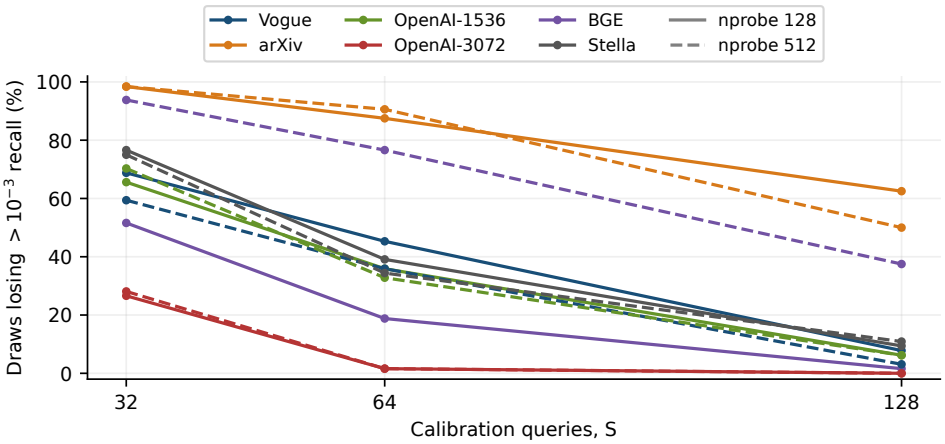


Figure 7.6. Held-out risk across sample sizes. The vertical axis is the percentage of calibration draws losing more than 10^{-3} held-out recall relative to the best tested budget. Increasing S also tightens the allowed slot-disagreement fraction because equation (7.1) keeps threshold one.

Figure 7.6 shows the effect of sample size. A larger sample usually makes selection more conservative, but also tightens the normalized allowance $1/(10S)$. Same-sample enumeration agrees with bisection in all 2,304 draws ($12 \times 3 \times 64$), while taking 1.75–2.01 times as long in the printed logs.

Corollary 6.2 covers 2,176 of the 2,304 draws. The remaining 128 OpenAI-3072 draws at $S = 128$ use distance-only cutoff ties and agree with enumeration empirically. Path changes in eight cells concern calibration-to-evaluation transfer, separately from this check.

Across the eight fixed choices, $\alpha = 64$ and 100 each qualify in ten of twelve cells against the diagnostic quality target; $\alpha = 200$ qualifies in all twelve. Among qualifying cells, their maximum QPS deficits to the diagnostic oracle are 26.6%, 40.9% and 57.1%, respectively. The complete eight-arm comparison is retained in the repository supplement. Full-pool calibration is not identical to the oracle either: on Vogue it selects 100 at both probe depths while the oracle selects 64. Agreement-based selection and fastest quality-constrained selection

Table 7.9. Calibration economics, $S = 128$. Rates are kQPS and rule rate is mean selected-arm QPS. Ratios compare with fixed 100 and fixed 64. The "Batches" column gives payback relative to fixed 100 using 500 queries, or 499 for Stella; "Never" denotes no throughput payback. The quality column separately marks whether fixed 100 meets the held-out loss target.

Dataset	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	vs 100	vs 64	Cal. ms	Batches	Fixed 100 quality
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	Pass
arXiv	128	207.9	188.8	0.908	0.821	4.2	Never	Fail
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	Pass
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	Pass
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	Pass
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	Pass
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	Pass
arXiv	512	83.1	76.2	0.917	0.870	9.7	Never	Fail
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	Pass
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	Pass
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	Pass
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	Pass

solve different problems, even with a larger calibration sample. Full-pool calibration costs 9.5–60.3 ms. In these twelve cells $\alpha = 200$ attains the best measured recall and therefore serves as a high-budget coverage control; its 12/12 qualification is not an independent quality guarantee. The short fixed-arm timings have apparent small-budget inversions. Supplementary alternating repetitions resolve the cited OpenAI inversion as timing variation and establish a low-budget plateau (section 7.5.5). A minimum accepted budget remains a quality decision, rather than a guarantee of the fastest measured arm.

7.5.4 Estimated throughput and calibration economics

Using the mean selected-arm QPS estimate specified in section A.3, $S = 128$ improves over fixed $\alpha = 100$ in ten cells by 1.028–1.691×. On arXiv the ratios are 0.908 and 0.917: mean recall improves, but throughput falls, giving no throughput payback.

The stronger $\alpha = 64$ comparison reduces the aggregate gain: mean selected-arm QPS wins in 9/12 cells, with geometric-mean ratio 1.054× and range 0.821–1.367×. It is nearly unchanged on Vogue and improves only 0.6% on BGE at 512 probes; OpenAI gains range from 1.052 to 1.367×. Both fixed 64 and fixed 100 qualify in 10/12 cells, failing the two arXiv cells. The useful property of calibration is matching heterogeneous refinement demands, rather than uniformly beating an already sensible fixed budget.

ArXiv's threshold crossings are relative to the best tested held-out recall, not relative to the default configuration. Fixed 100 already loses 0.0012 and 0.0016 recall at probes 128 and 512, while the sampled mean loss is 0.0009 in both cells. Mean quality therefore improves over the default despite some draws exceeding the oracle-relative target.

Calibration costs 3.9–21.3 ms under the boundary in section A.3. For an equal-sized subsequent batch,

$$B^* = \frac{T_{\text{cal}}}{T_{100} - T_{\text{rule}}}, \qquad T_{\text{rule}} < T_{100}. \tag{7.2}$$

Using $T_{\text{rule}} \approx n/\overline{Q}_{\text{rule}}$ gives the estimates in table 7.9: 2.4–55.4 subsequent held-out-sized batches against fixed 100 in favourable cells. Against fixed 64, the corresponding range is 5.7–404.3 batches in nine favourable cells; Vogue at 512 probes has no throughput payback. The two arXiv controls also miss the quality target, so their timing comparisons are diagnostics

rather than comparisons at matched quality. At the control rates, the largest positive $S = 128$ break-even estimates correspond to 0.54 s versus fixed 100 and 2.72 s versus fixed 64 (about 27.7K and 202.2K queries). Full-pool positive cases reach at most 1.52/2.25 s and 64.9K/78.7K queries against those controls, using their own calibration costs. These short modelled horizons favour reuse on long stable jobs. The estimates use 500 queries per held-out batch, or 499 for Stella.

These estimates inherit the mean-rate inversion and stationary-reuse assumptions analysed in section 6.5. Direct execution below checks their net-cost implication on the recorded query stream.

For historical comparison, the earlier scalar-refinement implementation (v57) reported 1.054–2.773× gains over fixed 100 in 10/12 cells and 5.2–30.8 ms calibration. The newer lane-refined pipeline's smaller gains are consistent with less removable refinement time. The runs share the budget protocol and grid, but this cross-campaign comparison does not isolate refinement as the sole cause of the change.

7.5.5 Complete decision traces and finite-pool risk

The supplementary harness preserves the original grid, trained state, query split and seeded draws, adding complete records. All twelve fixed-arm recalls reproduce at the archived precision; full-pool choices, sampled median choices and crossing counts reproduce. Timing changes the GT oracle's fastest arm in two near-tied cells, illustrating that this diagnostic also depends on measured speed.

All 2,304 sampled disagreement curves are monotone and their selected budgets match enumeration. The twelve full-pool curves also pass both checks. Moreover, summing full-pool per-query counts over each recorded sample's actual IDs reproduces all 18,432 grid counts. This is stronger than checking the selected budget alone, and supports using equation (6.9) for these observations despite the scan-path changes. It checks aggregate counts on the traced subsets, rather than per-query invariance on every possible subset. The repository supplement retains all twelve cells of per-draw maxima.

At $S = 128$, maxima reach 0.0050 for arXiv/512 and 0.0046 for BGE/512; at $S = 32$ the largest is 0.0294. A low mean loss therefore coexists with occasional materially worse chosen budgets. FULL's maximum across the twelve cells remains 0.0006.

For arXiv/128, the finite-pool model predicts crossing probabilities 0.965, 0.875 and 0.620 at $S = 32, 64, 128$, against 0.984, 0.875 and 0.625 observed (figure 7.7). Acceptance uses only reference outputs; translating its selection probabilities into GT-loss crossings uses the fixed-arm held-out quality map. The calculation is therefore an offline risk explanation, not a label-free GT guarantee. Across the 36 comparisons, the largest absolute gap is 0.144 for BGE/512 at $S = 128$ (0.519 predicted, 0.375 observed). The fixed-pool calculation isolates sampling risk without fitting a population model to held-out recall gaps.

Ten alternating remeasurements also remove the apparent small-budget speed inversion. OpenAI-3072/128 gives 142.3/142.7 kQPS at $\alpha = 4/2$, versus 145.0/128.7 in the original short timing. BGE/128 and Vogue/512 likewise approach a plateau. This supports treating the archived inversions as timing variation; it does not imply monotone runtime for all budgets or resource thresholds.

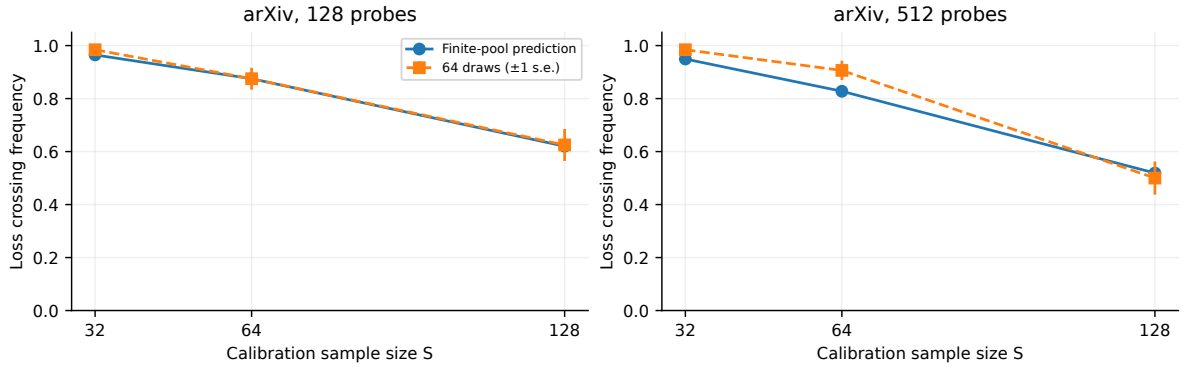


Figure 7.7. Finite-pool prediction and observed arXiv loss crossings. Bars show one estimated standard error over 64 independently drawn subsets conditional on the fixed pool, not uncertainty over future workloads.

7.5.6 Direct calibration-plus-reuse measurement

Four cells directly execute fixed 64, fixed 100, sampled $S = 128$ and FULL policies, rotating policy order over five rounds. Each calibrated round gathers its sample, calibrates once, explicitly installs the chosen multiplier, then times 500 consecutive held-out batches. The first batch includes any graph recapture. The same held-out pool is replayed, rather than sampling new traffic. Index construction, prior warm-up and the separately timed schedule calibration are outside this boundary. Table 7.10 reports the cumulative measurements.

Table 7.10. Direct total milliseconds for 500 reuse batches, averaged over five rounds, including budget calibration for sampled and FULL policies. Each batch has 500 queries. Quality is discussed with the timing results.

Dataset/probes	Fixed 100	Fixed 64	Rule $S = 128$	FULL
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

On OpenAI-3072/128, the sampled policy selects 4 in every round and reduces total time from 2,757 to 1,769 ms against fixed 100, about 36%, at the same measured recall 0.9252. FULL selects the same budget and takes 1,783 ms including its larger calibration cost. On OpenAI-1536/128, FULL takes 860 versus 1,491 ms at the same recall 0.9280; the sampled policy averages 852 ms and 0.9276 recall.

The steady-rate fit to the sampled stream gives break-even counts of 3.6/5.5 batches against fixed 100/64 on OpenAI-1536, and 3.5/8.9 on OpenAI-3072. These fractional values fit directly timed calibration and steady batches; they are not directly observed fractional crossings. Figure 7.8 instead plots measured cumulative totals, including first-batch setup. The first positive mean cumulative savings occur at 4/6 batches on OpenAI-1536 and 4/9 on OpenAI-3072 against fixed 100/64. BGE/512 has a smaller sampled saving, 4,921 versus 5,185 ms against fixed 100, but mean recall 0.9658 versus 0.9670; its FULL policy restores 0.9664 at 5,062 ms. ArXiv's FULL policy selects 200 for higher quality and is slower than fixed 100/64, both of which miss the quality target. Thus timing and quality jointly determine whether calibration is worthwhile. The measurements support reuse economics without requiring the old fitted payback values to match exactly in every cell.

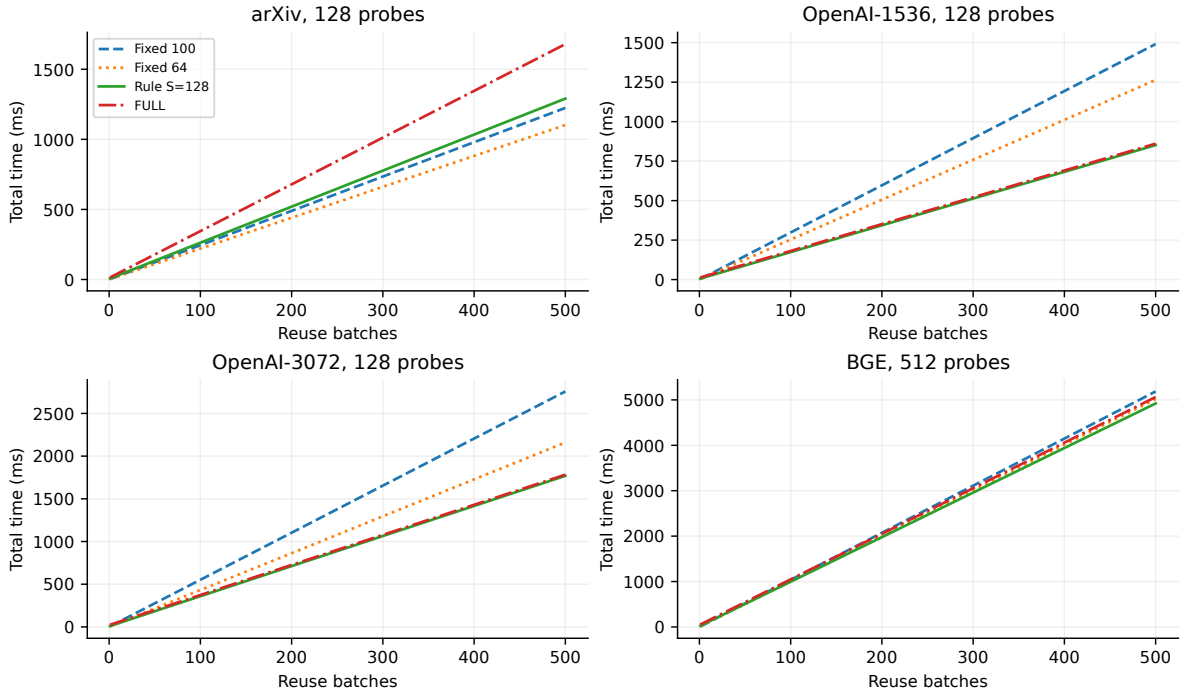


Figure 7.8. Direct mean cumulative time, including budget calibration and first-batch setup. Each curve replays one fixed held-out query pool. Policies can have different recall; timings alone are not a quality-matched ranking. Dataset/probe settings match table 7.10.

7.6 Batch Regimes, G Scheduling and Workload Skew

This section answers the performance-regime part of RQ2: when additional scan groups supply useful parallelism, and when their extra work dominates.

7.6.1 Parallelism versus repeated selection work

The scan experiments reveal two execution regimes. With one block per query, batch 8 exposes only eight blocks on a 170-SM device. Candidate partitioning increases the available block parallelism. In the independent-bound grouped-scan experiment, eight groups improve end-to-end throughput by 2.06–2.76× at batch 8. Profiling supports a stage-level explanation: arXiv's scan falls from about 849 to 216 μs, Vogue's from 1,059 to 246 μs, and OpenAI-3072's from 728 to 155 μs. The additional merges cost approximately 16.7, 9.1 and 3.7 μs, respectively. The net reduction is large even after paying for merging.

At batch 1,000 there are already enough independent queries to expose many blocks. Additional groups repeat table staging, local-buffer admission and sorting, and produce extra merge traffic. The independent-bound grouped path can fall to about 0.46× its one-block baseline on arXiv at $\alpha = 200$. The full-batch profiles show scan increasing from 2,922 to 6,079 μs at $\alpha = 100$ and from 3,530 to 7,652 μs at $\alpha = 200$, with an additional 2,624 μs merge in the latter case. This opposing result is central to the design: parallel decomposition solves insufficient concurrency, but incurs work that can dominate once concurrency is adequate. Shared bounds reduce that extra work but do not eliminate the regime change.

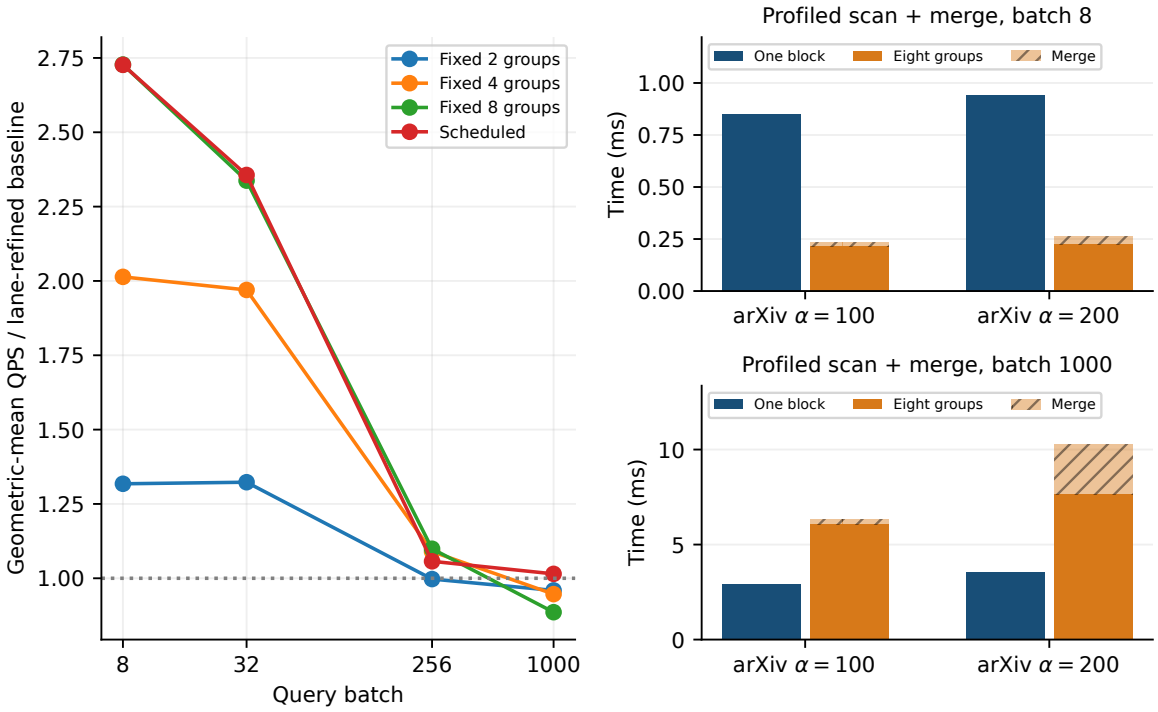


Figure 7.9. Execution regimes. Left: fixed shared-bound group counts and scheduled execution relative to the lane-refined one-block baseline (13 cells at small batches, 11 at batch 1,000). Right: paired scan and merge profiles at batches 8 and 1,000 for arXiv budgets 100 and 200. Hatching denotes merge time. The right panels use the independent-bound scan experiment; they explain a cost, not the scheduler's entire speedup.

Table 7.11. Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split.

Batch	Cells	$g_s = 2$	$g_s = 4$	$g_s = 8$	Scheduled	Held out
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

7.6.2 A fixed rule evaluated on further configurations

The scheduler follows the fill/skew rule and pseudoquery calibration in section 5.5. Four configurations set the constants before the additional configurations were measured.

Table 7.11 recomputes the results from the raw timing arrays. The scheduled geometric-mean gains are 2.727×, 2.356×, 1.057× and 1.015× at batches 8, 32, 256 and 1,000. There are thirteen configurations at the first three batch sizes, including an additional high-budget arXiv case, but only eleven at batch 1,000: Stella's 999-query pool cannot provide a complete 1,000-query batch. The held-out subsets contain nine and seven configurations, respectively. A plot connecting these means summarizes regimes rather than a perfectly balanced repeated-measures population.

Fixed eight-group execution matches the scheduled mean closely at batch 8 but falls to 0.886× at batch 1,000. Scheduling largely retains the small-batch gain while avoiding that full-batch loss. It is nevertheless a heuristic, not an oracle. At batch 256 its worst ratio of best fixed throughput to scheduled throughput is 1.217, on a BGE configuration. Candidate-count CV does not capture every cost of list traversal, selection or merging. This failure identifies a limitation of the chosen feature rather than evidence that execution scheduling is unnecessary.

Vogue illustrates both the value and limits of the skew feature. Its E1 candidate-count CV ranges from 0.338 to 0.622 as probes change, forcing four groups at batch 1,000. The integrated path/one-block ratio ranges from 0.895 to 1.175. It loses at $(n_{\text{probe}}, \alpha) = (16, 16)$, $(16, 64)$ and $(32, 64)$, with ratios 0.955, 0.895 and 0.955. Candidate-count dispersion alone does not encode absolute per-query work; the rule can add overhead to these low-probe cases while enabling Vogue's 0.97 cross-system lead in section 7.2. This complements the BGE batch-256 regret: balancing work and paying for decomposition are distinct scheduling decisions.

Finally, correctness is checked independently of timing. Across 250 variant/configuration comparisons and 234,320 query comparisons, the integrated campaign reports no non-tie primary-selection differences and no different final identifier sets. Final order can differ at ties; the entire campaign should not be described as universally bit-identical. The throughput results concern sequential, synchronous submissions and do not establish tail latency under an open-loop arrival process.

7.7 Cold Construction, Resident Memory and Tested Scale

RQ1 is assessed through cold construction, resident memory and completed large-dataset searches; the E1 memory comparison also informs RQ4.

7.7.1 Cold builds and the lifecycle boundary

The principal search sweeps reuse trained-state caches. Loading that state and adding vectors is not a cold build. Separate logs explicitly train with an empty trained-state cache and then encode/add the database; table 7.12 reports their recorded training and add times. The supplementary repetitions use the same empty-cache construction boundary; table 7.12 now reports the medians of three normal runs per dataset. The complete six-dataset repetition record is retained in the repository supplement; the ranges and sampled peaks below explain the construction boundary. Vogue's training is 298–303 ms, so its normal total is about 0.56 s; the archived 1.33 s total (including 1.056 s training) is an unusually slow single observation. Stella's normal totals are 10.54–10.65 s, close to the historical 10.40 s record.

Construction and startup intervals follow the distinct lifecycle boundaries in sections A.2 and 3.3.

Figure 7.10 contrasts JHQ's normal-build repetitions with baseline configuration ranges. As specified in its caption, RaBitQ's build-function time excludes the serialization round trip required before search.

On Stella, successful isolated CAGRA int8 builds take approximately 74.55 and 82.45 s including dataset quantization, depending on graph degree; the measured IVF-PQ build median is 86.59 s across its three configurations. JHQ's 10.54–10.65 s repeated interval is substantially shorter under the reported boundaries, but does not establish a universally fastest builder or equal downstream recall. Construction speed, searchable quality and resident cost are separate deployment dimensions.

Table 7.12. Cold JHQ construction and separately measured resident memory. Train/add are component medians of three supplementary normal builds; Total is the median paired sum. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query.

Dataset	Train s	Add s	Total s	Used GiB	Raw GiB	Residual
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

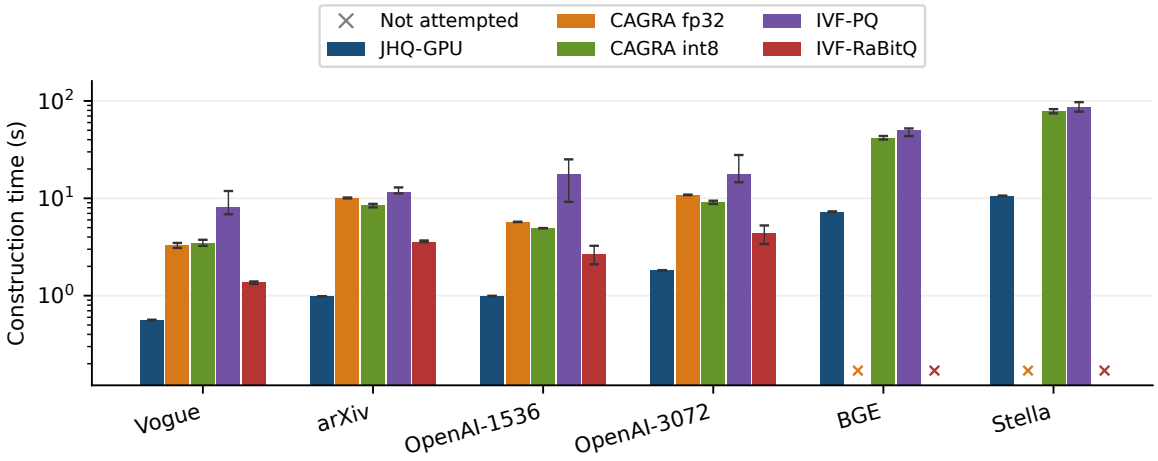


Figure 7.10. Recorded construction costs, with CAGRA int8 included. JHQ is the median and range of three supplementary normal cold builds with warm input pages; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic.

Across five independent builds per dataset at fixed 128-probe points, the largest recall range is 0.0012 on arXiv; normal timing ranges use only the three uninstrumented builds. The broader startup boundary is material: Stella takes 18.2–18.6 s with warm input pages and 129.9 s on its session-first read. The phase diagnostic synchronises stages and changes the pipeline to one buffer, so its timing is excluded from normal-build ranges. Twenty-millisecond device samples observe 26.08 GiB for BGE near add entry and 22.58 GiB for Stella during repack. BGE's three assignment-product buffers total 12 GiB; they belong to the early pipeline lifetime, not to the later 23.05 GiB gather payload (equation (4.13)). These results show why resident memory alone understates construction capacity.

7.7.2 Resident allocation and where compression helps

Figure 7.11 combines the component model in section 4.4 with measured resident totals. Its unattributed remainder is retained, and the preceding construction measurements give the sampled peaks.

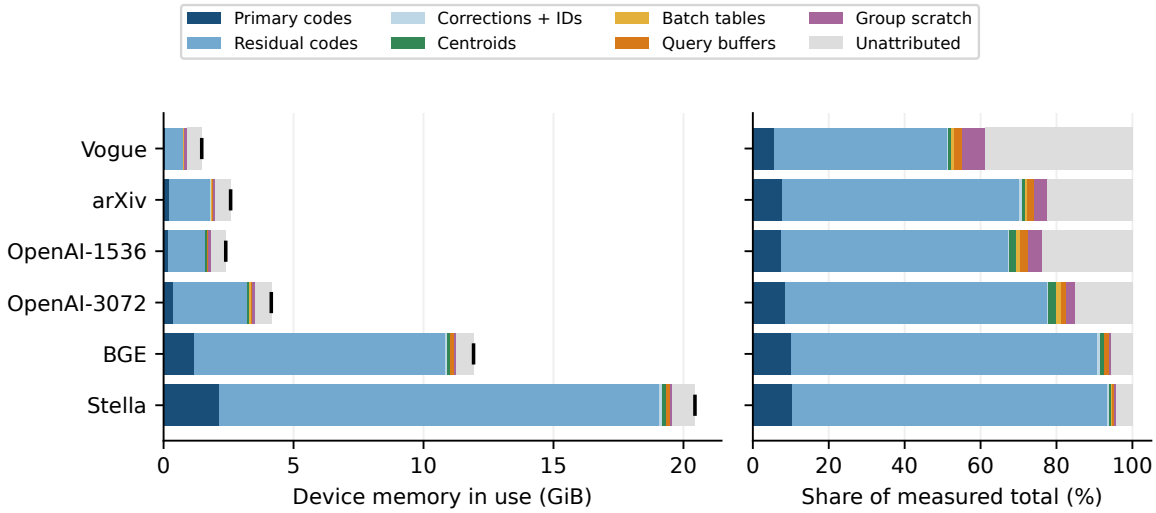


Figure 7.11. Resident-memory decomposition at two-group tables, 128 probes and $\alpha = 100$. Component sizes are modelled, markers are measured used totals, and hatching is the unattributed remainder. The totals include context and workspace. The preceding construction measurements give sampled peaks; these resident markers are not peaks.

Residual codes dominate the large datasets: approximately 80.7% of BGE's 11.93 GiB and 82.9% of Stella's 20.44 GiB used totals. The corresponding raw FP32 vector arrays alone would require 38.50 and 67.81 GiB. JHQ therefore enables the tested resident search on one card by retaining a compressed hierarchy. Lowering α reduces residual codes accessed and scored per query, but leaves the stored residual array allocated. Similarly, Cartesian LUT factorization reduces query-specific table storage, not the database's residual-code precision.

Workspace also matters. The reported list-group scratch is approximately 93.8 MiB in these $\alpha = 100$ measurements, reserved for a batch capacity of 1,024 and up to eight groups even when the selected execution uses the one-block path. A smaller actual group count therefore does not imply that the allocation shrank proportionally. Conversely, these resident samples do not capture temporary construction peaks. The baseline harness also initializes some search memory pools after its index-memory measurement, so an after-build index delta cannot be used as a complete peak process-memory budget across methods.

The E1 measurements provide a separate cross-system view in table 7.13. Subtracting the baseline-measured 507.2 MiB context from JHQ's process total gives approximately 0.29–0.39 of CAGRA fp32's index allocation. JHQ's figure includes workspace, whereas CAGRA's is the build-time index delta. These are parameter-sweep ranges, not memory-matched recall points or a comparison of equal peak budgets. The CAGRA int8 allocations in the same table are of a similar order to JHQ's: compression relative to fp32 is shared by competing designs. The component model in table 4.1 explains this comparison.

7.7.3 Large-dataset feasibility and competitive regions

BGE contains 10,091,524 vectors and Stella 17,776,615, both with 1,024 dimensions and 32,768 IVF lists. Stella's queries are sampled from the corpus; its result therefore characterizes that benchmark rather than an independently collected external-query distribution. Figure 7.12 includes successful CAGRA int8 and IVF-PQ measurements. Full-precision CAGRA, RaBitQ configurations and FP32 brute force were not attempted in the large-scale comparison when their computed storage requirements exceeded the available device capacity. Those omissions

Table 7.13. E1 allocation ranges. JHQ is process total minus a 507.2 MiB context estimate, including query workspace; both CAGRA columns are index allocations measured during build. Ranges cover the parameter sweep.

Dataset	JHQ net process GiB	CAGRA fp32 index GiB	CAGRA int8 index GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

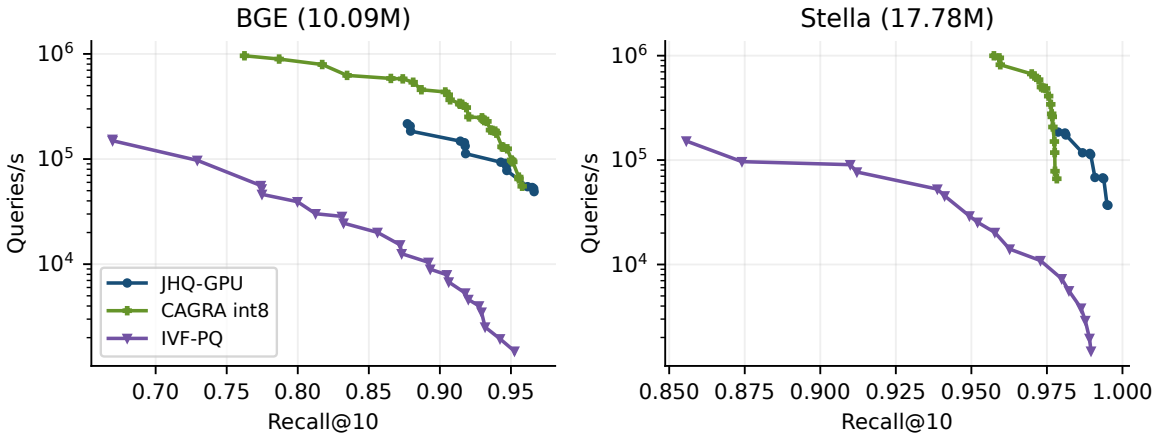


Figure 7.12. Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE uses 1,000 queries and Stella 999. An endpoint is the end of the tested grid, not an intrinsic recall limit.

are planned feasibility exclusions, not observed out-of-memory errors or measured recall failures.

Successful isolated reruns show that CAGRA int8 fits Stella in the tested configurations. On BGE, int8 is approximately 1.35–2.96× faster at the sampled common targets 0.88/0.90/0.92/0.94/0.95. At its measured maximum recall 0.9579, JHQ's interpolated QPS is 1.086× int8's. Thus the advantage changes direction near the high-recall end. The maximum tested recalls are 0.9579/0.9783 for int8, 0.9663/0.9952 for JHQ and 0.9525/0.9896 for PQ on BGE/Stella.

On Stella, one concrete high-end comparison is 185.4 kQPS at recall 0.978679 for JHQ versus 66.3 kQPS at 0.978278 for int8, a 2.80× rate ratio with slightly different recall. This is an explicit pair of measured points, not a uniform matched-recall gain: much of JHQ's higher-recall region has no int8 coverage, and JHQ's highest-recall points trade throughput for quality. Corrected-run exclusions and harness diagnostics are recorded in Appendix A.

The result is evidence of tested scale and useful high-recall operating regions, rather than arbitrary-size scalability. The largest completed search uses 17.78 million vectors on one GPU; there is no multi-GPU experiment, no scaling curve holding data distribution fixed and only sampled construction peaks, rather than an allocation-exact proof near the card's capacity. Stella's searches use its actual 999-query pool.

7.8 Numerical Robustness, Anomalies and Recall Attribution

The robustness checks qualify the RQ2 exactness claims, the RQ3 risk assessment and the small cross-system margins in RQ4.

7.8.1 Quality resolution and repeated execution

With $k = 10$ and 1,000 queries, one recovered neighbour changes aggregate Recall@10 by 10^{-4}; with 500 held-out queries the resolution is 2×10^{-4}. Stella's 499-query split has a slightly coarser resolution. This discretization is not an uncertainty interval, but it helps interpret the small differences reported above. A fourth-decimal change may represent only one or two result slots, while a small mean loss can still hide several queries with substantially changed answers.

Repeated calls and repeated index builds answer different questions. E1 retains 369 baseline repeat-call identifier flags, including tie-order changes; these points remain in the frontiers. The 401-count convention and cross-protocol checks are separated in section A.2.

Five-round ranges describe timing within a fixed build. A separate frontier-slope calculation quantifies sensitivity to a horizontal recall shift: continuous log-QPS segments adjacent to 0.97 imply about 1.01–3.17% rate change per 10^{-3} recall, before applying the near-target snapping convention. The supplementary five-build recall ranges are 0.0008 for Vogue, 0.0012 for arXiv, 0.0004 for OpenAI-1536 and 0.0002 for OpenAI-3072 at fixed operating points. Applied purely as horizontal sensitivity scales to the local slopes, these imply changes of order a few percent, rather than the larger OpenAI fp32-CAGRA margin. The experiment does not repeat whole frontiers or baseline builds; it therefore supports local stability assessment, not a universal bound on matched ratios. The precision intervention in section 7.2.3 directly establishes that the OpenAI advantage survives removal of TF32 under its stated cached-model boundary. ArXiv and the LUT near ties retain their narrower interpretation.

Across each dataset's RaBitQ configurations, the median five-round range divided by median QPS is 3.79%, 3.34%, 2.87% and 5.25% for Vogue, arXiv, OpenAI-1536 and OpenAI-3072. These descriptive spreads are comparable in scale to the small Vogue advantage, rather than confidence intervals for its matched ratio. Raw call traces also contain long calls, including occasional 30–60 ms observations in otherwise faster configurations. As a sensitivity check, recomputing QPS as 10^6/ median(call latency in ms) for a 1,000-query call reduces the influence of long calls. At recall 0.97, the resulting JHQ/RaBitQ ratios are 1.063, 0.995, 1.118 and 1.100. The three favourable directions and arXiv near parity remain, so the result is not solely a consequence of long calls.

7.8.2 Exactness is attached to a particular stage

Table 5.1 distinguishes the score and selection targets; the stage contracts in Chapter 5 determine the scope of each exactness claim.

The lane-rounding, cutoff-tie and cross-process observations above therefore qualify different stages. Their unchanged real-arithmetic formulas do not establish raw-space exact kNN.

The archived reconstruction pilot measures the remaining gap to full-probe reconstructed scoring. Across seven workload–budget arms of 128 queries each, scoring all probed candidates in FP64 improves recall over GPU outputs at selected budgets by 0–0.002344. For arXiv at 128 probes, $\alpha = 200$ matches the full-IVF reconstructed answer set on all 128 queries. At 100, the recall is 0.9734375 versus 0.9750 for full-IVF scoring: an additional 0.0015625 budget deficit

beside the joint routing/reconstruction deficit of 0.0250 (section 6.2). These are pilot-specific aggregate differences; the seven arms include two budgets on one dataset–probe workload. FP64 reconstructed scoring also differs from the GPU arithmetic, so the pilot does not isolate a pure budget effect in every arm. Separating routing from representation requires additional stage-wise GT coverage.

7.8.3 Scope of the research findings

The results support representation-aware execution and workload-specific refinement effort, with separate evidence for correctness, speed and quality. Chapter 8 develops the cross-layer implications and negative cases; section 9.1 brings together the answers to RQ1–RQ4.

The claims apply to squared-L2 top-10 search, the recorded query pools and the tested RTX 5090 environment. Cross-device behaviour, changing queries, online updates and open-loop service latency remain untested. These boundaries accompany the results rather than replacing their positive findings.

Chapter 8

Discussion, Limitations and Future Research

The measurements favour a particular combination of structured codes, selective refinement and workload-dependent execution. Their wider value lies in explaining when an apparently cheaper operation becomes a faster search. This chapter develops three interpretations, examines two designs that did not justify general adoption, and turns the remaining limitations into testable research questions. The interpretations retain the distinction between measured effects, explanations and proposed extensions established in section 3.6.

8.1 Three Cross-Layer Insights

8.1.1 A useful representation exposes a useful unit of reuse

The grouping results in figure 7.4 show that compact state alone does not determine the best scorer. The reusable unit is the query-specific coordinate group: its distances are computed once and reused across candidates, as constructed in section 5.2.

The shared-memory transition and word-access costs in sections 5.2 and 5.3 call for joint consideration of storage, instructions and reuse. The exceptional LUT cells and Mailman case below counter a universal smallest-table rule. Two groups are an empirical default for this device, rather than a device-independent optimum.

The compatibility condition in section 5.2 limits this opportunity: improving a codebook's reconstruction can remove its execution advantage. A representation should therefore be assessed by the attainable recall–cost frontier of its scorer and residual hierarchy, alongside reconstruction error.

8.1.2 Useful parallelism is bounded by duplicated state and selection

The batch trend in figure 7.9 reflects a trade-off between available parallelism and duplicated state/selection. Once query-level concurrency is sufficient, staging, local buffers and merging can consume the benefit of additional blocks. Maximum occupancy alone is therefore the wrong objective.

Vogue's path-sensitive frontier and BGE's scheduling regret (sections 7.2 and 7.6) connect execution design to the competitive region. They also expose the limitation of CV: it measures imbalance without absolute work or staging/merge costs. The combined path comparison remains distinct from the shared-bound-only ablation.

A practical consequence is to retain a query-centric path and choose grouping from both available parallelism and expected work per query. Candidate count, survivor capacity and local-buffer cost are plausible inputs for a richer decision. A fitted selector would need held-out workloads and its own calibration-cost accounting; the present observations do not validate one. The existing heuristic is valuable precisely because its gains and failure cells can be checked separately.

8.1.3 Execution speed changes the value of avoiding refinement

Budget reduction removes residual reads, arithmetic and final selection, while routing and broad primary-score production remain. Its potential saving depends on the fraction of latency occupied by those removable operations. Faster lane-cooperative refinement can consequently reduce the reward for lowering the survivor budget. Faster primary scanning can have the opposite effect on refinement's relative share. Neither transformation changes the quality consequence of discarding a particular candidate.

This is a cost-model implication, rather than a causal explanation for every historical/newer budget difference. A causal test would replay identical budget decisions and candidate sets while changing only the refinement kernel. The cross-campaign comparison and direct reuse measurements are reported in sections 7.5 and 7.5.6.

The FULL working point and sampled tail risks in table 7.7 and figure 7.6 imply that calibration effort should reflect the quality requirement and expected reuse. Optimising the selector alone, or reporting only a favourable selected-arm mean, leaves that engineering decision incomplete.

8.2 Two Deep Negative-Result Case Studies

8.2.1 Mailman-style shared sums: arithmetic reuse meets scatter cost

The sign-code form of primary distance suggests reusing sums across candidates that share bit patterns. The Mailman algorithm supplies the general finite-alphabet matrix–vector idea [20]. The archived pilot tests GPU implementations on frozen Vogue, arXiv and OpenAI-3072 candidate streams, at 128 probes and batches 8/1,000. Its baseline reproduces the packed two-table scoring loop. Atomic bucket accumulation, private histograms and group widths 4/6/8 are alternatives within the experiment; the mathematical score and input codes remain fixed.

The fastest alternative in each of the six cells achieves only 0.187–0.581 of baseline scoring throughput (figure 8.1). The phase profiles place most instrumented cycles in bucket accumulation. Biased list-local patterns create repeated bucket destinations; private histograms avoid atomics at the cost of larger state and shared-memory read–modify–write work. Reducing group width and avoiding dominant buckets improves particular variants, but none reaches the baseline. This supports a stopping decision for these implementations and inputs. It does not establish that all finite-alphabet reuse is ineffective: the winning two-table baseline already reuses coordinate sums in a different direction.

The pilot writes primary scores rather than returning final neighbours. Its FP32/FP64 comparisons expose near-cutoff numerical differences, not final-pipeline equivalence. With measured scorer speed ratio s and an estimated replaceable latency fraction f, the optimistic replacement model is $1/[(1-f)+f/s]$. Its predicted slowdown is a model, since integration was stopped before an end-to-end run. An auxiliary lossless dictionary study also stopped at structural statistics: its limited byte/work savings did not justify a GPU implementation. Neither route contributes a measured system point to the main frontiers.

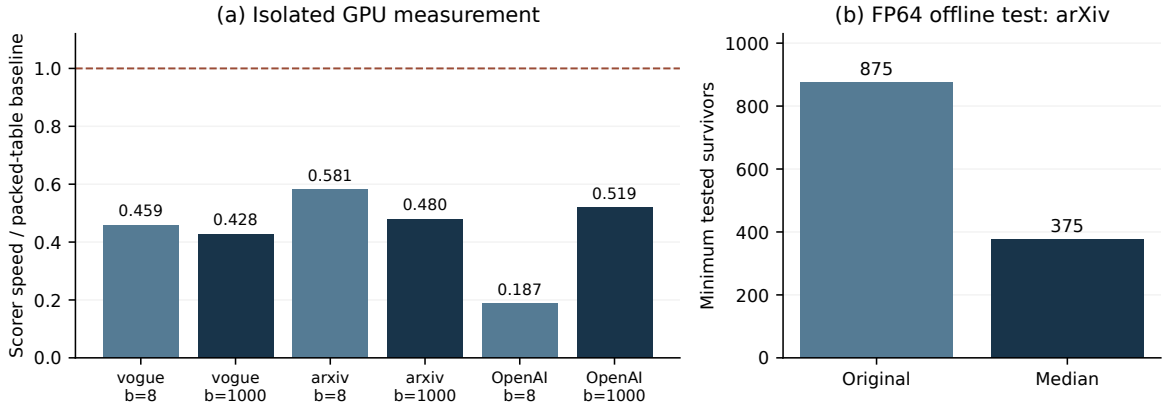


Figure 8.1. Two distinct evidence boundaries. Left: best measured Mailman scorer / packed-table scorer speed on an RTX 5090, with Vogue, arXiv and OpenAI-3072 at two batches and 128 probes; all ratios are below one. Right: minimum survivor counts meeting the original JHQ recall target on arXiv's 400-query FP64 offline test split. The candidate reduction is not a GPU speedup. Both panels are regenerated from archived records by the thesis audit script.

8.2.2 Rank-oriented coding: a local gain is not a general replacement

The rank-oriented study asks whether changing one-bit thresholds or scoring can retain useful neighbours with fewer survivors at the same nominal code budget. Its main comparisons replay exported candidates in FP64, with 400 calibration, 200 validation and 400 final-test queries. It examines likelihood weighting, median thresholds, learned ranking losses, linear-MMSE decoding and list-dependent thresholds. Different objectives expose different failure modes: reducing reconstruction or inner-product MSE can worsen the order of the few nearest candidates; changing primary levels also changes the residual reference point.

On arXiv's test split, median thresholds reduce the minimum tested survivor count meeting the original target from 875 to 375. That success does not generalise to the other two workloads under the same strict criterion. The shared-rotation Gaussian intuition also leaves real corpus bias intact: rotating a mean vector preserves its norm. Centered coding can address some of this mismatch, but its reconstructed-score ceiling must be evaluated alongside primary ranking. The stopped study therefore supports a workload-specific representation improvement, rather than a new generally rank-optimal quantiser.

The later median-only GPU follow-up is separate evidence. It imports the changed thresholds, levels and retrained residuals into the same search pipeline, then compares independently swept budgets at measured recall thresholds. At arXiv's original $\alpha = 100$ target it yields ratios of approximately 1.40 at batch 8 and 1.21 at batch 1,000. Vogue's tested median frontier stops at 0.9630, below the original 0.9644 working point; OpenAI-3072 is essentially tied. These results revise a blanket "never GPU-tested" description, while preserving the decision against a universal replacement. The follow-up uses its own index states, query pool and three-round protocol; its rates are not inserted into E1.

8.3 Threats to Internal and External Validity

Table 8.1 consolidates threats to the main interpretations. The shared-input, timing and provenance controls are defined in Chapter 3; section 7.8 evaluates repeated execution, fixed-point rebuilding and precision sensitivity within their recorded boundaries.

Table 8.1. Threats that can reverse or narrow a conclusion.

Claim at risk	Alternative or reversal	Control and remaining test
Execution gain	Changed scores, ties or index state explain an apparent speedup	Strict T4 CPU sorting and table-form diagnostics; finite coverage, with candidate dumps no longer retained
Budget quality	Finite-pool sampling or drift hides damaging queries	Disjoint GT evaluation, full traces, subset-sum checks and FULL enumeration; future-query and arbitrary-subset invariance remain untested
Competitive margin	Grid density, call tails or API overhead changes the ordering	Both recall estimators, retained slow calls and int8 baseline; TF32/FP32 sensitivity; T5 exposes small-batch reversals; cross-device comparisons remain unmeasured
Capacity/build claim	Transient allocations exceed a resident sample	Phase bytes, repeated empty-cache builds and 20 ms device peaks; sub-interval allocations and full retrained frontiers remain unmeasured

External validity is limited by one GPU, squared-L2 top-10 and six prepared embedding datasets. The batch-dependent operating regions are reported in section 7.2.5. Query removal by position does not certify the absence of near-duplicate embeddings, and the exports are not a relevance-labelled retrieval benchmark. Database updates, concurrent traffic, deployment latency objectives and changing query populations remain untested. The resident-index results supply no billion-scale or multi-device capacity evidence. These are the remaining checks before transferring an observed region to deployment.

8.4 Testable Research Directions

8.4.1 Risk-controlled agreement with a reuse-aware objective

A next policy could minimise calibration plus expected reuse time while certifying a query-level *reference-agreement* loss, using valid selection-aware confidence bounds. The inputs would be matched-regime per-query disagreement curves and a fixed reference; variables would include sample size and survivor budget. An absolute GT target additionally needs independent labelled validation or assumptions linking the reference to GT. Stationarity, query-level sampling and a common score/order contract must be explicit; a drift trigger would invalidate and renew the certificate.

The first experiment should compare the existing fixed-sample rules, rank-curve reuse and a cost-aware policy on held-out pools, recording total time and tail agreement loss. The archived sequential controller already failed to beat fixed sampling in an older four-cell replay because extra search launches consumed sample savings. Repeating sequential testing alone is therefore insufficient. The new question is whether reusable intermediate information and a quality certificate jointly improve *total* cost. Failure means violating the stated risk limit or saving no time against the quality-matched fixed-sample control.

8.4.2 Capacity-aware residual placement

The resident residual array dominates large-index storage. A complementary study could keep primary codes resident and cache residual blocks under an explicit device-memory cap, using

host memory for misses. Holding codes, routing, survivor sets and refined arithmetic fixed would isolate placement from representation quality. The minimum experiment measures cache hits, transfer bytes, overlap, tail latency and throughput while varying memory cap, batch and query locality. A result is useful only if it expands feasible capacity with an acceptable total-time frontier; a slower cache at an already feasible resident size is not sufficient. This is an unimplemented storage extension, distinct from the existing residual-bound pilot and from lowering α, which leaves the full array allocated.

8.5 Conditional Deployment Recommendation

JHQ-GPU merits evaluation when a compressed resident hierarchy fits the device and the required recall lies within its measured competitive region. E1 supports high-recall throughput at batch 1,000; T5 instead favours CAGRA fp32 at recall 0.97 and batch 8 on all four datasets. At batch 32, OpenAI-3072 is close under interpolation and favourable to JHQ under the measured threshold; the other three favour CAGRA. Lower-recall JHQ advantages also depend on dataset and grid. Grouped scheduling helps JHQ internally without guaranteeing a cross-system lead (section 7.2.5).

Start calibration from the recorded FULL working point, then validate quality and direct reuse cost on deployment queries. Compare total time and resident memory under the actual batch regime, subject to the external-validity limits in section 8.3.

Chapter 9

Conclusion

JHQ-GPU demonstrates how a structured quantised hierarchy can be translated into a useful GPU search pipeline, and why its execution and refinement effort must be evaluated together. The four research questions now have concrete, bounded answers.

9.1 Answers to the Four Research Questions

RQ1: Construction and capacity. GPU training, batched encoding, IVF placement and packed arrays produce a searchable resident hierarchy. The largest tested database contains 17.78 million vectors at 20.44 GiB resident device memory; the corresponding supplementary train-plus-add range is 10.54–10.65 s (section 7.7). Its sampled construction peak is 22.58 GiB, while BGE reaches 26.08 GiB at add entry. Repeated independent builds and startup intervals establish the measured capacity boundary more fully.

RQ2: Correct and efficient execution. The inherited Cartesian code admits exact real-arithmetic table factorisation, and complete-key retention/merge and shared-bound arguments explain the grouped path's selection conditions. Two-group tables lead in 22/24 measured cells. Integrated scheduling achieves 2.727× geometric-mean throughput at batch 8 but only 1.015× at batch 1,000 against the lane-refinement reference (sections 7.3 and 7.6). Thus useful parallelism depends on batch and work distribution. T4's strict CPU reference matches all 55,296 grouped instances in complete-key order; 56 one-block set differences are exclusively cutoff ties (section 7.4.1).

RQ3: Refinement budget and quality. Unlabelled reference agreement selects workload-level survivor effort without a learned predictor. FULL calibration meets the tested relative recall-loss target in 12/12 cells, with maximum observed loss 0.0006. At $S = 128$, one cell instead has 40/64 threshold crossings (section 7.5). The reference-based analysis explains conditional budget search, not a population GT guarantee. Ten FULL cells still lack a general complete-key nestedness guarantee; nevertheless all twelve FULL finite-grid choices match enumeration. Direct calibration-plus-reuse measurements on four cells support savings on suitable workloads and retain the quality/time tradeoff on difficult ones.

RQ4: Competitive operating regions. At recall 0.97, the interpolated full-batch E1 comparison places JHQ first in three datasets and near parity with IVF-RaBitQ on arXiv (section 7.2). Vogue's lead depends on the grouped path. The OpenAI E1 runs use TF32. Removing it under the cached-model boundary retains numerical advantages over archived baselines in D, a cross-session comparison. Separate T1 anchors retain this direction with contemporaneous baselines and grid-sensitive margins (section B.4.1); neither intervention fully retrains FP32 models. CAGRA int8 wins in lower-recall regions, and IVF-RaBitQ wins at some higher targets. T5 narrows that region: CAGRA fp32 leads on all four datasets at recall 0.97 and batch 8; batch-32 borderlines depend on the recall estimator (section 7.2.5). Competitive rankings therefore depend on batch, quality and grid, alongside separately reported memory use. Cross-device rankings remain untested.

9.2 Closing Principle and Outlook

The resulting principle is to co-evaluate representation, execution and effort under a stated quality objective. A smaller table can add work; more groups can duplicate it; less refinement can sacrifice the neighbours that matter. The negative designs show why mathematical reuse and lower reconstruction error need not improve the search frontier. The next step is to test whether reusable calibration information can support valid risk control at lower total cost, and whether explicit residual placement can extend capacity without unacceptable tail latency (section 8.4). These are hypotheses grounded in the measured limits, rather than capabilities already supplied by the present system.

Bibliography

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[5] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3(4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[6] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[7] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[8] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[9] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[10] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[11] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[12] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL `https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html`.

[13] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*,

pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf.

[14] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL https://proceedings.mlr.press/v119/guo20h.html.

[15] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[16] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL https://arxiv.org/abs/1702.08734.

[18] Karthik V., Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbor search using a single GPU, 2025. URL https://arxiv.org/abs/2401.11324v4. Version 4.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL https://www.cs.yale.edu/publications/techreports/tr1402.pdf.

[21] Rui Luo and Mengxuan Zhang. JHQ-GPU: Representation-driven GPU execution and workload-adaptive hierarchical refinement, 2026. Project manuscript artifact; its historical measurement versions are distinguished in this thesis.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html. Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-

execution/index.html. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html. PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M. Accessed 28 October 2026.

[30] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-3072-1M. Accessed 28 October 2026.

[31] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[32] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL https://huggingface.co/datasets/macrocosm/arxiv_abstracts. Accessed 28 October 2026; exported embedding preparation is project-specific.

[33] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding. Corpus split; accessed 28 October 2026.

[34] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3. Italian configuration; accessed 28 October 2026.

[35] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html.

[36] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf.

Appendix A

Reproducibility and Experiment Manifest

This appendix records the contracts needed to interpret the experiments. The expanded protocol, executable names, raw-source paths and result-to-figure mapping are retained in `docs/experiment-protocol.md` in the report repository.

A.1 Source, Environment and Trained States

The principal environment is RTX 5090, driver 595.71.05, CUDA 13.0 and, for E1, cuVS 26.08.01. JHQ targets compute_80 PTX and uses driver JIT compilation. Table A.1 separates run-header evidence from versions stated in reports. The analysis checkout in `data/ch7/audit.json` identifies the inspected workspace; a run-head commit identifies source visible to that run. Neither supplies a missing executable or trained-state fingerprint.

Dataset shapes and actual query counts are in table 3.2. Comparisons reuse the exported vectors and squared-L2 ground truth per dataset. Search sweeps load the recorded `arms6_cache` states; cold-build records start without trained state. Input ledgers hash the consumed analysis records, rather than every vector file, cache or binary. Initial large-baseline attempts that retained an old index are superseded by isolated reruns. Stella's skipped batch-1,000 files contribute no search points; corrected large-search records use its 999 queries.

A.2 Measurement Contracts

E1 reports the median of five round throughputs, each computed as total queries divided by elapsed host-to-host time. A round normally uses 24 calls; a warm-up threshold reduces this count at 274 points. The Python decision interval includes first-call conversion and validation, so it differs from steady-state latency. Resource reuse, preallocated outputs, pinned input staging and timed int8 query quantisation improve comparability; output-memory types and Python/C++ boundaries still differ. Device-only baseline timing is auxiliary to this common call boundary.

The 369 baseline repeat-call identifier flags include tie-order changes and do not alone exclude points. The earlier count of 401 additionally includes cross-protocol disagreements; the exact union and method counts are in the expanded protocol. JHQ does not populate the same flag. Cold-build train/add timers exclude object construction, host QR and constructor library

Table A.1. Recorded component provenance. Report and script identities are distinguished from raw run headers; full executable and input paths are in the expanded protocol.

Component	Recorded source evidence
E1 final pipeline	Environment logs include `a31c22d`; extensions retain separate metadata
Lane refinement (v61)	Raw header `b37efa4`
Grouped scan (v62)	Report `862543a`; run script `e0d0518`
Shared bound (v63)	Source and build flags archived; no inferred run head
Batch scheduling (v65)	Raw header `d425ab8`
Radix probe (v69)	Comparative runs and source archived; binary SHA unrecorded
Budget / table sweep	Raw header `d5e1526`
Large int8 rerun	Stella degree-64 metadata `ad433a4`
Historical budget	Raw header `bf31b41`, v57 family

warm-up. Construction diagnostics sample device used memory, including context, every 20 ms; synchronised one-buffer phase runs remain separate from normal pipeline measurements.

A.3 Budget Harness Contract

The campaign explicitly sets the eight-point grid beginning at 200, overriding a fallback string beginning at 100. Even query indices form the 500-query calibration pool; odd indices form the 500-query held-out pool, or 499 for Stella. Each $S \in \{32, 64, 128\}$ has 64 seeded draws without replacement. The initial warm-up traverses all queries, while its outputs and GT remain outside the agreement decision. Scan-schedule calibration uses separate database pseudoqueries.

Each fixed arm receives one warm-up and three timed calls. Calibration times reference/candidate searches and the $O(Sk^2)$ host agreement comparison with graph capture bypassed; prior $O(Sd)$ sample gathering and separate scan calibration are excluded. The logged p95 is sorted index $\lfloor 0.95(64-1) \rfloor$. The helper returns a grid index and leaves the last tested multiplier installed: callers must apply `set_alpha(A[index])`. Selected-arm rates in the original campaign reuse already timed fixed arms.

Original enumeration checks cover 2,304 subsampled decisions, excluding the twelve FULL calls. Supplementary enumeration also matches all twelve FULL decisions. At FULL size, two Vogue cells use four groups and ten cells use distance-only selection; common cutoff ordering remains the condition for a general binary-search guarantee. Finite-pool prediction models uniform subsets and requires per-query counts invariant to subset membership and position under the relevant execution regime. All 18,432 sampled grid counts equal their recorded full-pool subset sums. Mapping acceptance to GT-loss crossings additionally uses the held-out fixed-arm quality labels. These checks establish the recorded cases, rather than unknown query-population quality.

A.4 Supplementary Evidence

Records under `JHQ_GPU/experiments/thesis_evidence/` retain campaign identities and timing boundaries. A1 separates first-read, normal warm-input and synchronised phase runs on six datasets; five-build recall is measured at fixed 128 probes. C1 checks four complete database layouts and sampled re-encodings. Its FP64 norm checks use the stored rotation matrix, rather than actual GPU rotation outputs. A3 checks every OpenAI query row and 10,299 strided base rows; original entity-ID mappings are absent.

A2 extracts the recorded executable's sm_80 PTX and reassembles it with CUDA 13.0.88 `ptxas` for sm_120. The observed 64-bit bound read and atomic-min publication use an aligned query-key array with eight-byte stride. The argument uses PTX alignment, scope and proxy semantics; toolkit SASS corroborates it without supplying the driver's loaded JIT code. Binary SHA and operation listings are in the expanded protocol and the archived evidence.

D alternates v69/v72 over the E1 OpenAI grid, reusing TF32-trained states and changing add/query GEMMs. Its baseline denominators remain archived E1. The later contemporaneous T1 anchors are reported separately in section B.4.1. Neither experiment constitutes FP32 retraining.

B reproduces the original `srand(12345)` decisions and adds per-query curves, FULL enumeration and direct reuse traces. Direct timing uses a separate seeded sequence, five rotated-policy rounds and 500 repetitions of the held-out batch. It includes gathering, calibration, budget

installation and first-batch graph setup; construction, schedule calibration and warm-up remain outside that interval. Schedule calibration startup costs are separately reported.

A.5 Strict Selection, Small Batches and Artifact Access

T4's v73 source matches v71 after removing its two debug-export blocks. CPU sorting consumes GPU-produced primary and retained refined scores. The exported-score equality check covers returned valid primary entries; the padding counter checks valid-entry counts. Natural short pools are covered, while deliberately empty or adversarial lists remain separate tests. Table-form comparisons align candidate positions at 128 probes, $\alpha = 100$, 256 queries, shared bounds enabled, $g_{\text{scan}} \in \{0, 2\}$ and $g_{\text{LUT}} \in \{1, 2, 4, 8\}$. Temporary candidate dumps were deleted: retained JSONL/stdout and LUT summaries support the reported finite-case counts. The same-model Vogue cross-process discrepancy remains unexplained; within-run T4 selection checks address a different question.

T5 partitions queries into fixed small batches and includes staging, transfers, applicable int8 query quantisation and synchronisation. Each point is the median of five round throughputs; null-call costs remain included. JHQ/CAGRA reuse E1 grids and RaBitQ has its own recorded sweep. Numeric JSON supplies recall thresholds. Interpolated estimates retain qualifying endpoints below the minimum frontier recall; strict measured thresholds remain separate. Python/C++ interfaces and warm-up policies differ, and these measurements do not establish concurrent-service latency.

Source, records, manifests and offline checkers are archived in `evidence/ch5/v71/` and `evidence/review/`. With the required inputs, run `make ch7-data`, then `make pdf` or `make pdf-zh`. GPU reruns additionally require original vectors, trained states, matching builds and environment.

Appendix B

Supplementary Results and Correctness Checks

This appendix records parameter coverage, stage-specific validation and secondary outcomes supporting Chapters 3–8. Measurement families retain their own versions and timing boundaries. Their source mapping is in Appendix A.

B.1 Complete Parameter Registry

The E1 comparison contains 1,590 valid measured configurations, with coverage in table B.1. Each point's complete parameter JSON, recall, host-to-host QPS and source filename is retained in `data/appendix/e1-configurations.csv`. The registry is generated by `scripts/build_appendix.py`, with source identity recorded in `data/appendix/audit.json`. The sets below describe observed values; their Cartesian products are not assumed to have been measured.

Table B.1. Exact E1 configuration counts, including coverage extensions. Counts are parameter points, not independent index builds.

Family	Vogue	arXiv	OpenAI-1536	OpenAI-3072	Total
JHQ-GPU v69	27	29	37	39	132
CAGRA fp32	64	64	64	64	256
CAGRA int8	64	64	64	64	256
IVF-RaBitQ	132	132	264	198	726
IVF-PQ	54	54	54	54	216
GPU brute force	1	1	1	1	4
Total	342	344	484	420	1590

JHQ-GPU. All E1 points use $B = B_r = 8$, $M = d/8$, $k = 10$ and batch 1,000. Vogue uses 4,096 IVF lists; the other three use 8,192. Vogue observes probes 16/32/64/96/128/192/256/384/512 and $\alpha \in \{16, 32, 64, 128\}$ over 27 tuples. ArXiv adds probe 1,024 and uses $\alpha \in \{25, 50, 100, 200\}$ over 29 tuples. Both OpenAI grids use probes 32/64/128/256/512/1,024; their observed multipliers are respectively $\{4, 8, 16, 32, 64, 128, 256, 512\}$ and $\{2, 4, 8, 16, 32, 64, 128, 256\}$, over 37 and 39 tuples. The reported scheduled path uses four groups on Vogue and one block elsewhere. OpenAI uses TF32 rotation/centroid GEMMs in historical v69; the v72 FP32 sensitivity is a separate family (section 7.2.3).

CAGRA fp32 and int8. Each representation has 64 points per dataset: graph degrees 32/64, intermediate degree 128, `ivf_pq` construction, `itopk_size` in 32/64/96/128/192/256/384/512 and search width 1/2/4/8. Int8 records quantile 0.99 for dataset scaling and includes query quantisation in the host search boundary.

IVF-PQ. Eight-bit subcodes use 192/384/768-byte codes at dimension 768, and 384/768/1,536-byte codes on OpenAI. Each length has fp16/fp32 lookup tables and probes 16/32/64/96/128/192/256/384/512: 54 points per dataset. The recorded refinement ratio is one and list counts match JHQ. Code lengths and memory budgets differ.

IVF-RaBitQ and brute force. RaBitQ records 4/8/9 bits per dimension and probes 16/32/64/96/128/192/256/384/512/768/1,024. The 768-dimensional and OpenAI-1536 records contain LUT16/LUT32/QUANT4/QUANT8 modes; OpenAI-3072 contains LUT32/QUANT4/QUANT8. Additional 4,096-list OpenAI indexes extend the default 8,192-list coverage. Exact combinations are in the registry. Each dataset also has one FP32 squared-L2 brute-force point. An absent mode or higher-recall point is not a measured failure.

Auxiliary grids. LUT evaluation fixes $\alpha = 100$ and probes 8/32/128/512 on six datasets with $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$: 24 cells and 96 timed arms. Budget evaluation uses probes 128/512 and $A = (200, 100, 64, 32, 16, 8, 4, 2)$ on the same six datasets, with 64 draws at each $S \in \{32, 64, 128\}$ and $\epsilon = 1$. FULL separately uses all 500 calibration queries. Held-out counts are 500, or 499 for Stella. Direct reuse tests four cells, four policies and five rounds of 500 repeated held-out batches. Boundaries are specified in sections A.3 and A.4.

B.2 Correctness and Numerical Evidence

Table B.2 separates numerical, algorithmic and memory evidence. Chapter 5's ordering arguments concern computed scores under their stated contracts; raw-vector recall is evaluated separately.

Table B.2. Validation coverage and the object actually checked.

Boundary	Completed evidence and scope
Representation	Four all-row offset/ID audits; sampled raw-input FP64 recoding and stored-matrix geometry; TF32 deviations identified
Cartesian LUTs	Real-arithmetic factorisation; 96 timed arms; T4 per-candidate FP32 error and ranking diagnostics across four table forms
Global selection	T4: 270 configurations, 69,120 query instances; 55,296 grouped instances match strict CPU complete-key sorting in order
Shared bound	48 paired checks; extracted scalar 64-bit PTX and toolkit SASS support whole-key access; actual driver SASS not captured
Scheduling	250 variant/configuration and 234,320 query comparisons, retaining distinctions between ID sets and tie order
Probe selector	1,792 synthetic cases with zero failures; real-query ID/offset checks, relative to computed routing scores
Budget search	2,304 monotone sampled curves and 12 FULL enumeration matches; 18,432 sampled grid counts reproduce full-pool subset sums

The smaller representation-audit databases have identical primary and residual bytes across one-/two-pass builds when joined by ID. Corrections differ by at most 8.94×10^{-8} and 1/1/40 rows change list assignment; Stella validates its natural two-pass path (section 4.6.2). T4 now supplies the strict computed-score selection reference (section A.5). The controlled same-model Vogue cross-process diagnosis remains unperformed: archived processes with matching probe/primary dumps still report final recall 0.9642/0.9644. Within-run selection checks do not localise that separate anomaly.

B.3 Secondary Results and Evidence Status

Adverse regimes. The scheduler's largest recorded regret is 1.217× at batch 256. Scheduled/one-block throughput on Vogue spans 0.895–1.175 in E1: the skew rule enabling its recall-0.97 lead also adds overhead in some low-probe cells (section 7.6). These adverse results accompany the favourable internal scheduling findings. In T5's cross-system comparison,

CAGRA fp32 leads on all four datasets at recall 0.97 and batch 8; batch-32 near ties depend on the recall estimator (section 7.2.5).

Representation alternatives. Mailman-style GPU scorers are isolated kernel measurements; their end-to-end replacements are optimistic models. The rank-oriented study is an FP64 offline replay. A separate median-code GPU follow-up improves arXiv at the recorded target by 1.402× at batch 8 and 1.208× at batch 1,000, misses Vogue's strict target and is nearly tied on OpenAI-3072. Thresholds, levels and residual training change together, preventing a threshold-only attribution (section 8.2).

Calibration and precision. Finite-pool prediction explains sampling risk conditional on stable query counts; its GT-loss mapping uses held-out fixed-arm quality. Direct reuse measures cumulative cost on four repeated pools. The precision intervention changes add/query GEMMs on one TF32-trained cache. Full FP32 retraining, future-query drift, cross-device rankings and online updates are outside these experiments.

B.4 Construction Capacities and Precision Anchors

Table B.3 separates input staging from independently allocated assignment products. Source defaults are three buffers, 65,536 configured add rows and 128 MiB per input/rotated float array. These are allocation capacities, not current sub-batch sizes or total process peaks.

Table B.3. Default construction-buffer capacities. The last column is $4n_b n_{\text{list}} a_{\text{assign}}/2^{30}$; substituting a_{in} would give an incorrect general model.

d	n_{list}	a_{in}	a_{assign}	Dots GiB
768	4,096	43,690	8,192	0.375
1,024	32,768	32,768	32,768	12.000
1,536	8,192	21,845	32,768	3.000
3,072	8,192	10,922	32,768	3.000

B.4.1 Contemporaneous precision anchors

T1 is a later campaign than D: it remeasures JHQ and competing baselines near recall 0.97 in one session, with the E1 workspace pools enabled. Each JHQ configuration has two repetitions bracketing the baselines; each repetition uses five timing rounds. Per-configuration QPS combines the two round medians geometrically. There are twelve paired configurations per OpenAI dataset. The FP32/TF32 geometric means are 0.956/0.964; both variants reuse a TF32-trained cache and change add/query arithmetic.

Table B.4. T1 at recall 0.97 and batch 1,000. Baseline columns report interpolation / strict measured threshold; the FP32/TF32 column is interpolated. All methods here are contemporaneous; these values are separate from D's cross-session baseline ratios.

Dataset	FP32/TF32 at 0.97	FP32 / RaBitQ	FP32 / CAGRA
OpenAI-1536	0.980	1.126 / 1.053	1.752 / 1.633
OpenAI-3072	0.973	1.328 / 1.316	1.825 / 2.019

Both estimators retain a numerical FP32 advantage in this finite grid. The OpenAI-3072 RaBitQ bracket spans recall 0.9669–0.9796, exposing substantial interpolation sensitivity. These

anchor sweeps do not replace E1's full grid, establish a universal margin, or constitute FP32 retraining. The raw JSON and reconstruction are mapped in `data/full-review/audit.json`.

B.5 Artifact Access

The report repository contains the parameter registry, derived tables, figure sources, analysis scripts and 209-file v71 source/measurement bundle under `evidence/ch5/v71/`. The offline command `python3 evidence/ch5/v71/check.py` verifies that bundle. Selected T4/T5, T1 and A2 source/record inputs are also mirrored in `evidence/review/`. Its checker verifies hashes and can rerun the two corresponding offline audits using that local source root. The broader raw records remain in the sibling `JHQ_GPU` project at the ledger paths. Complete vectors, trained states and all executables are not packaged here. Offline recomputation requires the stated inputs; GPU reproduction also requires the corresponding build and environment.